CRASH COURSE PLAN — v3 (đã audit theo docs Laravel 13.x)

Laravel Crash Course — REST API

> Lộ trình học chi tiết từng phần, kèm giải thích cơ chế bên trong — và lộ trình để master Laravel

Repo: crash-course / laravel-crash-course

Bám theo Laravel 13.x (phát hành 17/03/2026, PHP 8.3–8.5) — mọi API trong tài liệu đã đối chiếu với laravel.com/docs (tháng 09/2026)

Kiểm tra phiên bản dự án của bạn bằng `php artisan --version`

Mục lục

PHP PHP 8.3+ cho người viết Laravel — NỀN TẢNG NGÔN NGỮ

00 Kiến trúc tổng thể & Request Lifecycle — NỀN TẢNG

01 Khởi tạo dự án & cấu trúc thư mục

SAIL Chạy Laravel bằng Docker với Sail (tuỳ chọn) — MỚI

02 Routing & Controllers cho API

03 Service Container, DI & Service Providers

SQL SQL nền tảng — học trước Part 04 — MỚI

04 Database: Migration, Model (Eloquent), Factory & Seeder

05 Validation với Form Request

06 API Resources — tầng transform response

07 Middleware & Exception Handling (chuẩn hoá lỗi JSON)

08 Quan hệ dữ liệu (Relationships) & vấn đề N+1

09 Authentication với Laravel Sanctum

10 Chuẩn hoá REST API cho production

SEC Bảo mật ứng dụng API — tư duy & kiểm tra — MỚI

11 Testing (Pest / PHPUnit)

12 Pagination chuyên sâu & xử lý dữ liệu lớn — MỚI

13 Database Transactions — MỚI

14 Queue & Jobs — chuyên sâu — MỚI

15 Events, Listeners & Observers — MỚI

16 Cache & Atomic Locks — MỚI

17 Task Scheduling (cron) — MỚI

18 File Storage & Upload — MỚI

19 Authorization chuyên sâu: Gate & Policy — MỚI

20 Mail & Notifications (queued) — MỚI

21 Kiến trúc dự án lớn: Service, Action, DTO, Enum — MỚI

22 Production: deploy, worker, Octane, quan sát hệ thống — MỚI

23 Lộ trình master Laravel — làm gì tiếp theo — MỚI

Cách dùng tài liệu này: Bạn tự code 100% từng phần. Mỗi phần có mục "Việc cần làm" (task) — không có code giải sẵn để copy, chỉ có ví dụ cú pháp minh hoạ để hiểu cách viết. Khi xong một phần, quay lại chat để mình review và giải thích tiếp phần sau.

Bản v2 khác gì bản gốc: (1) mọi API/cú pháp đã đối chiếu lại với docs chính thức Laravel 13.x — các khung màu xanh như khung này đánh dấu chỗ đã sửa hoặc bổ sung sau audit; (2) thêm 12 phần mới (Part 12–23) đi sâu vào những thứ phân biệt "biết Laravel" với "master Laravel": pagination cho dữ liệu lớn, transaction, queue/job, event, cache, scheduling, storage, authorization, notification, kiến trúc dự án lớn, vận hành production; (3) phần cuối là lộ trình tự học tiếp để master.

Bản v3 bổ sung (sau khi rà các chỗ còn hở): (1) **PART SQL** — SQL nền tảng (JOIN, GROUP BY, index, EXPLAIN, transaction) đặt ngay trước Part 04 vì không hiểu SQL thì Eloquent và N+1 chỉ là học thuộc; (2) **PART SAIL** — chạy Laravel bằng Docker (tuỳ chọn, hữu ích khi tới Part 14/20/22); (3) **PART SEC** — gom tư duy bảo mật vào một chỗ, kèm bài kiểm tra bằng test; (4) mục **1.6 Git cho dự án** trong Part 01; (5) ghi rõ **phạm vi**: tài liệu này chỉ làm API, giao diện (Blade, Livewire, Inertia, Vite, starter kit) nằm ngoài phạm vi. Các khối mới đã đối chiếu docs Laravel 13.x (Sail, installation, database, queries, csrf, releases) và **mọi câu SQL, kết quả, kế hoạch EXPLAIN trong Part SQL đã chạy thật** trên SQLite. Bài tập có đáp án đi kèm nằm ở file `php-bai-tap.pdf` : Chương 1–11 luyện PHP, **Chương 12 luyện SQL**.

Phạm vi của tài liệu: chỉ xây **REST API** (không làm giao diện). Nếu bạn muốn làm cả giao diện, Laravel có Blade, Livewire, Inertia, Vite và các starter kit chính thức; xem mục "Next Steps" trong trang Installation của docs và Part 23.2. Mọi thứ trong Part 00–22 (routing, Eloquent, queue, cache, test...) vẫn dùng nguyên cho cả hai kiểu dự án.

Dự án xuyên suốt: Xây một API quản lý bài viết — Post (tiêu đề, nội dung), Comment (bình luận thuộc về một Post), User (đăng ký / đăng nhập bằng token). Đây đúng là bài toán bạn đã làm bên NestJS, nên xuyên suốt tài liệu sẽ có các bảng đối chiếu NestJS ↔ Laravel để bạn thấy cùng một khái niệm được hai framework giải quyết thế nào. Các Part mới mở rộng chính dự án này: đăng bài xong thì đẩy job xử lý nền, bắn event, gửi notification, cache danh sách, dọn dẹp theo lịch.

PART PHP — NỀN TẢNG NGÔN NGỮ

PHP 8.3+ cho người viết Laravel

Mục tiêu: Nắm đủ PHP hiện đại để đọc/viết Laravel không bị "ngợp cú pháp". Phần này bắt buộc nếu bạn mới sang từ JS/TS — vì 90% khó khăn khi học Laravel thực chất là khó khăn với PHP, không phải với framework.

P.1 Mô hình thực thi: PHP khác Node.js ở chỗ căn bản nhất

Đặc điểm	Node.js / NestJS	PHP / Laravel (PHP-FPM)
Tiến trình	1 process sống mãi, xử lý mọi request	Mỗi request là một lần chạy script từ đầu, xong thì xoá sạch bộ nhớ

Bất đồng bộ	Event loop, mọi I/O đều async	Đồng bộ (blocking) theo mặc định — code chạy tuần tự từ trên xuống
Biến toàn cục / cache RAM	Giữ được giữa các request	Không giữ được — muốn cache phải dùng Redis/file/DB (Part 16)
Rò rỉ bộ nhớ	Nguy hiểm, tích tụ dần	Gần như không có — process chết sau mỗi request
Việc chạy nền / lâu	Promise, worker thread trong cùng process	Đẩy sang Queue worker — một process PHP riêng chạy mãi (Part 14)

Vì sao phải biết điều này: nó giải thích tại sao Laravel "singleton" chỉ sống trong 1 request (Part 03), tại sao Laravel không có async/await ở tầng controller, tại sao mọi việc nặng đều đẩy ra Queue (Part 14), và tại sao Laravel Octane (Part 22) là một thay đổi lớn về mô hình chứ không chỉ là "tăng tốc".

P.2 Cú pháp cốt lõi — đọc nhanh phần khác JS

```php
<?php
declare(strict_types=1);          // NÊN có ở đầu mọi file: bật kiểm tra kiểu nghiêm ngặt

$name = 'Nhat';                   // biến LUÔN bắt đầu bằng $
const MAX = 10;                   // hằng số (không có $)

// Nháy đơn vs nháy kép — khác biệt quan trọng
echo 'Xin chào $name';            // → Xin chào $name      (KHÔNG nội suy)
echo "Xin chào $name";            // → Xin chào Nhat       (CÓ nội suy)
echo "Tổng: {$order->total}";     // nội suy biểu thức phức tạp thì bọc {}

// Nối chuỗi bằng dấu CHẤM, không phải dấu +
echo 'Xin ' . 'chào';             // 'Xin chào'   ('a' + 'b' là LỖI trong PHP 8)

// So sánh: LUÔN dùng === (== có ép kiểu ngầm, nguồn bug kinh điển)
0 == '0';                         // true
0 === '0';                        // false  ← dùng cái này
```

P.3 Toán tử hiện đại (PHP 8) — Laravel dùng liên tục

Cú pháp	Ý nghĩa	Tương đương JS/TS
`$a ?? 'default'`	Null coalescing: nếu null/không tồn tại thì lấy default	`a ?? 'default'`
`$a ??= 5`	Gán nếu đang null	`a ??= 5`
`$user?->profile?->city`	Nullsafe: null thì dừng, không lỗi	`user?.profile?.city`
`$a <=> $b`	Spaceship: trả -1 / 0 / 1 — dùng cho sort	hàm compare
`...$items`	Spread / variadic	`...items`
`fn ($x) => $x * 2`	Arrow function (tự bắt biến ngoài)	`(x) => x * 2`

```php
// match (PHP 8) — thay thế switch, so sánh === và TRẢ VỀ giá trị
$label = match($status) {
    200, 201 => 'success',
    404      => 'not found',
    default  => 'unknown',        // thiếu default + không khớp → throw UnhandledMatchError
};

// match(true) — mẹo rất hay dùng trong Laravel
$tier = match(true) {
    $total > 1000 => 'vip',
    $total > 100  => 'normal',
    default       => 'new',
};
```

P.4 Array — chỉ có MỘT kiểu, gánh cả object của JS

```php
// Indexed array (như Array của JS)
$tags = ['php', 'laravel'];

// Associative array (như Object/Map của JS) — RẤT hay gặp trong Laravel
$post = ['title' => 'Hello', 'views' => 10];
echo $post['title'];                          // truy cập bằng [] chứ không phải .

foreach ($post as $key => $value) { /* ... */ }

// Các hàm array phải thuộc lòng
array_map(fn ($p) => $p['title'], $posts);       // map
array_filter($posts, fn ($p) => $p['views'] > 5); // filter (GIỮ NGUYÊN key!)
array_reduce($posts, fn ($c, $p) => $c + $p['views'], 0);
array_column($posts, 'title');                   // lấy 1 cột từ mảng 2 chiều
array_keys($post); array_values($post); count($post);
in_array('php', $tags, true);                    // tham số 3 = so sánh strict
```

Bẫy: `array_map` đảo thứ tự tham số so với `array_filter` (callback trước / mảng trước), và `array_filter` giữ nguyên key cũ nên mảng kết quả có thể "khuyết index" — khi trả JSON sẽ thành object thay vì array. Dùng `array_values(...)` để reset index. Trong Laravel bạn sẽ ít gặp chuyện này vì Collection (`collect($arr)->map()->filter()->values()`) xử lý gọn hơn nhiều — nhưng phải biết array thuần trước.

P.5 Function & type declaration

```php
// Khai kiểu đầy đủ — bắt buộc nếu muốn code Laravel chuyên nghiệp
function slugify(string $title, int $maxLength = 60): string
{
    return substr(strtolower(str_replace(' ', '-', $title)), 0, $maxLength);
}

function find(int $id): ?Post { }        // ?Post = Post hoặc null
function parse(string|int $v): array { } // union type
function log(string ...$lines): void { } // variadic: nhận n tham số

// Named arguments (PHP 8) — Laravel dùng nhiều, vd apiPrefix: 'api/v1'
slugify(title: 'Hello World', maxLength: 20);

// Closure vs arrow fn: closure PHẢI khai báo use() để lấy biến ngoài
$rate = 1.1;
$f1 = function ($x) use ($rate) { return $x * $rate; };  // tường minh
$f2 = fn ($x) => $x * $rate;                             // tự bắt, gọn hơn
```

P.6 OOP — phần Laravel dùng nhiều nhất

```php
namespace App\Services;                 // namespace ~ đường dẫn thư mục

use App\Models\Post;                    // import class (giống import của JS)

class PostService
{
    // Constructor property promotion (PHP 8): khai báo + gán trong 1 dòng
    public function __construct(
        private readonly Repository $repo,   // readonly: gán 1 lần, không sửa được
        protected int $perPage = 15,
    ) {}

    public function latest(): array
    {
        return $this->repo->all();          // $this->   (không phải this.)
    }

    public static function make(): static
    {
        return new static();                // self:: / static:: cho thành viên static
    }
}
```

Khái niệm	Ý nghĩa & nơi Laravel dùng
`interface`	Hợp đồng (chỉ khai method, không có thân). Laravel gọi là Contract — vd `Illuminate\Contracts\Auth\Authenticatable`, `ShouldQueue`
`abstract class`	Class không tạo instance trực tiếp, để kế thừa — vd `Model`, `FormRequest`

`trait`	Ghép code vào class (PHP không đa kế thừa). Laravel dùng khắp nơi: `HasApiTokens`, `SoftDeletes`, `HasFactory`, `Queueable`, `RefreshDatabase`
`enum` (PHP 8.1)	Tập giá trị cố định — rất nên dùng cho status. Cast được thẳng vào Eloquent (Part 04, 21)
`static` vs `self`	`static` = late static binding (theo class con), `self` = cố định class hiện tại
`readonly class` (PHP 8.2)	Mọi property đều readonly — lý tưởng cho DTO/Value Object (Part 21)

```php
// Trait — hiểu nó là hiểu "phép thuật" của Laravel
trait Sluggable
{
    public function slug(): string { return slugify($this->title); }
}

class Post extends Model
{
    use Sluggable, HasFactory;              // "dán" method của trait vào class này
}

// Enum backed — dùng cho trạng thái
enum PostStatus: string
{
    case Draft = 'draft';
    case Published = 'published';

    public function label(): string
    {
        return match($this) {
            self::Draft => 'Bản nháp',
            self::Published => 'Đã xuất bản',
        };
    }
}
```

P.7 Attribute (PHP 8) — cú pháp giống decorator của TS

```php
// Attribute là METADATA gắn lên class/method/property, đọc lại bằng Reflection
#[Fillable(['title', 'body'])]
#[Table('posts')]
class Post extends Model { }
```

Điểm khác decorator của TypeScript/NestJS: attribute của PHP hoàn toàn thụ động — nó không tự chạy code, chỉ ngồi đó cho tới khi có ai chủ động đọc nó bằng Reflection. Laravel 13 chuyển nhiều cấu hình sang attribute và bạn sẽ gặp cú pháp này liên tục trong tài liệu.

> **Đã đối chiếu docs — danh sách attribute Laravel 13 dùng trong tài liệu này:** Eloquent: `#[Table]`, `#[Fillable]`, `#[Unguarded]`, `#[RouteKey]`, `#[Scope]`, `#[ScopedBy]`, `#[ObservedBy]`, `#[UsePolicy]`, `#[WithoutTimestamps]`, `#[Connection]`. Controller: `#[Middleware]`, `#[WithoutMiddleware]`, `#[Authorize]`. Queue/Listener: `#[Tries]`, `#[Timeout]`, `#[Backoff]`, `#[MaxExceptions]`, `#[FailOnTimeout]`, `#[UniqueFor]`, `#[DebounceFor]`, `#[WithoutRelations]`, `#[Queue]`, `#[Connection]`, `#[Delay]`. Tất cả đều có trong docs chính thức 13.x; cú pháp cũ (`protected $fillable`, `public $tries`...) vẫn chạy song song.

P.8 Namespace, Composer & autoload PSR-4

```
// composer.json — phần quan trọng nhất
{
    "require":     { "php": "^8.3", "laravel/framework": "^13.0" },
    "require-dev": { "pestphp/pest": "^4.0" },
    "autoload":    { "psr-4": { "App\\": "app/" } }
}
```

PSR-4 là quy ước: namespace phải phản chiếu đúng đường dẫn thư mục. `App\Services\PostService` ⇒ file `app/Services/PostService.php`. Sai một chữ (kể cả sai hoa/thường) là "class not found". Không cần `require` file thủ công — `vendor/autoload.php` tự nạp khi bạn `use`.

Lệnh Composer	Tác dụng
`composer install`	Cài đúng version trong `composer.lock` (dùng khi deploy / clone repo)
`composer update`	Nâng version theo constraint + ghi lại lock (cẩn thận, đừng chạy khi deploy)
`composer require vendor/pkg`	Thêm package mới
`composer dump-autoload`	Nạp lại map autoload khi thêm class mà báo "not found"

P.9 Exception

```
try {
    $post = $service->publish($id);
} catch (ModelNotFoundException $e) {
    // bắt loại cụ thể trước
} catch (\Throwable $e) {
    // Throwable = cha của MỌI lỗi (Exception + Error). Bắt cuối cùng.
    report($e);                      // helper Laravel: ghi log rồi tiếp tục
} finally {
    // luôn chạy
}

throw new \InvalidArgumentException('ID phải > 0');
```

Trong Laravel bạn sẽ rất ít viết try/catch. Triết lý là: cứ throw, để Exception Handler tập trung lo việc biến nó thành response (Part 07). Try/catch chỉ dùng khi bạn thực sự xử lý được lỗi tại chỗ đó (ví dụ rollback transaction thủ công — Part 13).

P.10 Bảng tra nhanh PHP ↔ TypeScript

TypeScript / NestJS	PHP / Laravel
`const x = 1`	`$x = 1;` (biến), `const X = 1;` (hằng)
`this.foo`	`$this->foo`
`obj.prop`	`$obj->prop` (object) / `$arr['key']` (array)
`Class.staticMethod()`	`Class::staticMethod()`
`` `text ${v}` ``	`"text $v"` hoặc `'text ' . $v`
`arr.map(f)`	`array_map($f, $arr)` / `collect($arr)->map($f)`
`interface` + `implements`	Giống hệt (PHP có `interface`, `implements`)
Mixin / kế thừa nhiều	`trait`
`@Decorator()`	`#[Attribute]` (thụ động, cần Reflection đọc)
`async / await`	Không có ở tầng app — code chạy đồng bộ; việc nền dùng Queue
`npm / package.json`	`composer / composer.json`
eslint + prettier	Laravel Pint (chuẩn PSR-12) — chạy `./vendor/bin/pint`

Việc cần làm (làm bằng PHP thuần, chưa cần Laravel):

1. Tạo thư mục `php-warmup/`, viết file `index.php` có `declare(strict_types=1)`, chạy bằng `php index.php`
2. Viết hàm `slugify(string $title): string` có khai báo kiểu đầy đủ; thử truyền số vào để xem `strict_types` báo lỗi thế nào
3. Tạo mảng 5 post (associative array), dùng `array_filter` + `array_map` + `array_values` để lấy tiêu đề các post có views > 10, in ra bằng `json_encode`
4. Viết class `Post` + interface `Publishable` + trait `Sluggable` + enum `PostStatus: string` — dùng constructor property promotion và `readonly`
5. Tạo `composer.json` với autoload PSR-4 `"App\\": "app/"`, đặt class vào `app/`, chạy `composer dump-autoload` rồi `use` class đó từ `index.php`
6. Viết một exception tự định nghĩa và throw / catch nó
7. Trả lời được: vì sao PHP không giữ được biến giữa 2 request, và điều đó ảnh hưởng gì tới cách bạn cache dữ liệu

PART 00 — NỀN TẢNG

Kiến trúc tổng thể & Request Lifecycle

> **Mục tiêu:** Hiểu Laravel là gì ở tầng kiến trúc trước khi code. Laravel không phải "một đống helper function" — nó là một IoC container (kho chứa và tự lắp ráp object) được bootstrap bởi các Service Provider, rồi đẩy request qua một pipeline middleware. Hiểu 3 khái niệm đó là hiểu 80% Laravel.

1. Điểm khởi đầu: public/index.php → bootstrap/app.php

Mọi request HTTP đều được web server (Nginx/Apache) trỏ vào đúng một file duy nhất: `public/index.php`. File này rất ngắn, nó chỉ làm 3 việc: nạp Composer autoloader, lấy instance ứng dụng từ `bootstrap/app.php`, rồi gọi `handleRequest()`.

```
// bootstrap/app.php — "file cấu hình trung tâm" của Laravel 11+
return Application::configure(basePath: dirname(__DIR__))
    ->withRouting(
        web: __DIR__.'/../routes/web.php',
        api: __DIR__.'/../routes/api.php',       // chỉ có sau khi chạy install:api
        commands: __DIR__.'/../routes/console.php',
        health: '/up',
    )
    ->withMiddleware(function (Middleware $middleware) {
        // đăng ký / sắp xếp middleware ở đây
    })
    ->withExceptions(function (Exceptions $exceptions) {
        // tuỳ biến cách báo lỗi & render lỗi ở đây
    })->create();
```

> **Điểm khác biệt lớn so với Laravel 10 trở về trước:** từ Laravel 11, các file `app/Http/Kernel.php`, `app/Exceptions/Handler.php`, `app/Providers/RouteServiceProvider.php` đã bị xoá. Toàn bộ cấu hình middleware, exception, routing gom hết về `bootstrap/app.php`. Nếu bạn xem tutorial cũ trên YouTube/blog mà thấy sửa `Kernel.php` → đó là tài liệu lỗi thời, đừng làm theo.

2. Service Container (IoC Container) — trái tim của Laravel

Service Container là một "kho" biết cách tự khởi tạo object và tự tiêm dependency. Khi bạn type-hint một class ở constructor (hoặc ở tham số method của controller), Laravel dùng PHP Reflection để đọc kiểu dữ liệu đó, tự tạo instance rồi truyền vào — bạn không bao giờ phải tự `new PostService()`.

```
class PostController extends Controller
{
    public function __construct(
        protected PostService $posts,     // container tự resolve và tiêm vào
    ) {}
}
```

Đây chính xác là cơ chế Dependency Injection bạn đã gặp ở NestJS. Khác biệt: NestJS đọc metadata do decorator sinh ra lúc compile; Laravel đọc trực tiếp type-hint qua Reflection lúc runtime — không cần đăng ký

class vào một mảng `providers` nào cả nếu class đó có thể tự khởi tạo được.

3. Service Providers — nơi mọi thứ được "lắp ráp"

Service Provider là các class chịu trách nhiệm khai báo cho container biết: "gặp interface X thì dựng class Y", "service Z là singleton"... Mỗi provider có 2 method, và thứ tự chạy của chúng rất quan trọng:

Method	Chạy khi nào	Được làm gì
`register()`	Chạy trước, trên TẤT CẢ provider	Chỉ được đăng ký binding vào container. KHÔNG được dùng service của provider khác (vì có thể chưa đăng ký xong)
`boot()`	Chạy sau, khi mọi provider đã register xong	Làm mọi thứ cần dùng tới service khác: định nghĩa rate limiter, gate/policy, event listener, `Model::preventLazyLoading()`...

Danh sách provider của ứng dụng nằm ở `bootstrap/providers.php`; provider mặc định của bạn là `app/Providers/AppServiceProvider.php`.

4. Request Lifecycle — thứ tự một request đi qua hệ thống

Đây là sơ đồ cần thuộc. So với NestJS (Middleware → Guard → Interceptor → Pipe → Handler), Laravel gom gọn hơn: mọi thứ chặn/lọc request đều là middleware, còn validate dữ liệu thì nằm ở Form Request.

```
Client Request          HTTP request tới web server → trỏ vào public/index.php
      ▼
  Bootstrap             Nạp autoloader, tạo Application (container), đọc .env + config,
                        đăng ký & boot toàn bộ Service Provider
      ▼
  Global Middleware     Chạy cho MỌI request: TrustProxies, HandleCors, ValidatePostSize,
                        TrimStrings, ConvertEmptyStringsToNull...
      ▼
  Router                Tìm route khớp URI + HTTP method. Không khớp →
NotFoundHttpException (404)
      ▼
  Route Middleware      Middleware của nhóm api + của riêng route: SubstituteBindings
(route model
                        binding), auth:sanctum, throttle, can... Đây là chỗ tương đương
Guard bên NestJS
      ▼
  Form Request          Nếu controller type-hint một FormRequest → validate TRƯỚC khi thân
hàm chạy.
                        Sai → tự trả 422. Tương đương ValidationPipe + DTO bên NestJS
      ▼
  Controller            Method controller chạy, gọi Model/Service, trả về Resource hoặc
response
      ▼
  Response ← Middleware Response đi NGƯỢC ra qua đúng chuỗi middleware đó rồi send() về
client
      ▼
  Terminate             Middleware có terminate() chạy SAU khi response đã gửi (FastCGI)

⚠ Bất kỳ bước nào throw exception → nhảy thẳng tới Exception Handler (withExceptions)
  → render thành response lỗi. Request có Accept: application/json → Laravel tự trả JSON.
```

5. Facade thực chất là gì?

Khi bạn viết `Route::get(...)` hay `DB::table(...)`, trông như gọi static method, nhưng thực ra không có static method nào cả. Facade là một class proxy: nó bắt lời gọi static bằng `__callStatic()`, hỏi container lấy instance thật đang được đăng ký, rồi gọi method instance trên đó. Nghĩa là Facade chỉ là "lối tắt cú pháp" để lấy đồ trong container — object bên dưới vẫn mock được khi test (`Queue::fake()`, `Event::fake()`, `Storage::fake()` ở các Part sau đều là ứng dụng của điều này).

6. Đối chiếu nhanh NestJS ↔ Laravel

NestJS	Laravel	Ghi chú
Module (`@Module`)	Service Provider	Laravel không đóng gói theo module; code tổ chức theo thư mục + provider
IoC container + `providers: []`	Service Container (tự resolve qua Reflection)	Laravel không cần khai báo trước nếu class tự dựng được
Controller + `@Get()`	Controller + `routes/api.php`	Laravel tách route ra file riêng thay vì gắn decorator
DTO + class-validator	Form Request	Cùng vai trò: validate input trước khi vào handler
Guard (`CanActivate`)	Middleware (`auth:sanctum`, `can:...`)	Laravel gộp Guard vào middleware
Interceptor (transform response)	API Resource	Laravel transform ở tầng Resource thay vì bọc response
Exception Filter	`withExceptions()` trong `bootstrap/app.php`	Cùng mục đích: chuẩn hoá format lỗi
TypeORM Repository (Data Mapper)	Eloquent Model (Active Record)	Eloquent: bản thân model tự biết lưu/xoá chính nó
Bull/BullMQ queue	Queue + Job (`ShouldQueue`)	Laravel tích hợp sẵn, không cần package ngoài (Part 14)

Việc cần làm:

1. Chưa code gì cả — đọc hiểu sơ đồ lifecycle ở mục 4, tự vẽ lại ra giấy
2. Trả lời được: khác nhau giữa `register()` và `boot()` là gì, và tại sao phải tách làm 2 bước
3. Trả lời được: Facade khác gì với static method thật

PART 01

Khởi tạo dự án & cấu trúc thư mục

> **Mục tiêu:** Dựng được project Laravel chạy được, hiểu mỗi thư mục dùng để làm gì, và bật chế độ API (điều mà Laravel không bật sẵn).

1.1 Cài đặt

Yêu cầu: PHP 8.3 trở lên (Laravel 13 hỗ trợ PHP 8.3–8.5) và Composer. Kiểm tra bằng `php -v` và `composer -V`.

```
# Cách 1 — dùng Composer (không cần cài thêm gì)
composer create-project laravel/laravel laravel-crash-course

# Cách 2 — dùng Laravel installer (cài 1 lần, dùng mãi)
composer global require laravel/installer
laravel new laravel-crash-course

# Chạy dev server
cd laravel-crash-course
php artisan serve           # → http://127.0.0.1:8000
```

Khi cài sẽ có bước hỏi chọn starter kit / testing framework. Với crash course API này, chọn không dùng starter kit (none) vì ta không làm giao diện. Phần test framework chọn Pest (mặc định) — sẽ dùng ở Part 11. Nếu chọn SQLite, installer tự tạo `database/database.sqlite` và chạy migration mặc định.

Cách khác để có môi trường chạy: nếu không muốn cài PHP/MySQL/Redis trực tiếp lên máy, bạn có thể chạy cả stack trong Docker bằng **Laravel Sail** (xem **PART SAIL** ngay sau Part này). Trên macOS và Windows còn có **Laravel Herd** (PHP + Nginx dựng sẵn). Trong tài liệu này mọi lệnh viết dạng `php artisan ...` / `composer ...`; khi dùng Sail, thay bằng `sail artisan ...` / `sail composer ...`.

1.2 Bật API — bước bắt buộc, rất nhiều người quên

Laravel 11+ không tạo sẵn file `routes/api.php`. Dự án mới chỉ có `routes/web.php`. Muốn làm API bạn phải chạy:

```
php artisan install:api
```

Lệnh này làm 3 việc: (1) cài Laravel Sanctum (authentication bằng API token — Part 09), (2) tạo file `routes/api.php`, (3) thêm dòng `api:` vào `withRouting()` trong `bootstrap/app.php` đồng thời publish migration bảng `personal_access_tokens`.

Hệ quả quan trọng: mọi route khai trong `routes/api.php` được tự động thêm prefix `/api` và gán nhóm middleware `api`. Khai `Route::get('/posts', ...)` → URL thật là `/api/posts`. Đừng tự thêm `/api` vào route nữa, sẽ thành `/api/api/posts`.

1.3 Cấu hình môi trường (.env)

File `.env` chứa config theo từng máy (DB, key, debug) và không bao giờ được commit lên git (đã có sẵn trong `.gitignore`). File `.env.example` mới là file được commit — nó là "bản kê khai" các biến cần có để người khác biết đường tự tạo `.env` của họ.

```
APP_NAME=Laravel
APP_ENV=local
APP_KEY=base64:...            # sinh bởi php artisan key:generate — dùng để mã hoá cookie/session
APP_DEBUG=true                # PRODUCTION BẮT BUỘC = false (nếu true sẽ lộ config, stack trace)
APP_URL=http://localhost

DB_CONNECTION=sqlite          # mặc định của Laravel 11+ — nhẹ, không cần cài server
# Muốn dùng MySQL thì đổi thành:
# DB_CONNECTION=mysql
# DB_HOST=127.0.0.1 / DB_PORT=3306 / DB_DATABASE=... / DB_USERNAME=... / DB_PASSWORD=...

QUEUE_CONNECTION=database     # Part 14 sẽ giải thích; 'sync' = chạy ngay, không có queue thật
CACHE_STORE=database          # Part 16
```

Quy tắc vàng: KHÔNG gọi `env('APP_DEBUG')` rải rác trong code. Chỉ được gọi `env()` bên trong các file `config/*.php`, còn code ứng dụng thì đọc qua `config('app.debug')`. Lý do: khi chạy `php artisan config:cache` ở production, mọi lời gọi `env()` ngoài config sẽ trả về null — đây là lỗi kinh điển làm sập app khi deploy.

1.4 Cấu trúc thư mục cần nhớ

Đường dẫn	Vai trò
`public/index.php`	Entry point duy nhất — web server phải trỏ document root vào thư mục `public/`
`bootstrap/app.php`	Cấu hình trung tâm: routing, middleware, exception (Laravel 11+)
`bootstrap/providers.php`	Danh sách Service Provider của ứng dụng
`routes/api.php`	Route API (prefix `/api`, middleware group `api`)
`routes/web.php`	Route web (có session + CSRF) — API không dùng tới
`routes/console.php`	Artisan command tự viết + **lịch chạy định kỳ (Part 17)**
`app/Http/Controllers/`	Controller
`app/Http/Requests/`	Form Request — nơi validate input (Part 05)
`app/Http/Resources/`	API Resource — nơi định dạng output (Part 06)

`app/Models/`	Model Eloquent
`app/Jobs/`, `app/Events/`, `app/Listeners/`, `app/Notifications/`, `app/Policies/`	Sinh ra khi bạn chạy `make:job`, `make:event`... (Part 14, 15, 19, 20)
`app/Providers/`	Service Provider của bạn
`database/migrations/`	Lịch sử thay đổi cấu trúc bảng
`database/factories/`, `seeders/`	Sinh dữ liệu giả để dev & test
`config/`	File cấu hình (Laravel 11+ rút gọn, chạy `php artisan config:publish` để lấy thêm)
`storage/logs/laravel.log`	Log — chỗ đầu tiên phải xem khi có lỗi 500
`storage/app/`	File người dùng upload (Part 18) — không nằm trong `public/`

1.5 Vài lệnh artisan sống còn

Lệnh	Dùng để
`php artisan list`	Xem toàn bộ lệnh có sẵn
`php artisan route:list --path=api`	Liệt kê route API đã đăng ký (kiểm tra nhanh nhất)
`php artisan make:...`	Sinh class (controller, model, request, resource, job, event...)
`php artisan migrate`	Chạy migration lên database
`php artisan tinker`	REPL — gõ thẳng code PHP có sẵn toàn bộ app (test Eloquent cực nhanh)
`php artisan optimize:clear`	Xoá mọi cache (config/route/view/event) khi thấy "sửa mà không ăn"
`php artisan about`	Tổng quan môi trường: version, driver cache/queue/session đang dùng

1.6 Git cho dự án — thói quen cần có ngay từ đầu

Dự án này sẽ qua hơn 20 Part. Nếu không dùng Git đều đặn, bạn sẽ không thể quay lại khi một thay đổi làm hỏng mọi thứ, và cũng không có "lịch sử học" để xem lại. Quy tắc cốt lõi: **commit nhỏ, mỗi commit một việc, message nói rõ việc đó**.

Lệnh	Dùng khi
`git status`	Xem file nào đã đổi, file nào chưa theo dõi. Chạy trước mỗi lần commit
`git diff` / `git diff --staged`	Xem chính xác dòng nào đã đổi (chưa add / đã add) — đọc lại trước khi commit
`git add -p`	Chọn từng đoạn thay đổi để add, giúp mỗi commit chỉ chứa một việc

`git commit -m "..."`	Lưu một mốc. Viết thể hiện việc đã làm, ví dụ `Part 04: thêm migration và model Post`
`git log --oneline`	Xem lịch sử gọn, mỗi commit một dòng
`git switch -c part-05-validation`	Tạo nhánh mới để làm một Part mà không đụng `main`
`git restore <file>`	Bỏ thay đổi chưa commit của một file (cẩn thận: mất thay đổi đó)

Cái gì không bao giờ được commit

- `.env` — chứa mật khẩu DB, `APP_KEY`, khoá API. Docs Laravel nói rõ file này không được đưa vào source control vì đó là rủi ro bảo mật nếu repo bị lộ. Thay vào đó commit `.env.example` (không chứa bí mật thật).
- `vendor/` và `node_modules/` — sinh lại được bằng `composer install` / `npm install`; project mới của Laravel đã ignore sẵn các thư mục này.
- File database SQLite thật (`database/database.sqlite`), file log, file upload trong `storage/` .

Lỡ commit `.env` thì sao? Xoá file khỏi commit là chưa đủ, vì nó vẫn nằm trong lịch sử. Phải coi mọi khoá trong đó là đã lộ: đổi mật khẩu DB, chạy `php artisan key:generate` tạo `APP_KEY` mới, thu hồi khoá API của bên thứ ba. Chỉ sau đó mới dọn lịch sử.

Quy ước làm việc đề nghị cho dự án này

1. Mỗi Part làm trên một nhánh `part-NN-ten` , test chạy xanh rồi mới gộp vào `main` .
2. Mỗi lần xong một "Việc cần làm" thì commit một lần, không dồn cả Part vào một commit.
3. Chạy `git status` và `git diff --staged` trước khi commit để chắc không lẫn `.env` hoặc file rác.

Việc cần làm:

1. Tạo project `laravel-crash-course` trong repo `crash-course`
2. Chạy `php artisan install:api` , xác nhận file `routes/api.php` đã xuất hiện và `bootstrap/app.php` có thêm dòng `api:`
3. Chạy `php artisan serve` , mở `http://127.0.0.1:8000/up` (health check) — phải thấy trang báo application up
4. Chạy `php artisan route:list` và đọc hiểu output: cột Method / URI / Action / Middleware
5. Mở `bootstrap/app.php` đọc kỹ 4 method `withRouting` / `withMiddleware` / `withExceptions` / `create`
6. Chạy `php artisan about` , ghi lại driver queue/cache/session đang dùng — sẽ cần ở Part 14/16
7. Chạy `git ls-files | grep -E '^\.env$'` — lệnh này **không được in ra gì** (nghĩa là `.env` chưa bị theo dõi). Chạy `git check-ignore -v .env` để xem dòng nào trong `.gitignore` chặn nó
8. Commit project vừa tạo với message rõ ràng, rồi tạo nhánh `part-02-routing` bằng `git switch -c` trước khi bắt đầu Part 02

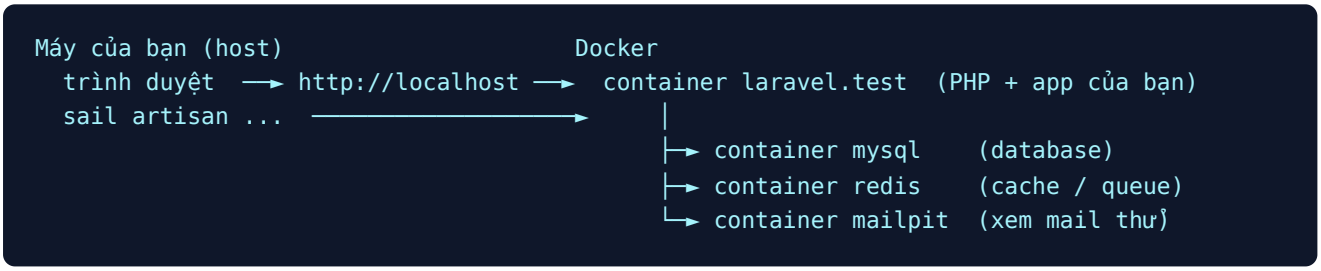
PART SAIL — CHẠY LARAVEL BẰNG DOCKER (TUỲ CHỌN) — MỚI

Laravel Sail: môi trường dev trong Docker

> **Mục tiêu:** Chạy được dự án Laravel cùng MySQL, Redis, Mailpit trong container mà không cài gì trực tiếp lên máy, biết chuyển mọi lệnh `php artisan` / `composer` sang dạng `sail ...`, và hiểu Docker ở mức đủ để đọc phần deploy ở Part 22. **Phần này tuỳ chọn**: nếu máy bạn đã có PHP 8.3+ và Composer chạy tốt, bạn có thể bỏ qua và quay lại khi cần Redis/MySQL/Mailpit mà không muốn cài tay.

S.1 Sail là gì, cần gì

Laravel Sail là một giao diện dòng lệnh nhẹ để dùng môi trường Docker mặc định của Laravel. Về bản chất nó chỉ gồm hai thứ nằm ở thư mục gốc dự án: file `compose.yaml` (khai báo các container) và script `sail` (gói các lệnh Docker thành lệnh ngắn). Bạn không cần biết Docker trước. Sail hỗ trợ macOS, Linux và Windows (qua **WSL2**); máy phải cài sẵn Docker.



Container chính tên là `laravel.test` chạy ứng dụng. Các service khác là container riêng, nói chuyện với nhau qua tên (xem S.4).

S.2 Cài Sail vào dự án có sẵn

```
composer require laravel/sail --dev
php artisan sail:install
./vendor/bin/sail up
```

1. `composer require laravel/sail --dev`: cài package Sail (chỉ dùng khi phát triển).
2. `php artisan sail:install`: sinh file `compose.yaml` ở thư mục gốc và sửa `.env` thêm biến để kết nối các service Docker. Bước này cho bạn chọn các service cần dùng. Muốn thêm service sau này: `php artisan sail:add`.
3. `./vendor/bin/sail up`: khởi động mọi container. Lần đầu có thể mất vài phút vì phải tải image.

> **Lưu ý trước khi `sail up`:** docs yêu cầu tắt các web server và database đang chạy trên máy bạn (cổng 80, 3306 bị chiếm sẽ gây xung đột). Nếu dùng Docker Desktop cho Linux, hãy dùng context mặc định: `docker context use default`; nếu gặp lỗi quyền file trong container, đặt biến `SUPERVISOR_PHP_USER` thành `root`.

S.3 Alias `sail` và vòng đời container

Gõ `./vendor/bin/sail` mỗi lần rất mệt. Docs đề nghị tạo alias (thêm vào `~/.bashrc` hoặc `~/.zshrc` rồi mở lại terminal):

```
alias sail='sh $([ -f sail ] && echo sail || echo vendor/bin/sail)'
```

Lệnh	Tác dụng
`sail up`	Khởi động, giữ terminal; Ctrl+C để dừng
`sail up -d`	Khởi động chạy nền (detached)
`sail stop`	Dừng các container đang chạy nền
`sail build --no-cache`	Build lại image từ đầu (sau khi sửa `compose.yaml`, đổi PHP/Node version)

Khi `sail up` chạy xong, mở `http://localhost` (không phải cổng 8000) để xem ứng dụng.

S.4 Mọi lệnh phải chạy BÊN TRONG container

Đây là điểm người mới hay vấp nhất. Ứng dụng chạy trong container, tách biệt với máy bạn. Các ví dụ trong docs Laravel viết `php artisan ...` mặc định là chạy trên máy thật; khi dùng Sail phải thêm tiền tố `sail`:

Không dùng Sail	Dùng Sail
`php artisan migrate`	`sail artisan migrate`
`php artisan queue:work`	`sail artisan queue:work` (Part 14)
`composer require laravel/sanctum`	`sail composer require laravel/sanctum`
`php script.php`	`sail php script.php`
`php artisan test` / `./vendor/bin/pest`	`sail test` (Part 11) — dùng được mọi tuỳ chọn của Pest/PHPUnit
`php artisan tinker`	`sail tinker`
`npm run dev`	`sail npm run dev`
(mở terminal trong container)	`sail shell` (hoặc `sail root-shell` nếu cần quyền root)

Lỗi kinh điển: chạy `php artisan migrate` trên máy thật khi `.env` có `DB_HOST=mysql`. Tên `mysql` chỉ tồn tại **bên trong mạng Docker**, máy thật không phân giải được nên báo không kết nối được database. Hãy chạy `sail artisan migrate`.

S.5 Kết nối các service (khớp với các Part sau)

Service	Cấu hình trong `.env`	Dùng ở Part

MySQL	`DB_HOST=mysql`. Từ máy thật (TablePlus...) kết nối `localhost:3306`, user/mật khẩu theo `DB_USERNAME` / `DB_PASSWORD`. Lần đầu khởi động Sail tạo sẵn database `DB_DATABASE` và một database tên `testing` để test không đụng dữ liệu dev	04, 11
Redis	`REDIS_HOST=redis`; từ máy thật: `localhost:6379`	14, 16, 22
Mailpit (bắt mail thử)	`MAIL_HOST=mailpit`, `MAIL_PORT=1025`, `MAIL_ENCRYPTION=null`. Xem mail đã "gửi" tại `http://localhost:8025`	20
Dữ liệu	MySQL và Redis dùng **Docker volume** nên dữ liệu còn nguyên khi bạn stop rồi start lại container	

Về file upload: nếu muốn thử `disk s3` ở Part 18 mà không đụng AWS thật, Sail có thể cài service lưu trữ tương thích S3 (docs đề cập RustFS) và bạn chỉ đổi biến `AWS_*` trong `.env`. Cũng có thể thêm service bằng `sail:add`.

S.6 Đổi phiên bản PHP, Node, thêm extension

- Sail hỗ trợ nhiều phiên bản PHP; mặc định hiện tại là **PHP 8.5** và Node 24. Đổi PHP bằng cách sửa `build.context` của service `laravel.test` trong `compose.yaml` (ví dụ `./vendor/laravel/sail/runtimes/8.4`), rồi `sail build --no-cache` và `sail up`.
- Thêm extension PHP: đặt build arg `PHP_EXTENSIONS` (danh sách cách nhau bằng dấu cách) cho `laravel.test`, rồi build lại.
- Laravel 13 yêu cầu **PHP 8.3 trở lên** — đừng chọn runtime thấp hơn.

S.7 Khi nào dùng Sail, khi nào cài trực tiếp

Tình huống	Nên chọn
Máy đã có PHP 8.3+ và Composer chạy tốt, chỉ làm API với SQLite	Cài trực tiếp, dùng `php artisan serve`. Đơn giản và nhanh nhất
Cần MySQL/PostgreSQL + Redis + Mailpit giống production, không muốn cài tay	Sail
Làm việc nhóm, muốn mọi người có cùng phiên bản PHP/DB	Sail (cả nhóm chạy cùng `compose.yaml`)
Dùng macOS/Windows, thích công cụ có giao diện, không cần Docker	Laravel Herd (PHP + Nginx dựng sẵn; bản Pro có thêm MySQL/Redis/Mailpit)

S.8 Docker bạn cần hiểu để đọc Part 22

- **Image**: khuôn dựng sẵn (PHP + extension + công cụ). **Container**: một thể hiện đang chạy của image.
- **compose.yaml**: file khai báo nhiều container và mạng nối giữa chúng; `sail up` chính là chạy file này.
- **Volume**: ổ lưu trữ tồn tại ngoài vòng đời container (dữ liệu DB nằm ở đây).
- Production dùng cùng ý tưởng: mỗi loại process ở Part 22.1 (web, worker, scheduler) là một container riêng cùng chạy một image ứng dụng.

Việc cần làm (tuỳ chọn):

1. Cài Docker, chạy `composer require laravel/sail --dev` và `php artisan sail:install`, chọn MySQL + Redis + Mailpit
2. Tạo alias `sail`, chạy `sail up -d`, mở `http://localhost` và `http://localhost/up`
3. Chạy `sail artisan migrate`, kết nối MySQL từ máy thật bằng một GUI client (TablePlus, DBeaver...) tới `localhost:3306` để xem bảng vừa tạo
4. Thử cố ý chạy `php artisan migrate` (không có `sail`) khi `DB_HOST=mysql`, ghi lại thông báo lỗi để nhận ra lần sau
5. Gửi một mail thử (Part 20) và xem nó ở Mailpit `http://localhost:8025`
6. Chạy `sail test` và xác nhận test dùng database `testing`, không đụng dữ liệu dev
7. Chạy `sail stop`, rồi `sail up -d` lại và kiểm tra dữ liệu MySQL vẫn còn (nhờ volume)

PART 02

Routing & Controllers cho API

Mục tiêu: Map được HTTP request tới đúng method trong Controller theo chuẩn REST, và hiểu Route Model Binding — thứ giúp bạn xoá sạch code "tìm bản ghi, không thấy thì trả 404".

2.1 Khai báo route

```php
// routes/api.php — nhớ: đã tự có prefix /api
use App\Http\Controllers\PostController;

Route::get('/posts', [PostController::class, 'index']);
Route::post('/posts', [PostController::class, 'store']);
Route::get('/posts/{post}', [PostController::class, 'show']);
Route::put('/posts/{post}', [PostController::class, 'update']);
Route::delete('/posts/{post}', [PostController::class, 'destroy']);
```

2.2 apiResource — viết 5 route trên bằng 1 dòng

```php
Route::apiResource('posts', PostController::class);
```

Verb	URI	Method	Route name
GET	`/api/posts`	index	posts.index
POST	`/api/posts`	store	posts.store
GET	`/api/posts/{post}`	show	posts.show
PUT/PATCH	`/api/posts/{post}`	update	posts.update
DELETE	`/api/posts/{post}`	destroy	posts.destroy

`apiResource` khác `resource` ở chỗ nó bỏ 2 route `create` và `edit` — hai route đó chỉ để trả về form HTML, API thuần không cần. Sinh controller tương ứng bằng:

```
# --api: sinh sẵn 5 method (không có create/edit)
# --model: type-hint sẵn model Post vào tham số (bật route model binding)
# --requests: sinh luôn StorePostRequest + UpdatePostRequest
php artisan make:controller PostController --api --model=Post --requests
```

PUT hay PATCH? Chuẩn HTTP: PUT = thay thế toàn bộ resource (client gửi đủ mọi field), PATCH = sửa một phần. `apiResource` đăng ký cả hai cùng trỏ vào `update()`, nên bạn tự quyết định: nếu rule validation cho phép thiếu field (`sometimes`) thì thực chất bạn đang làm PATCH — hãy để client gọi PATCH cho đúng ngữ nghĩa.

2.3 Route Model Binding — tính năng "ăn tiền" nhất

Thay vì nhận `$id` rồi tự query, bạn type-hint thẳng Model. Middleware `SubstituteBindings` (có sẵn trong nhóm `api`) sẽ tự query bản ghi theo giá trị URI, và tự trả 404 nếu không tìm thấy:

```
// Route: /api/posts/{post}   →   tên tham số {post} phải TRÙNG tên biến $post
public function show(Post $post)
{
    return $post;   // tới được đây nghĩa là chắc chắn tìm thấy
}
```

So sánh với NestJS: bên đó bạn phải viết `ParseIntPipe` + tự `throw new NotFoundException()` trong service. Laravel làm sẵn cả hai. Muốn tìm theo cột khác `id` (ví dụ slug) — có 2 cách:

```
// Cách 1: khai ngay tại route (chỉ áp dụng cho route đó)
Route::get('/posts/{post:slug}', [PostController::class, 'show']);

// Cách 2 (Laravel 13): gắn attribute lên Model → áp dụng cho MỌI route bind Post
use Illuminate\Database\Eloquent\Attributes\RouteKey;

#[RouteKey('slug')]
class Post extends Model { }
```

Đã đối chiếu docs Routing 13.x: `#[RouteKey('slug')]` đúng là attribute chính thức (namespace `Illuminate\Database\Eloquent\Attributes\RouteKey`). Cách cũ `getRouteKeyName()` vẫn dùng được. Với route lồng (`/posts/{post}/comments/{comment}`), Laravel tự "scope" comment phải thuộc post đó khi bạn dùng custom key — xem Part 08.

2.4 Controller gọn gàng

```php
namespace App\Http\Controllers;

use App\Models\Post;

class PostController extends Controller
{
    public function index()
    {
        return Post::latest()->paginate(15);
    }

    public function store(StorePostRequest $request)      // validate tự chạy trước
    {
        $post = Post::create($request->validated());

        return response()->json($post, 201);            // 201 Created
    }

    public function destroy(Post $post)
    {
        $post->delete();

        return response()->noContent();                 // 204 No Content
    }
}
```

Nguyên tắc "controller mỏng": controller chỉ nên làm 3 việc — nhận request đã validate, gọi model/service, trả response. Không đặt logic nghiệp vụ phức tạp, không viết query dài dòng ở đây. Part 21 sẽ chỉ chỗ đặt những thứ đó.

2.5 Nhóm route & middleware

```php
Route::apiResource('posts', PostController::class)->only(['index', 'show']);

// Nhóm route cần đăng nhập
Route::middleware('auth:sanctum')->group(function () {
    Route::apiResource('posts', PostController::class)->except(['index', 'show']);
});

// Hoặc chỉ định theo từng action — gọn hơn (docs 13.x: middlewareFor /
withoutMiddlewareFor)
Route::apiResource('posts', PostController::class)
    ->middlewareFor(['store', 'update', 'destroy'], 'auth:sanctum');

// Hoặc khai ngay trên controller bằng attribute (Laravel 13)
use Illuminate\Routing\Attributes\Controllers\Middleware;

#[Middleware('auth:sanctum', except: ['index', 'show'])]
class PostController extends Controller { }
```

Việc cần làm:

1. Tạo `PostController` bằng `make:controller --api`
2. Đăng ký `Route::apiResource('posts', ...)` trong `routes/api.php`
3. Tạm thời trả về mảng cứng trong bộ nhớ (chưa cần database) cho đủ 5 action
4. Chạy `php artisan route:list --path=api`, đối chiếu với bảng ở mục 2.2
5. Test 5 endpoint bằng Postman/curl. Nhớ gửi header `Accept: application/json` — thiếu header này Laravel sẽ trả HTML khi có lỗi
6. Thử gọi `/api/posts/999` và quan sát response khi chưa có model binding

---

PART 03

Service Container, DI & Service Providers

Mục tiêu: Tách logic nghiệp vụ ra khỏi Controller, và hiểu container tự lắp ráp object như thế nào — nền tảng để code test được.

3.1 Tự động resolve (zero config)

Container đọc type-hint bằng Reflection và tự dựng object. Với class không có dependency đặc biệt, bạn không cần khai báo gì cả:

```php
namespace App\Services;

class PostService
{
    public function publish(Post $post): Post
    {
        $post->update(['published_at' => now()]);

        return $post;
    }
}

// Controller — chỉ cần type-hint, container lo phần còn lại
class PostController extends Controller
{
    public function __construct(protected PostService $posts) {}
}
```

Khác NestJS ở đây: Nest bắt buộc khai `providers: [PostsService]` trong module, quên là lỗi "Nest can't resolve dependencies". Laravel không cần — chỉ khi bạn muốn can thiệp vào cách dựng object mới phải đăng ký binding.

3.2 Khi nào phải bind thủ công

Khi type-hint là interface (container không biết dựng interface), hoặc khi cần truyền tham số đặc biệt, hoặc muốn dùng chung một instance:

```php
// app/Providers/AppServiceProvider.php
public function register(): void
{
    // bind: mỗi lần resolve tạo instance MỚI (giống scope TRANSIENT bên Nest)
    $this->app->bind(PaymentGateway::class, StripeGateway::class);

    // singleton: tạo 1 lần, dùng lại mãi (giống scope DEFAULT bên Nest)
    $this->app->singleton(PostService::class, function ($app) {
        return new PostService(config('services.posts.per_page'));
    });

    // scoped: 1 instance cho mỗi request (quan trọng khi chạy Octane – Part 22)
    $this->app->scoped(RequestContext::class, fn () => new RequestContext());
}
```

Cách bind	Vòng đời instance	Tương đương NestJS
`bind`	Tạo mới mỗi lần được resolve	Scope.TRANSIENT
`singleton`	Một instance duy nhất cho cả app	Scope.DEFAULT
`scoped`	Một instance cho mỗi request	Scope.REQUEST

Lưu ý về vòng đời process: PHP-FPM mặc định huỷ toàn bộ bộ nhớ sau mỗi request — nên "singleton" của Laravel thực chất chỉ sống trong một request, khác hẳn Node.js/NestJS nơi singleton sống suốt đời tiến trình server. Đây là lý do bạn không thể "cache dữ liệu vào thuộc tính static" trong Laravel như bên Nest. Ngoại lệ: Queue worker và Octane là process sống lâu — lúc đó singleton sống thật, và `scoped` mới trở nên quan trọng.

3.3 Bind theo interface — vì sao nên làm

```php
// Contract
interface NotifierInterface { public function send(string $msg): void; }

// Ở AppServiceProvider::register()
$this->app->bind(NotifierInterface::class, MailNotifier::class);
```

Lợi ích: đổi `MailNotifier` → `SlackNotifier` chỉ sửa 1 dòng; khi test thì bind sang một fake object. Đây là nguyên tắc Dependency Inversion — code phụ thuộc vào abstraction, không phụ thuộc implementation cụ thể.

3.4 register() vs boot() — lỗi hay gặp

```php
public function register(): void
{
    // ✔ CHỈ đăng ký binding ở đây
    $this->app->singleton(PostService::class);
    // ✘ SAI: Route::pattern(...) / RateLimiter::for(...) — lúc này provider khác có thể chưa sẵn sàng
}

public function boot(): void
{
    // ✔ Mọi thứ cần dùng service khác thì đặt ở đây
    RateLimiter::for('api', fn ($request) =>
        Limit::perMinute(60)->by($request->user()?->id ?: $request->ip()));

    Model::preventLazyLoading(! $this->app->isProduction());   // Part 08
}
```

Việc cần làm:

1. Tạo `app/Services/PostService.php`, chuyển toàn bộ logic từ Controller sang đây
2. Inject `PostService` vào Controller qua constructor — xác nhận chạy được mà KHÔNG cần đăng ký gì
3. Tạo interface `PostRepositoryInterface` + một implementation, bind trong `AppServiceProvider::register()`, rồi type-hint interface đó ở service
4. Thử cố tình bind sai (bind interface tới class không tồn tại) để xem thông báo lỗi của container trông thế nào
5. Trả lời: vì sao `RateLimiter::for()` phải nằm trong `boot()` chứ không phải `register()`

PART SQL — NỀN TẢNG DỮ LIỆU (HỌC TRƯỚC PART 04) — MỚI

SQL nền tảng: bảng, JOIN, index, EXPLAIN

Mục tiêu: Hiểu những gì xảy ra ở tầng database bên dưới Eloquent: bảng, khoá chính/ngoại, `SELECT` / `WHERE` / `GROUP BY`, `JOIN`, index và kế hoạch thực thi. Xong phần này, N+1, `withCount`, `whereHas` và index trong migration không còn là "phép thuật", và bạn đọc được SQL mà Eloquent sinh ra.

SQL.1 Vì sao phải học SQL trước khi học Eloquent

Eloquent và Query Builder chỉ là cách viết PHP để **sinh ra SQL**. 90% vấn đề hiệu năng của ứng dụng thật nằm ở câu SQL, không phải ở PHP. Nếu không biết SQL, bạn không thể trả lời "câu này chậm vì sao" hay "N+1 nghĩa là gì".

Bạn viết (Eloquent / Query Builder)	SQL thực sự chạy

`Post::where('views', '>', 100)->orderByDesc('views')->limit(2)->get()`	`SELECT * FROM posts WHERE views > 100 ORDER BY views DESC LIMIT 2`
`$post->comments` (lazy load)	`SELECT * FROM comments WHERE post_id = ?` — **một câu cho mỗi post**
`Post::with('comments')->get()`	2 câu: lấy posts; `SELECT * FROM comments WHERE post_id IN (...)`
`Post::withCount('comments')`	Một truy vấn con `COUNT(*)` nằm trong `SELECT`
`->paginate(15)`	`... LIMIT 15 OFFSET ...` kèm một câu `SELECT COUNT(*)` để biết tổng
`->whereNull('published_at')`	`WHERE published_at IS NULL`
`$post->increment('views')`	`UPDATE posts SET views = views + 1 WHERE id = ?`
`DB::transaction(fn () => ...)`	`BEGIN ... COMMIT` (hoặc `ROLLBACK` nếu có lỗi)

Xem SQL thật mà Laravel sinh ra: gắn `->dumpRawSql()` (in rồi chạy tiếp) hoặc `->ddRawSql()` (in rồi dừng) vào cuối một query để thấy câu SQL với giá trị đã thay vào. Muốn xem mọi câu của cả request, đăng ký `DB::listen()` trong `boot()` của `AppServiceProvider` (có `$query->sql`, `$query->time` ...). Hai công cụ này là cách bạn đếm query khi săn N+1 ở Part 08.

Cách luyện: Laravel 13 mặc định dùng **SQLite** (project mới tự tạo file `database/database.sqlite`), nên bạn có database để thử ngay, không cần cài thêm. Hai cách chạy SQL: (1) `php artisan db` mở client dòng lệnh của database (cần có client tương ứng cài trên máy); (2) chạy bằng `PDO` của PHP — đây là cách **Chương 12 của file php-bai-tap.pdf** làm, có sẵn 11 bài kèm đáp án.

SQL.2 Bảng, khoá chính, khoá ngoại — schema blog

Dự án xuyên suốt của tài liệu là blog: `users`, `posts`, `comments`. Quan hệ: một user có nhiều post, một post có nhiều comment.

```
users (id, name, email)
   1 ──── n  posts (id, user_id, title, views, published_at)
               1 ──── n  comments (id, post_id, body)
```

```sql
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    email TEXT NOT NULL UNIQUE
);

CREATE TABLE posts (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id INTEGER NOT NULL REFERENCES users(id),
    title TEXT NOT NULL,
    views INTEGER NOT NULL DEFAULT 0,
    published_at TEXT
);

CREATE TABLE comments (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    post_id INTEGER NOT NULL REFERENCES posts(id),
    body TEXT NOT NULL
);
```

- **Khoá chính** (`PRIMARY KEY`): định danh duy nhất của mỗi dòng. `AUTOINCREMENT` tự tăng (MySQL viết `AUTO_INCREMENT`).
- **Khoá ngoại** (`REFERENCES`): ràng buộc `posts.user_id` phải là một `users.id` có thật. Nhờ đó database không cho tồn tại bài viết "mồ côi". Trong migration: `$table->foreignId('user_id')->constrained()` .
- **Ràng buộc** `NOT NULL` , `UNIQUE` , `DEFAULT` giúp database tự chặn dữ liệu sai, không phụ thuộc vào code PHP. Đây là lớp phòng thủ cuối cùng khi validation ở Part 05 bị bỏ sót.
- Docs Laravel: khoá ngoại **được bật mặc định** cho kết nối SQLite (tắt bằng `DB_FOREIGN_KEYS=false`). Nếu tự dùng SQLite ngoài Laravel (ví dụ client dòng lệnh hay PDO thuần) thì phải bật bằng `PRAGMA foreign_keys = ON` , vì SQLite mặc định **không** kiểm tra.

SQL.3 Đọc và lọc dữ liệu

Dữ liệu mẫu: 3 user (An, Bình, Chi — Chi chưa có bài), 5 post (`Học PHP` 120 view, `Học Laravel` 300, `Học SQL` 80 và chưa xuất bản, `Docker cơ bản` 150, `Redis là gì` 45), 6 comment (post 3 và 5 chưa có comment nào).

```sql
SELECT title, views FROM posts WHERE views > 100 ORDER BY views DESC;

SELECT title, views FROM posts ORDER BY views DESC LIMIT 2 OFFSET 2;

SELECT title FROM posts WHERE title LIKE 'Học%';

SELECT COUNT(*), COUNT(published_at) FROM posts;
```

Câu	Kết quả thật khi chạy
views > 100, giảm dần	`Học Laravel 300` , `Docker cơ bản 150` , `Học PHP 120`
`LIMIT 2 OFFSET 2` (bỏ 2 dòng đầu, lấy 2 dòng)	`Học PHP 120` , `Học SQL 80` — đây là cơ chế của phân trang

`COUNT(*)` và `COUNT(published_at)`	`5` và `4` — `COUNT(*)` đếm dòng, `COUNT(cột)` chỉ đếm dòng có giá trị khác NULL

Thứ tự thực thi của một câu SELECT

Viết theo thứ tự `SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT` nhưng database chạy theo thứ tự: `FROM` (chọn bảng) → `WHERE` (lọc dòng) → `GROUP BY` (gom nhóm) → `HAVING` (lọc nhóm) → `SELECT` (chọn cột) → `ORDER BY` → `LIMIT`. Hiểu thứ tự này giải thích vì sao `WHERE` không dùng được hàm gộp như `SUM()`.

NULL: cái bẫy lớn nhất

> **NULL nghĩa là "không biết giá trị"**, không phải 0 hay chuỗi rỗng. Hệ quả: `WHERE published_at = NULL` không bao giờ đúng với dòng nào (kết quả **0 dòng**) — phải viết `IS NULL` (tìm thấy 1 bài nháp). Tương tự `published_at != '2026-01-05'` **loại luôn** các dòng NULL, nên muốn lấy cả bài nháp phải thêm `OR published_at IS NULL`. Trong Laravel: `whereNull()`, và cột phải khai báo `->nullable()` trong migration mới chứa được NULL.

Cẩn thận với `UPDATE` và `DELETE`: **luôn có** `WHERE`. Thiếu nó thì mọi dòng bị sửa hoặc xoá mà database không hỏi lại. Khi tăng bộ đếm, dùng `UPDATE posts SET views = views + 1 WHERE id = 1` (một câu, nguyên tử) thay vì đọc ra PHP, cộng, rồi ghi lại — cách đó mất lượt xem khi hai request chạy cùng lúc.

SQL.4 Gộp dữ liệu: GROUP BY, HAVING

```sql
SELECT user_id, COUNT(*) AS bai, SUM(views) AS tong_view
FROM posts
GROUP BY user_id
HAVING SUM(views) > 200;
```

Kết quả: chỉ còn `user_id = 1` với `3` bài và `500` lượt xem (user 2 có `2` bài, `195` lượt nên bị loại).

- `GROUP BY` gom các dòng cùng giá trị cột thành nhóm; hàm gộp (`COUNT`, `SUM`, `AVG`, `MIN`, `MAX`) tính trên từng nhóm.
- `WHERE` **lọc dòng trước khi gom,** `HAVING` **lọc nhóm sau khi gom.** Điều kiện trên kết quả hàm gộp bắt buộc để ở `HAVING`.
- Cột trong `SELECT` không nằm trong hàm gộp thì phải có trong `GROUP BY`.

SQL.5 JOIN: ghép bảng theo khoá ngoại

```
SELECT p.title, u.name
FROM posts p
JOIN users u ON u.id = p.user_id;

SELECT u.name, COUNT(p.id) AS so_bai
FROM users u
LEFT JOIN posts p ON p.user_id = u.id
GROUP BY u.id;

SELECT p.title
FROM posts p
LEFT JOIN comments c ON c.post_id = p.id
WHERE c.id IS NULL;
```

Loại	Giữ lại những dòng nào	Kết quả với dữ liệu mẫu
INNER JOIN (viết `JOIN`)	Chỉ dòng ghép được ở **cả hai** bảng	Câu 1: 5 bài kèm tên tác giả (An: 3 bài, Bình: 2 bài)
LEFT JOIN	**Mọi dòng bảng bên trái**; không ghép được thì cột bên phải là NULL	Câu 2: `An 3`, `Bình 2`, `Chi 0` — Chi vẫn có mặt với 0 bài
LEFT JOIN + `WHERE ... IS NULL`	Dòng bên trái **không có** dòng tương ứng bên phải (anti-join)	Câu 3: `Học SQL`, `Redis là gì` (hai bài chưa có comment)

Hai điều hay nhầm: (1) Với `LEFT JOIN`, dùng `COUNT(p.id)` chứ không phải `COUNT(*)`: dòng của Chi có `p.id` là NULL nên `COUNT(p.id)` cho `0` đúng, còn `COUNT(*)` đếm cả dòng đó và ra `1` sai. (2) Đặt điều kiện lọc của bảng bên phải trong `WHERE` sẽ biến `LEFT JOIN` thành `INNER JOIN` vì dòng NULL bị lọc mất; điều kiện đó nên để trong `ON`.

Liên hệ: `Post::doesntHave('comments')` là anti-join ở trên; `whereHas` / `withCount` (Part 08) sinh ra truy vấn con tương tự. Quan hệ `belongsTo` / `hasMany` của Eloquent **không dùng JOIN** mà chạy truy vấn riêng cho mỗi bảng (xem SQL.6).

SQL.6 N+1 nhìn ở tầng SQL

Muốn in 5 bài viết kèm tên tác giả. Có ba cách, đếm số câu SQL chạy thật:

Cách	Câu SQL	Số query với 5 bài
A — N+1	1 câu lấy posts, rồi **mỗi post** một câu `SELECT name FROM users WHERE id = ?`	**6** (1 + N)
B — eager loading	1 câu lấy posts + 1 câu `SELECT ... FROM users WHERE id IN (1, 2)`, ghép trong PHP	**2**, bất kể bao nhiêu bài
C — JOIN	Một câu `JOIN` duy nhất	**1**

Cách A với 1000 bài là **1001** lần gọi database. Cách B chính là điều `Post::with('author')->get()` làm: không phải phép màu, chỉ là một câu `IN` và một bước ghép. Với quan hệ một-nhiều (post và comment),

JOIN nhân bản dòng cha theo số con nên Eloquent chọn cách B. Ở Part 08 bạn sẽ bật `Model::preventLazyLoading()` để Laravel ném lỗi khi có lazy load.

SQL.7 Index và EXPLAIN

Index giống mục lục cuối sách: thay vì đọc từng trang để tìm một từ, bạn tra mục lục rồi nhảy thẳng tới trang. Về cấu trúc, index thường là **B-tree**, cho phép tìm trong cỡ `log n` bước thay vì `n` (cùng ý tưởng binary search). Thử trên bảng `logs` có 20000 dòng, lọc `user_id = 42`:

```
EXPLAIN QUERY PLAN SELECT * FROM logs WHERE user_id = 42;

CREATE INDEX idx_logs_user_id ON logs (user_id);

EXPLAIN QUERY PLAN SELECT * FROM logs WHERE user_id = 42;
```

Tình huống	Kế hoạch thực thi (kết quả thật)	Nghĩa là
Chưa có index	`SCAN logs`	Quét **toàn bộ** 20000 dòng
Đã có index `user_id`	`SEARCH logs USING INDEX idx_logs_user_id (user_id=?)`	Nhảy thẳng tới các dòng của user 42
Lọc cột chưa có index (`action`)	`SCAN logs`	Index chỉ giúp đúng cột nó đánh
Index hợp `(user_id, action)`, lọc cả hai cột	`SEARCH logs USING COVERING INDEX idx_logs_user_action (user_id=? AND action=?)`	Mọi cột cần dùng đã nằm sẵn trong index, không phải đọc thêm bảng

- **Nên đánh index** cho cột hay xuất hiện ở `WHERE`, `JOIN ... ON` (khoá ngoại), `ORDER BY`. Trong migration: `$table->index('user_id')`, `$table->index(['user_id', 'action'])`, `$table->unique('email')`.
- **Quy tắc tiền tố trái**: index `(a, b)` giúp `WHERE a = ?` và `WHERE a = ? AND b = ?`, nhưng **không** giúp `WHERE b = ?` đứng riêng. Thứ tự cột trong index hợp quan trọng.
- **Cái giá của index**: tốn dung lượng và làm `INSERT/UPDATE/DELETE` chậm hơn vì phải cập nhật cả index. Đừng đánh index mọi cột.
- Mẫu `LIKE '%từ%'` (dấu `%` ở đầu) **không dùng được index B-tree thường**, đó là lý do Part 23 nhắc tới Scout cho full-text search.
- MySQL tự đánh index cho cột khoá ngoại; SQLite thì không, nên hãy chủ động `->index()` khi cần.
- Xem kế hoạch trên MySQL/PostgreSQL bằng `EXPLAIN SELECT ...` (trong client SQL hoặc `DB::select('EXPLAIN SELECT ...')`); tìm dòng quét toàn bảng (`type: ALL` ở MySQL). Xem các index hiện có: `php artisan db:table posts` hoặc `Schema::getIndexes('posts')`.

SQL.8 Transaction trong một đoạn

Transaction gom nhiều câu lệnh thành một đơn vị: **hoặc tất cả thành công (`COMMIT`), hoặc không gì thay đổi (`ROLLBACK`)**. Ví dụ chuyển tiền: trừ ở tài khoản A rồi cộng ở tài khoản B; nếu bước hai lỗi mà không có transaction thì tiền biến mất. Trong Laravel, `DB::transaction(function () { ... })` tự commit khi chạy xong và tự rollback rồi ném lại exception khi có lỗi. Bẫy hay gặp (job bắn trong transaction chạy trước khi commit) được học kỹ ở Part 13.

SQL.9 SQL injection — chỉ cần nhớ ba quy tắc

1. Giá trị từ người dùng luôn đi qua **tham số** (`?` hoặc Query Builder/Eloquent). Docs Laravel: query builder dùng PDO parameter binding để chống SQL injection, bạn không cần làm sạch chuỗi.
2. **Tên cột, tên bảng, hướng sắp xếp** không tham số hoá được. Docs: "PDO does not support binding column names. Therefore, you should never allow user input to dictate the column names referenced by your queries, including order by columns." Luôn đi qua **danh sách cho phép (allowlist)**.
3. **Không bao giờ** nội suy `$biến` hay nối `.` vào câu SQL thô. Với `DB::raw`, `whereRaw`, `selectRaw`, `orderByRaw`, truyền giá trị qua tham số thứ hai: `whereRaw('views > ?', [$min])`. Docs cảnh báo: Laravel không thể đảm bảo query dùng raw expression an toàn trước SQL injection.

Chương 12.11 của `php-bai-tap.pdf` cho bạn thấy đầu vào `' OR '1'='1` lấy ra **cả 3 user** khi ghép chuỗi, nhưng **0 dòng** khi tham số hoá. Tổng hợp bảo mật: xem **PART SEC**.

SQL.10 Bảng tra nhanh

Việc	SQL	Query Builder / Eloquent
Lọc, sắp xếp	`WHERE ... ORDER BY ...`	`where()`, `orderBy()`
Phân trang	`LIMIT n OFFSET m`	`paginate()`, `cursorPaginate()` (Part 12)
Đếm / tổng theo nhóm	`GROUP BY ... HAVING ...`	`groupBy()`, `having()`, `withCount()`
Ghép bảng	`JOIN`, `LEFT JOIN`	`join()`, `leftJoin()`, `with()`
Dòng không có quan hệ	`LEFT JOIN ... WHERE x.id IS NULL`	`doesntHave()`
Tìm nhanh theo cột	`CREATE INDEX`	`$table->index()` trong migration
Tất cả hoặc không gì	`BEGIN` / `COMMIT` / `ROLLBACK`	`DB::transaction()`

Việc cần làm: (không có đáp án chép sẵn trong phần này; kết quả mong đợi để bạn tự đối chiếu, lời giải chi tiết nằm ở **Chương 12** của `php-bai-tap.pdf`)

1. Dựng schema và dữ liệu mẫu như SQL.2–SQL.3 (bằng `php artisan db` hoặc bằng PDO như bài 12.1). Kiểm tra: `3` user, `5` post, `6` comment
2. Viết SQL lấy 3 bài có nhiều view nhất. Mong đợi: `Học Laravel 300`, `Docker cơ bản 150`, `Học PHP 120`
3. Viết SQL lấy các bài chưa xuất bản bằng `IS NULL`, rồi thử `= NULL` và giải thích vì sao ra 0 dòng. Mong đợi: `IS NULL` ra `Học SQL`
4. Tổng view và số bài của mỗi user. Mong đợi: user 1 có `3` bài, `500` view; user 2 có `2` bài, `195` view. Thêm `HAVING` để chỉ giữ user có tổng view lớn hơn 200: còn lại user 1
5. Liệt kê mọi user kèm số bài (kể cả 0). Mong đợi: `An 3`, `Bình 2`, `Chi 0`. Thử đổi `COUNT(p.id)` thành `COUNT(*)` và giải thích vì sao Chi ra 1
6. Tìm các bài chưa có comment. Mong đợi: `Học SQL` và `Redis là gì`
7. Cài đặt ba cách ở SQL.6 và đếm số query. Mong đợi: `6`, `2`, `1`
8. Tạo bảng 20000 dòng, chạy `EXPLAIN QUERY PLAN` trước và sau khi thêm index, và chỉ ra chữ `SCAN` đổi thành `SEARCH`

9. Viết hàm chuyển tiền trong transaction; chứng minh khi chuyển quá số dư thì số dư hai tài khoản **không đổi**
10. Trong project Laravel: sau khi làm migration ở Part 04, chạy `php artisan db:table posts` và `Schema::getIndexes('posts')`, xác nhận cột `user_id` có index/khoá ngoại

---

PART 04

Database: Migration, Eloquent Model, Factory & Seeder

Mục tiêu: Thay mảng cứng bằng database thật, hiểu Eloquent (Active Record) khác gì TypeORM Repository (Data Mapper), và biết bảo vệ ứng dụng khỏi lỗ hổng mass assignment.

Học trước khi vào Part 04: nếu bạn chưa quen `JOIN`, `GROUP BY`, khoá ngoại và index, hãy học **PART SQL** ngay phía trên (kèm Chương 12 của `php-bai-tap.pdf` có đáp án). Migration, quan hệ và N+1 ở Part 04/08 sẽ dễ hiểu hơn nhiều khi bạn đã biết chúng sinh ra SQL gì.

4.1 Migration — "git cho cấu trúc database"

Migration là file PHP mô tả một thay đổi cấu trúc bảng, chạy theo thứ tự thời gian. Nhờ nó, đồng đội chỉ cần `git pull` + `php artisan migrate` là có database giống hệt bạn.

```
php artisan make:migration create_posts_table
```

```
public function up(): void
{
    Schema::create('posts', function (Blueprint $table) {
        $table->id();                                  // BIGINT UNSIGNED AUTO_INCREMENT PK
        $table->string('title');
        $table->text('body');
        $table->timestamp('published_at')->nullable();
        $table->timestamps();                          // tạo created_at + updated_at
    });
}

public function down(): void
{
    Schema::dropIfExists('posts');                     // dùng khi rollback
}
```

Lệnh	Tác dụng
`php artisan migrate`	Chạy các migration chưa chạy
`php artisan migrate:rollback`	Lùi lại batch migration gần nhất
`php artisan migrate:fresh --seed`	Xoá sạch mọi bảng, chạy lại từ đầu + seed. Chỉ dùng ở local
`php artisan migrate:status`	Xem migration nào đã chạy

`php artisan db:show` / `db:table posts`	Xem tổng quan DB / cấu trúc 1 bảng ngay trong terminal

Nguyên tắc: migration đã chạy trên production thì không sửa lại file cũ — phải tạo migration mới để thay đổi. Sửa file cũ sẽ khiến máy người khác (đã chạy rồi) không bao giờ nhận được thay đổi đó. Đây là điểm khác hẳn `synchronize: true` của TypeORM mà bạn dùng bên NestJS — Laravel không bao giờ "tự đoán" schema.

4.2 Model Eloquent & quy ước ngầm

```
php artisan make:model Post -mf      # -m: kèm migration, -f: kèm factory
php artisan make:model Post -a       # -a: kèm
migration+factory+seeder+policy+controller+requests
```

Quy ước	Mặc định Eloquent hiểu là
Tên bảng	snake_case số nhiều của tên class: `Post` → `posts`
Khoá chính	Cột `id`, kiểu số tự tăng
Timestamps	Tự quản lý `created_at` / `updated_at`
Khoá ngoại	`Comment` thuộc `Post` → cột `post_id`

Muốn đổi quy ước, Laravel 13 dùng PHP attribute thay cho các thuộc tính protected kiểu cũ:

```
use Illuminate\Database\Eloquent\Attributes\Table;

#[Table('blog_posts', key: 'post_id')]
class Post extends Model { }

// Các attribute khác đã kiểm chứng trong docs 13.x:
#[Table(key: 'uuid', keyType: 'string', incrementing: false)]   // khoá chính UUID
#[WithoutTimestamps]                                             // bảng không có
created_at/updated_at
#[Connection('mysql')]                                           // model dùng connection
khác
```

4.3 Mass assignment — bảo mật, không phải thủ tục

Muốn dùng `Post::create($data)`, bạn phải khai báo field nào được phép gán hàng loạt. Đây là cơ chế chống việc kẻ tấn công gửi thêm field lạ (ví dụ `is_admin=1`, `user_id=999`) rồi ghi thẳng vào DB.

```php
use Illuminate\Database\Eloquent\Attributes\Fillable;

#[Fillable(['title', 'body', 'published_at'])]     // cú pháp Laravel 13
class Post extends Model
{
    protected function casts(): array               // ép kiểu khi đọc/ghi
    {
        return [
            'published_at' => 'datetime',           // string trong DB → Carbon object
trong PHP
            'is_featured'  => 'boolean',            // 0/1 → true/false
            'status'       => PostStatus::class,    // enum backed (P.6) — DB lưu 'draft',
PHP nhận enum
        ];
    }
}
```

Cú pháp cũ vẫn chạy: `protected $fillable = ['title','body'];` vẫn hoạt động bình thường và bạn sẽ gặp nó ở 99% tutorial trên mạng. Attribute `#[Fillable]` là cách viết mới mà docs Laravel 13 dùng — chọn một kiểu và dùng nhất quán trong cả project.

Bổ sung sau audit — bật "strict mode" cho Eloquent khi dev. Docs khuyến nghị đặt trong `AppServiceProvider::boot()` :

```php
Model::preventLazyLoading(! $this->app->isProduction());          // bắt N+1 (Part 08)
Model::preventSilentlyDiscardingAttributes($this->app->isLocal());  // field không nằm
trong fillable → throw thay vì âm thầm bỏ
```

Không có 2 dòng này, việc quên thêm field vào `fillable` sẽ khiến `create()` "thành công" nhưng cột đó rỗng — bug rất khó tìm.

4.4 CRUD với Eloquent

```php
// ĐỌC
Post::all();                                    // mọi bản ghi — cẩn thận với bảng lớn
Post::find(1);                                  // không thấy → null
Post::findOrFail(1);                            // không thấy → ModelNotFoundException → tự
thành 404
Post::where('title', 'like', '%nest%')->get();
Post::latest()->paginate(15);                   // phân trang, kèm săñ meta/links (Part 12)

// TẠO
$post = Post::create(['title' => 'Hello', 'body' => '...']);

// SỬA
$post->update(['title' => 'Updated']);

// XOÁ
$post->delete();

// UPSERT / firstOrCreate — hay dùng khi import
Post::firstOrCreate(['slug' => 'hello'], ['title' => 'Hello', 'body' => '...']);
```

Active Record vs Data Mapper: Eloquent là Active Record — bản thân object `$post` biết cách tự lưu/xoá chính nó (`$post->save()`). TypeORM bên NestJS là Data Mapper — entity chỉ là dữ liệu thuần, phải nhờ `repository.save(entity)` . Active Record viết nhanh hơn nhiều; đổi lại model gánh cả logic truy vấn nên dễ phình to — vì vậy dự án lớn hay tách bớt sang Service/Action/Query class (Part 21).

4.5 Eloquent hay Query Builder — 2 cách viết query, chọn đúng lúc nào

Laravel cho **2 cách viết query** hoàn toàn khác nhau cho cùng một việc "lấy dữ liệu từ DB". Người mới hay phân vân không biết dùng cái nào — thực ra chọn dựa trên 1 câu hỏi duy nhất: **bạn có cần kết quả trả về là Model không?**

Cách 1 — Eloquent (`Post::where()`)

Trả về **Model instance** — có đầy đủ quan hệ, accessor/cast, mass assignment protection, tự bắn model event (`creating` , `updating` ...). Đây là cách mặc định, dùng cho hầu hết mọi việc.

```
Post::where('title', 'like', '%nest%')
    ->where('published_at', '<=', now())
    ->orderByDesc('id')
    ->paginate(15);

// Lọc/đếm qua quan hệ — Query Builder KHÔNG có API tương đương gọn như vậy
Category::withCount('products')->get();                 // mỗi category có $category->products_count
Post::whereHas('comments', fn ($q) =>
    $q->where('is_approved', true))->get();              // chỉ lấy post có ít nhất 1 comment đã duyệt
```

Cách 2 — Query Builder (`DB::table()`)

Trả về **object/array thuần** (`stdClass`), không qua Model — không có quan hệ, không bắn event, không áp `$fillable` . Đổi lại: nhẹ hơn (không tốn công dựng Model), và join được nhiều bảng tự do mà không cần khai quan hệ Eloquent trước.

```
use Illuminate\Support\Facades\DB;

DB::table('users')
    ->join('orders', 'users.id', '=', 'orders.user_id')
    ->select('users.name', DB::raw('COUNT(orders.id) as total_orders'))
    ->groupBy('users.id', 'users.name')
    ->get();
```

Đã đối chiếu docs Query Builder 13.x: cú pháp `DB::table('users')->where('name', 'like', 'T%')->get()` , `whereBetween('votes', [1, 100])` , `join('contacts', 'users.id', '=', 'contacts.user_id')` đều khớp nguyên văn ví dụ chính thức. Docs mô tả Query Builder là "convenient, fluent interface" hoạt động độc lập với Eloquent — không phải bản "cấp thấp hơn" của Eloquent, mà là 1 API riêng thao tác thẳng lên bảng.

Nhu cầu	Cách 1: Eloquent	Cách 2: Query Builder

Cần gọi tiếp `->update()`, `->delete()`, quan hệ trên kết quả	✅ Bắt buộc — Query Builder trả object thường, không có các method này	❌ Không có
Đếm/lọc theo quan hệ (`withCount`, `whereHas`)	✅ Có sẵn, gọn	Phải tự viết `join` + `groupBy`
Mass assignment an toàn (`$fillable`), model event, cast	✅ Tự áp dụng	❌ Bỏ qua hoàn toàn — `DB::table()->insert()` ghi thẳng, không qua Model
JOIN nhiều bảng thống kê/báo cáo, không map thành quan hệ Eloquent	Gượng ép, phải tự thêm quan hệ giả	✅ `join()` tự do, không ràng buộc
Chỉ đọc dữ liệu để hiển thị/export, không cần thao tác tiếp	Tốn công dựng Model không cần thiết	✅ Nhẹ hơn, nhanh hơn

Quy tắc thực dụng: mặc định luôn dùng Eloquent. Chỉ rẽ sang `DB::table()` khi bạn chắc chắn không cần quan hệ/event/cast của Model cho kết quả đó — điển hình nhất là báo cáo, thống kê, export.

4.6 Factory & Seeder — dữ liệu giả

```php
// database/factories/PostFactory.php
public function definition(): array
{
    return [
        'title' => fake()->sentence(),
        'body'  => fake()->paragraphs(3, true),
    ];
}

// Dùng ở seeder / test / tinker
Post::factory()->count(30)->create();
Post::factory()->has(Comment::factory()->count(3))->create();   // kèm quan hệ (Part 08)
```

Việc cần làm:

1. Tạo model + migration + factory: `php artisan make:model Post -mf`
2. Viết schema bảng posts (title, body, timestamps), chạy `php artisan migrate`
3. Khai báo fillable + `casts()` cho Post
4. Bật `preventLazyLoading` + `preventSilentlyDiscardingAttributes` trong `AppServiceProvider::boot()`
5. Viết `PostFactory`, seed 30 bản ghi, kiểm tra bằng `php artisan tinker`: gõ `Post::count()`
6. Chuyển toàn bộ 5 action trong Controller sang dùng Eloquent thật (dùng `paginate` cho index)
7. Thử bỏ field ra khỏi fillable rồi gọi `create()` — quan sát exception nhờ strict mode ở bước 4

PART 05

Validation với Form Request

Mục tiêu: Đảm bảo dữ liệu client gửi lên luôn đúng định dạng trước khi chạm tới Controller — và trả lỗi theo đúng chuẩn 422 Unprocessable Content.

5.1 Hai cách validate

```
// Cách 1 — validate ngay trong controller (nhanh, cho case đơn giản)
public function store(Request $request)
{
    $data = $request->validate([
        'title' => ['required', 'string', 'max:255'],
        'body'  => ['required', 'string'],
    ]);

    return Post::create($data);
}
```

```
# Cách 2 — Form Request (KHUYÊN NGHỊ cho API): tách rule ra class riêng
php artisan make:request StorePostRequest
```

```
namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StorePostRequest extends FormRequest
{
    public function authorize(): bool
    {
        return true;                        // false → tự trả 403 Forbidden (Part 19 sẽ
dùng thật)
    }

    public function rules(): array
    {
        return [
            'title' => ['required', 'string', 'max:255', 'unique:posts,title'],
            'body'  => ['required', 'string', 'min:10'],
        ];
    }

    public function messages(): array
    {
        return ['title.required' => 'Tiêu đề là bắt buộc.'];
    }
}
```

5.2 Cơ chế: nó chạy lúc nào?

Bạn chỉ cần type-hint Form Request vào controller. Container thấy class đó kế thừa `FormRequest` nên tự chạy validate ngay khi resolve tham số — tức là trước khi dòng code đầu tiên trong thân method chạy:

```
public function store(StorePostRequest $request)
{
    // Tới được đây = dữ liệu CHẮC CHẮN đã hợp lệ
    return Post::create($request->validated());
}
```

Thứ tự thực tế: `authorize()` → `rules()` → (sai thì 422 / không có quyền thì 403) → thân method.

5.3 Format lỗi 422 mà Laravel trả về

Nếu request có header `Accept: application/json` (hoặc là XHR), Laravel trả về đúng cấu trúc sau — hãy nhớ nó vì frontend sẽ dựa vào đây:

```
HTTP/1.1 422 Unprocessable Content

{
    "message": "The title field is required. (and 1 more error)",
    "errors": {
        "title": ["The title field is required."],
        "body":  ["The body field must be at least 10 characters."]
    }
}
```

Bẫy kinh điển: nếu không gửi header `Accept: application/json`, Laravel coi đây là request từ browser nên trả về redirect 302 kèm lỗi trong session thay vì JSON 422. Rất nhiều người tưởng API bị lỗi trong khi chỉ là thiếu header. Part 07 sẽ chỉ cách ép mọi request `/api/*` luôn nhận JSON.

5.4 Lấy dữ liệu đã validate

Cách gọi	Trả về
`$request->validated()`	Mảng chỉ chứa các field có trong rules — field lạ bị loại bỏ
`$request->safe()->only([...])`	Chỉ lấy một số field trong dữ liệu đã validate
`$request->safe()->except([...])`	Loại một số field
`$request->all()`	Toàn bộ input thô, chưa lọc — không nên đưa thẳng vào `create()`

Đối chiếu NestJS: `validated()` chính là tương đương của `whitelist: true` trong `ValidationPipe` — chỉ nhận field đã khai báo. Còn muốn giống `forbidNonWhitelisted: true` (có field lạ thì báo lỗi luôn), bên Laravel bạn thêm rule `'field_la' => 'prohibited'` hoặc kiểm tra trong `after()`.

5.5 Rule hay dùng cho API

Rule	Ý nghĩa
`required` / `nullable`	Bắt buộc có / cho phép null
`sometimes`	Chỉ validate NẾU field xuất hiện — dùng cho PATCH / UpdateRequest
`string`, `integer`, `boolean`, `array`	Kiểm tra kiểu
`max:255`, `min:10`, `between:1,10`	Giới hạn độ dài / giá trị
`email`, `url`, `date`, `uuid`	Định dạng
`unique:posts,title`	Không trùng trong bảng. Khi update phải bỏ qua chính bản ghi đó: `Rule::unique('posts')->ignore($this->post)`
`exists:posts,id`	Phải tồn tại trong bảng — dùng cho khoá ngoại
`confirmed`	password phải khớp `password_confirmation`
`Rule::enum(PostStatus::class)`	Giá trị phải là 1 case của enum
`File::image()->max(2048)`	Validate file upload (Part 18)

Việc cần làm:

1. Tạo `StorePostRequest` và `UpdatePostRequest`
2. `UpdatePostRequest` dùng `sometimes` để mọi field đều optional (tương đương `PartialType` bên NestJS)
3. Type-hint chúng vào `store()` / `update()`, dùng `validated()` khi tạo/cập nhật
4. Test: gửi body thiếu field → xác nhận nhận đúng 422 với cấu trúc `{message, errors}`
5. Test: gửi body thiếu header `Accept: application/json` → quan sát response 302 để hiểu cái bẫy ở mục 5.3
6. Thêm rule `unique` cho title, xử lý case update chính bản ghi đó mà không bị báo trùng

---

PART 06

API Resources — tầng transform response

Mục tiêu: Không bao giờ trả model thô ra ngoài. Có một tầng riêng quyết định "API này công khai những field nào, định dạng ra sao".

6.1 Vì sao không return thẳng model?

Trả `return $post;` thì Laravel tự serialize toàn bộ cột trong bảng thành JSON. Ba vấn đề: (1) rò rỉ dữ liệu — thêm cột `internal_note` vào DB là API tự động phơi ra; (2) API vỡ hợp đồng — đổi tên cột trong DB làm

frontend chết theo; (3) không kiểm soát được định dạng (ngày tháng, field tính toán). API Resource giải quyết cả ba.

```
php artisan make:resource PostResource
```

```
namespace App\Http\Resources;

use Illuminate\Http\Request;
use Illuminate\Http\Resources\Json\JsonResource;

class PostResource extends JsonResource
{
    public function toArray(Request $request): array
    {
        return [
            'id'         => $this->id,
            'title'      => $this->title,
            'body'       => $this->body,
            'created_at' => $this->created_at?->toIso8601String(),
        ];
    }
}
```

Vì sao gọi được `$this->id` khi `$this` là Resource chứ không phải Post? Vì `JsonResource` proxy mọi lời gọi property/method xuống model bên dưới. Model thật nằm ở `$this->resource`.

6.2 Dùng trong Controller

```
// 1 bản ghi
return new PostResource($post);
return $post->toResource();                              // cách gọn (Laravel 12+), tự tìm
PostResource

// nhiều bản ghi
return PostResource::collection(Post::all());
return Post::latest()->paginate(15)->toResourceCollection();

// kèm status code & header
return (new PostResource($post))->response()->setStatusCode(201);
```

Đã đối chiếu docs: `toResource()` tự tìm class theo quy ước — model `App\Models\Post` → tìm `App\Http\Resources\PostResource` (hoặc `Post`) ở namespace `Http\Resources` gần model nhất; có thể chỉ định tường minh `$post->toResource(CustomPostResource::class)`.

6.3 Data wrapping — response trông thế nào

Mặc định Laravel bọc kết quả trong key `data`:

```
{ "data": { "id": 1, "title": "Hello" } }
```

Khi phân trang qua Resource collection, response luôn có thêm `links` và `meta`:

```
{
    "data": [ ... ],
    "links": { "first": "...?page=1", "last": "...?page=4", "prev": null, "next": "...?
page=2" },
    "meta": { "current_page": 1, "from": 1, "last_page": 4, "per_page": 15, "to": 15,
"total": 54 }
}
```

Muốn bỏ lớp `data`, gọi `JsonResource::withoutWrapping()` trong `AppServiceProvider::boot()`. Lưu ý: response phân trang vẫn giữ `data` vì cần chỗ đặt meta/links.

> **Khuyến nghị:** giữ nguyên wrapping mặc định. Một API mà "khi thì trả object, khi thì trả object bọc data" là nguồn bug bất tận cho frontend. Chọn một kiểu và giữ nhất quán toàn bộ endpoint — đây cũng là bài học bạn đã rút ra khi làm `TransformInterceptor` bên NestJS.

6.4 Field có điều kiện

```
return [
    'id'    => $this->id,
    'title' => $this->title,

    // chỉ trả khi thoả điều kiện
    'secret' => $this->when($request->user()?->isAdmin(), $this->secret),

    // chỉ trả khi relation ĐÃ được eager load → tránh N+1 (xem Part 08)
    'comments' => CommentResource::collection($this->whenLoaded('comments')),

    // chỉ trả khi đã loadCount('comments')
    'comments_count' => $this->whenCounted('comments'),
];
```

6.5 JSON:API Resource (mới ở Laravel 13)

Laravel 13 bổ sung `JsonApiResource` — tạo response đúng chuẩn jsonapi.org: có `type`, `attributes`, `relationships`, hỗ trợ `?include=` và sparse fieldsets `?fields[posts]=title`, tự set header `application/vnd.api+json`.

```
php artisan make:resource PostResource --json-api
```

```php
use Illuminate\Http\Resources\JsonApi\JsonApiResource;

class PostResource extends JsonApiResource
{
    public $attributes = ['title', 'body', 'created_at'];

    public $relationships = ['author', 'comments'];
}
// Response: { "data": { "id": "1", "type": "posts", "attributes": { "title": "...",
"body": "..." } } }
```

> **Đã đối chiếu docs:** class chính xác là `Illuminate\Http\Resources\JsonApi\JsonApiResource` ; muốn kiểm soát chi tiết thì override `toAttributes(Request $request): array` (có thể trả closure để lazy-evaluate field nặng). Relationship chỉ được serialize khi client yêu cầu qua `?include=` . Docs gợi ý dùng kèm package Spatie Query Builder để parse filter/sort phía request.

Có nên dùng JSON:API ngay? Với crash course này thì không cần — hãy làm `JsonResource` thường cho vững đã. Biết là nó tồn tại, để khi gặp dự án yêu cầu chuẩn JSON:API thì không phải tự viết tay.

> **Việc cần làm:**
>
> 1. Tạo `PostResource` , chỉ expose: id, title, body, created_at (ISO 8601)
> 2. Sửa cả 5 action trong Controller để trả về Resource thay vì model thô
> 3. Endpoint index: dùng `paginate(15)` + resource collection, kiểm tra response có `meta` và `links`
> 4. Thêm 1 cột "nội bộ" vào bảng posts (vd `internal_note`), xác nhận nó KHÔNG lộ ra API nhờ Resource
> 5. Đảm bảo status code đúng: store → 201, destroy → 204

PART 07

Middleware & Exception Handling

> **Mục tiêu:** Tự viết middleware, và chuẩn hoá toàn bộ response lỗi của API về một format duy nhất — đây là thứ phân biệt API "tập viết" với API dùng được thật.

7.1 Viết middleware

```bash
php artisan make:middleware LogRequests
```

```php
public function handle(Request $request, Closure $next): Response
{
    $start = microtime(true);

    $response = $next($request);                 // đi sâu vào app

    Log::info('API', [
        'method' => $request->method(),
        'uri'    => $request->path(),
        'status' => $response->status(),
        'ms'     => round((microtime(true) - $start) * 1000),
    ]);

    return $response;                            // PHẢI return, nếu không request bị
treo
}
```

Middleware là pipeline hai chiều: code trước `$next()` chạy khi request đi vào, code sau `$next()` chạy khi response đi ra. Nó gánh vai trò của cả Middleware + Guard + Interceptor bên NestJS. Muốn làm việc nặng sau khi đã trả response (ghi log ra dịch vụ ngoài...), thêm method `terminate(Request $request, Response $response)` — Laravel gọi nó sau khi response đã gửi tới client (cần FastCGI).

7.2 Đăng ký middleware

```php
// bootstrap/app.php
->withMiddleware(function (Middleware $middleware) {
    $middleware->append(LogRequests::class);                    // global, chạy cho mọi
request

    $middleware->api(prepend: ['throttle:api']);                // thêm vào nhóm api

    $middleware->alias(['admin' => EnsureUserIsAdmin::class]);  // đặt tên ngắn
})
```

Sau khi có alias, gắn vào route: `Route::delete(...)->middleware('admin')`. Middleware nhận tham số qua dấu `:` — ví dụ `'role:editor,publisher'` tương ứng `handle($request, $next, ...$roles)`.

> **Đã đối chiếu docs Middleware 13.x — nhóm mặc định:** nhóm `api` chỉ có đúng 1 middleware `SubstituteBindings`; `throttle:api` và `EnsureFrontendRequestsAreStateful` (Sanctum SPA) đều **bị comment sẵn**. Nghĩa là rate limit KHÔNG bật sẵn — muốn có thì tự thêm như trên (chi tiết Part 10). Nhóm `web` ở Laravel 13 dùng `PreventRequestForgery` (tên mới của middleware CSRF, có kiểm tra origin) — API stateless không dùng tới.

7.3 Exception Handling — chuẩn hoá lỗi JSON

Mặc định Laravel đã trả lỗi dạng JSON khi request có `Accept: application/json`: 404 cho model không tìm thấy, 422 cho validation, 401 khi chưa đăng nhập. Nhưng format mỗi loại một kiểu. Muốn thống nhất, cấu hình trong `withExceptions()`:

```php
// bootstrap/app.php
->withExceptions(function (Exceptions $exceptions) {

    // 1) Mọi request /api/* đều được coi là muốn nhận JSON (hết bẫy 302 ở Part 05)
    $exceptions->shouldRenderJsonWhen(fn (Request $request, Throwable $e) =>
        $request->is('api/*') || $request->expectsJson());

    // 2) Đổi thông điệp 404 cho API (mặc định message rỗng, khó debug)
    $exceptions->render(function (NotFoundHttpException $e, Request $request) {
        if ($request->is('api/*')) {
            return response()->json(['message' => 'Resource not found.'], 404);
        }
    });

    // 3) Không report một số exception vô hại (giảm rác log)
    $exceptions->dontReport([PostAlreadyPublishedException::class]);
})
```

Bài học từ audit dự án NestJS của bạn: khi tự viết filter/handler lỗi, tuyệt đối đừng bọc lồng payload lỗi. Nếu response thành `{"message": {"message": "...", "statusCode": 404}}` thì client phải đọc `body.message.message` — sai chuẩn và rất dễ gây bug. Format lỗi nên phẳng và giống nhau ở mọi status code, ví dụ: `{ message, errors?, status }`.

7.4 Exception nghiệp vụ tự viết

```bash
php artisan make:exception PostAlreadyPublishedException
```

```php
class PostAlreadyPublishedException extends Exception
{
    // Có method render() → Laravel tự gọi, không cần khai gì trong bootstrap/app.php
    public function render(Request $request): JsonResponse
    {
        return response()->json([
            'message' => 'Bài viết này đã được publish trước đó.',
        ], 409);                            // 409 Conflict
    }
}
```

Trong service chỉ cần `throw new PostAlreadyPublishedException();` — không cần try/catch ở controller. Ngoài `render()`, exception còn có thể khai `report()` để tự quyết cách ghi log, và `context()` để thêm dữ liệu vào log.

7.5 Các exception → status code Laravel tự map

Exception	Status	Sinh ra khi
`ValidationException`	422	Form Request / `validate()` thất bại
`ModelNotFoundException`	404	`findOrFail()`, route model binding không thấy
`NotFoundHttpException`	404	Không route nào khớp

`AuthenticationException`	401	Middleware `auth` chặn
`AuthorizationException`	403	`authorize()` trả false / policy từ chối (Part 19)
`MethodNotAllowedHttpException`	405	Đúng URI nhưng sai HTTP verb
`ThrottleRequestsException`	429	Vượt rate limit

Việc cần làm:

1. Viết `LogRequests` middleware (log method + uri + status + thời gian xử lý), đăng ký global
2. Viết 1 middleware có tham số (vd `EnsureHeader:X-Client-Version`), đặt alias và gắn lên vài route
3. Cấu hình `withExceptions()`: dùng `shouldRenderJsonWhen` cho `api/*` và chuẩn hoá message 404
4. Tự viết 1 exception nghiệp vụ có `render()` trả 409, ném từ service và test
5. Gọi thử đủ 7 tình huống trong bảng 7.5 bằng curl, ghi lại status + body thực tế nhận được
6. Đặt `APP_DEBUG=false` rồi gây lỗi 500 — so sánh response với lúc true để thấy vì sao production bắt buộc false

PART 08

Quan hệ dữ liệu (Relationships) & vấn đề N+1

Mục tiêu: Thêm entity Comment liên kết với Post, và hiểu N+1 query — lỗi hiệu năng phổ biến nhất của mọi ORM.

Nền SQL của N+1: ba cách lấy "bài kèm tác giả" (N+1 / `IN` / `JOIN`) và số query chạy thật (6 / 2 / 1) được minh hoạ ở **SQL.6**. Đọc lại đó nếu thấy `with()` như phép màu.

8.1 Khai báo quan hệ 1-N

```
// migration của bảng comments
$table->foreignId('post_id')->constrained()->cascadeOnDelete();
// constrained(): tự suy ra bảng posts + tạo khoá ngoại
// cascadeOnDelete(): xoá post thì comment tự xoá theo

// app/Models/Post.php — một Post có NHIỀU Comment
public function comments(): HasMany
{
    return $this->hasMany(Comment::class);
}

// app/Models/Comment.php — một Comment thuộc về MỘT Post
public function post(): BelongsTo
{
    return $this->belongsTo(Post::class);
}
```

Quan hệ	Khoá ngoại nằm ở	Ví dụ
`hasMany`	Bảng con	Post → nhiều Comment
`belongsTo`	Chính bảng này	Comment → một Post
`hasOne`	Bảng con	User → một Profile
`belongsToMany`	Bảng trung gian (pivot)	Post ↔ Tag
`morphMany` / `morphTo`	Bảng con có cột `*_type` + `*_id`	Comment gắn được cho cả Post lẫn Video

8.2 Truy vấn quan hệ

```
$post->comments;                  // property → chạy query LÚC gọi, trả Collection
$post->comments();                // method → trả query builder, còn nối được điều kiện

$post->comments()->where('is_approved', true)->latest()->get();

$post->comments()->create(['body' => 'Hay quá']);   // tự gán post_id
```

8.3 N+1 — lỗi hiệu năng phải hiểu

```
// ✘ SAI — sinh 1 + N query
$posts = Post::all();                          // 1 query lấy 30 post
foreach ($posts as $post) {
    echo $post->comments->count();             // + 30 query nữa (mỗi post 1 query)
}
// → tổng 31 query cho một trang danh sách

// ✔ ĐÚNG — eager loading, chỉ 2 query
$posts = Post::with('comments')->get();
// query 1: select * from posts
// query 2: select * from comments where post_id in (1,2,3,...)

// ✔ Chỉ cần ĐẾM thì đừng load hết dữ liệu
$posts = Post::withCount('comments')->get();   // có $post->comments_count
```

Bắt N+1 tự động khi dev: `Model::preventLazyLoading(! $this->app->isProduction());` trong `AppServiceProvider::boot()` (đã bật ở Part 04). Từ đó bất kỳ chỗ nào quên `with()` sẽ throw exception ngay ở local thay vì âm thầm chạy chậm ở production. Đây là một trong những setting đáng bật nhất của Laravel.

Ghép với Part 06: Resource dùng `whenLoaded('comments')` — nếu controller quên `with('comments')` thì field đó đơn giản không xuất hiện, chứ không âm thầm bắn thêm N query. Hai tính năng này thiết kế để đi cùng nhau.

8.4 Eager load có điều kiện & lồng nhau

```php
Post::with(['comments' => fn ($q) => $q->where('is_approved', true)])->get();

Post::with('comments.author')->get();          // lồng 2 cấp

$post->load('comments');                        // load sau khi đã có model
$post->loadMissing('comments');                 // chỉ load nếu chưa có

Post::has('comments')->get();                   // chỉ post CÓ comment
Post::doesntHave('comments')->get();            // chỉ post KHÔNG có comment
Post::whereHas('comments', fn ($q) => $q->where('is_approved', true))->get();
```

8.5 Route lồng & scope tự động

```php
// routes/api.php — comment thuộc post
Route::apiResource('posts.comments', CommentController::class)->shallow();
// → GET  /api/posts/{post}/comments      index
//   POST /api/posts/{post}/comments      store
//   GET  /api/comments/{comment}         show    (shallow: con có id riêng thì không cần
lặp lại cha)

// Controller
public function store(StoreCommentRequest $request, Post $post)
{
    $comment = $post->comments()->create($request->validated());
    return $comment->toResource()->response()->setStatusCode(201);
}
```

Việc cần làm:

1. Tạo model + migration Comment (post_id, body), khai `foreignId()->constrained()->cascadeOnDelete()`
2. Khai `hasMany` / `belongsTo` ở hai model
3. Endpoint `GET /api/posts/{post}` trả post kèm comments (dùng `with()` + `whenLoaded()`)
4. Endpoint lồng: `GET /api/posts/{post}/comments` và `POST /api/posts/{post}/comments`
5. Cố tình bỏ `with()` để thấy exception của `preventLazyLoading`
6. Dùng `withCount('comments')` cho endpoint index và trả qua `whenCounted()`
7. Dùng `DB::listen()` (Part 13) hoặc Telescope để đếm số query thực tế của endpoint index — chứng minh đã hết N+1

PART 09

Authentication với Laravel Sanctum

> **Mục tiêu:** Hiểu trọn luồng đăng ký → đăng nhập → cấp token → bảo vệ route → đăng xuất, và biết vì sao password phải hash chứ không được mã hoá.

9.1 Sanctum là gì — và không phải là gì

Sanctum đã được cài sẵn khi bạn chạy `php artisan install:api` ở Part 01. Nó giải quyết 2 bài toán tách biệt:

- **API token** (personal access token, giống token của GitHub): token là chuỗi random, được hash SHA-256 rồi lưu vào bảng `personal_access_tokens`. Client gửi kèm header `Authorization: Bearer <token>`. → Đây là phần ta dùng.
- **SPA authentication**: dùng session cookie thay vì token, cho frontend cùng domain. Không dùng trong crash course này.

> **Sanctum khác JWT thế nào?** Token Sanctum là opaque (chuỗi vô nghĩa, phải tra DB mới biết của ai) — nên thu hồi được ngay lập tức bằng cách xoá dòng trong DB. JWT là self-contained (chứa payload, verify bằng chữ ký, không cần tra DB) — nhanh hơn nhưng không thu hồi được trước khi hết hạn. Với API thường, Sanctum là lựa chọn mặc định đúng. Cần OAuth2 đầy đủ thì mới dùng Laravel Passport.

9.2 Chuẩn bị model User

```php
use Laravel\Sanctum\HasApiTokens;

class User extends Authenticatable
{
    use HasApiTokens, HasFactory, Notifiable;     // thêm HasApiTokens
}
```

Trait `HasApiTokens` mang lại: `createToken()`, quan hệ `tokens()`, `currentAccessToken()`, `tokenCan()`. Trait `Notifiable` (có sẵn) sẽ dùng ở Part 20.

9.3 Luồng đầy đủ

```
1. POST /api/register    Validate → hash password → tạo user → (tuỳ chọn) cấp token luôn
        ▼
2. POST /api/login       Tìm user theo email → Hash::check() so password → khớp thì
createToken()
                         → trả plainTextToken
        ▼
3. Client lưu token      Mọi request sau gửi header Authorization: Bearer <token>
        ▼
4. auth:sanctum          Middleware hash token nhận được, tra bảng personal_access_tokens
                         → hợp lệ thì gán user vào $request->user(); sai/hết hạn → 401
        ▼
5. POST /api/logout      $request->user()->currentAccessToken()->delete() → token đó chết
ngay
```

9.4 Code các endpoint

```php
// Đăng nhập — cách docs Laravel khuyến nghị
public function login(LoginRequest $request)
{
    $user = User::where('email', $request->email)->first();

    if (! $user || ! Hash::check($request->password, $user->password)) {
        throw ValidationException::withMessages([
            'email' => ['The provided credentials are incorrect.'],
        ]);                              // → 422 với format lỗi chuẩn
    }

    return [
        'token' => $user->createToken($request->device_name)->plainTextToken,
    ];
}
```

Vì sao báo lỗi chung "credentials are incorrect" thay vì "email không tồn tại"? Nếu phân biệt hai thông báo, kẻ tấn công có thể dò xem email nào đã đăng ký (user enumeration). Luôn trả cùng một message cho cả hai trường hợp.

```php
// routes/api.php
Route::post('/register', [AuthController::class, 'register']);
Route::post('/login',    [AuthController::class, 'login'])->middleware('throttle:login');

Route::middleware('auth:sanctum')->group(function () {
    Route::get('/me',      fn (Request $r) => $r->user());
    Route::post('/logout', [AuthController::class, 'logout']);
    Route::apiResource('posts', PostController::class)->except(['index', 'show']);
});
```

9.5 Hash password — nguyên lý

Laravel dùng bcrypt (mặc định) qua `Hash::make($password)`. Đặc điểm: **một chiều** — không có hàm nào "giải mã" hash về lại password, khi login ta hash lại rồi so 2 hash (`Hash::check()`); **có salt ngẫu nhiên** — cùng một password hash 2 lần cho 2 chuỗi khác nhau ⇒ chống rainbow table; **chậm có chủ đích** — cost factor cao làm brute-force tốn kém.

```
// Cách gọn nhất: dùng cast 'hashed' — Laravel tự hash khi gán
protected function casts(): array
{
    return ['password' => 'hashed'];           // User model mặc định đã có sẵn dòng này
}

// Nhờ vậy chỉ cần:
User::create(['password' => $request->password]);   // tự hash, KHÔNG lưu plain text
```

9.6 Token abilities, hết hạn, thu hồi

```
// Cấp token kèm quyền (giống scope của OAuth)
$user->createToken('mobile', ['post:create', 'post:update'])->plainTextToken;

// Kiểm tra quyền trong code
if ($request->user()->tokenCan('post:create')) { /* ... */ }

// Hết hạn: config/sanctum.php → 'expiration' => 525600 (phút) — mặc định null = không hết
hạn
// Dọn token hết hạn định kỳ (routes/console.php — Part 17):
Schedule::command('sanctum:prune-expired --hours=24')->daily();

// Thu hồi
$request->user()->currentAccessToken()->delete();   // logout thiết bị hiện tại
$user->tokens()->delete();                            // logout mọi thiết bị
```

9.7 Phân quyền: Policy (giới thiệu — chi tiết ở Part 19)

Authentication = "bạn là ai". Authorization = "bạn có được làm việc này không". Việc thứ hai dùng Policy:

```
php artisan make:policy PostPolicy --model=Post
```

```
public function update(User $user, Post $post): bool
{
    return $user->id === $post->user_id;          // chỉ tác giả được sửa
}

// Trong controller — 3 cách tương đương, chọn 1 và dùng nhất quán:
Gate::authorize('update', $post);                 // không có quyền →
AuthorizationException → 403
if ($request->user()->cannot('update', $post)) abort(403);

#[Authorize('update', 'post')]                    // attribute (Laravel 13) — gọn nhất
public function update(UpdatePostRequest $request, Post $post) { /* ... */ }
```

⚠ **Đã sửa so với bản gốc:** bản trước dùng `$this->authorize('update', $post)` trong controller. Từ Laravel 11, class `Controller` gốc **không còn** trait `AuthorizesRequests` mặc định — gọi `$this->authorize()` sẽ lỗi "method does not exist" trừ khi bạn tự thêm `use AuthorizesRequests;` vào base Controller. Cách docs 13.x dùng là `Gate::authorize()`, `$request->user()->can()/cannot()`, middleware `can:update,post`, hoặc attribute `#[Authorize]` như trên.

Việc cần làm:

1. Thêm trait `HasApiTokens` vào model User, chạy migration cho bảng `personal_access_tokens`
2. Viết `AuthController` với 3 action: register, login, logout (+ route `/me`)
3. Tạo `RegisterRequest` / `LoginRequest` (email `unique:users`, password `confirmed`, `min:8`)
4. Bọc các route ghi (store/update/destroy) trong `auth:sanctum`
5. Test bằng curl: gọi `POST /api/posts` không token → phải nhận 401; có token → 201
6. Mở bảng `personal_access_tokens` trong DB, xác nhận cột `token` là hash chứ không phải chuỗi bạn nhận được
7. Thêm cột `user_id` vào posts + `PostPolicy`: chỉ tác giả được sửa/xoá. Test bằng 2 user khác nhau → user B sửa post của A phải nhận 403

PART 10

Chuẩn hoá REST API cho production

Mục tiêu: Biến API "chạy được" thành API "dùng được thật": status code đúng, filter / sort an toàn, versioning, rate limit, CORS, và checklist trước khi deploy.

10.1 Status code — dùng đúng, đừng trả 200 cho mọi thứ

Code	Tên	Dùng khi
200	OK	GET thành công, PUT/PATCH thành công (trả bản ghi sau khi sửa)
201	Created	POST tạo mới thành công — nên kèm header Location
202	Accepted	Đã nhận yêu cầu nhưng xử lý nền chưa xong (đẩy vào queue — Part 14)
204	No Content	DELETE thành công, không có body trả về
400	Bad Request	Request hỏng về mặt cú pháp (JSON sai định dạng)
401	Unauthorized	Chưa đăng nhập / token sai (thực chất là "unauthenticated")
403	Forbidden	Đã đăng nhập nhưng không có quyền
404	Not Found	Không tồn tại resource
409	Conflict	Xung đột trạng thái (publish 2 lần, trùng dữ liệu)
422	Unprocessable Content	Cú pháp đúng nhưng dữ liệu không hợp lệ (validation)

429	Too Many Requests	Vượt rate limit
500	Server Error	Lỗi phía server — không bao giờ được lộ stack trace ra client

Mẹo phân biệt 401 vs 403: 401 = "tôi không biết bạn là ai" (thiếu/sai token → client nên đi đăng nhập lại). 403 = "tôi biết bạn là ai, nhưng bạn không được phép" (đăng nhập lại vô ích).

10.2 Filter, sort an toàn (pagination chi tiết ở Part 12)

```
// GET /api/posts?page=2&per_page=20&search=nest&sort=-created_at
public function index(Request $request)
{
    $perPage = min($request->integer('per_page', 15), 100);          // CHẶN TRẦN

    $posts = Post::query()
        ->when($request->filled('search'), fn ($q) =>
            $q->where('title', 'like', '%'.$request->string('search').'%'))
        ->withCount('comments')
        ->latest()
        ->paginate($perPage)
        ->withQueryString();        // giữ lại query param trong link phân trang

    return PostResource::collection($posts);
}
```

Ba lỗi bảo mật/hiệu năng hay gặp ở endpoint list: (1) không chặn trần `per_page` → client gọi `?per_page=999999` làm sập DB; (2) cho sort theo tên cột client tự gửi mà không whitelist → SQL injection qua `orderBy`; (3) trả `Post::all()` không phân trang.

```
// Sort an toàn: LUÔN whitelist tên cột
$sortable = ['created_at', 'title'];
$sort = $request->string('sort', '-created_at');
$dir  = str_starts_with($sort, '-') ? 'desc' : 'asc';
$col  = ltrim($sort, '-');

if (in_array($col, $sortable, true)) {
    $query->orderBy($col, $dir);
}
```

10.3 Rate limiting

Nhớ Part 07: nhóm `api` của Laravel 13 không bật throttle sẵn. Làm 2 bước:

```php
// Bước 1 — định nghĩa limiter trong AppServiceProvider::boot()
RateLimiter::for('api', fn (Request $request) =>
    Limit::perMinute(60)->by($request->user()?->id ?: $request->ip()));

// Giới hạn chặt hơn cho login (chống brute-force) — nhiều Limit cùng lúc
RateLimiter::for('login', fn (Request $request) => [
    Limit::perMinute(5)->by($request->input('email')),
    Limit::perMinute(20)->by($request->ip()),
]);

// Tuỳ biến response 429 nếu muốn
RateLimiter::for('api', fn (Request $request) =>
    Limit::perMinute(60)->response(fn (Request $r, array $headers) =>
        response()->json(['message' => 'Slow down.'], 429, $headers)));

// Bước 2 — gắn vào route hoặc nhóm api
$middleware->api(prepend: ['throttle:api']);                // bootstrap/app.php
Route::post('/login', ...)->middleware('throttle:login');   // routes/api.php
```

Vượt hạn mức → Laravel tự trả 429 kèm header `Retry-After` và `X-RateLimit-Remaining`. Rate limiter lưu bộ đếm trong cache (Part 16) — nên cache driver phải dùng chung giữa các server nếu chạy nhiều máy.

10.4 CORS

Middleware `HandleCors` đã nằm trong stack global sẵn. Chỉ khi cần tuỳ biến mới publish config:

```bash
php artisan config:publish cors        # → tạo config/cors.php
```

Trong đó khai `allowed_origins` (đừng để `['*']` ở production nếu API có auth bằng cookie), `allowed_headers`, `supports_credentials`.

10.5 API versioning

```php
// Cách 1: prefix trong routes/api.php
Route::prefix('v1')->group(function () {
    Route::apiResource('posts', V1\PostController::class);
});     // → /api/v1/posts

// Cách 2: đổi prefix mặc định trong bootstrap/app.php
->withRouting(api: __DIR__.'/../routes/api.php', apiPrefix: 'api/v1')
```

Nguyên tắc: khi đã public API, không phá vỡ hợp đồng cũ — đổi tên/xoá field là breaking change, phải ra v2 chứ không sửa v1.

10.6 Checklist trước khi deploy (chi tiết vận hành ở Part 22)

Việc	Vì sao
`APP_DEBUG=false`	Quan trọng nhất. True ở production = lộ config, DB credential, stack trace
`APP_ENV=production`	Bật các tối ưu & tắt hành vi dev

`php artisan key:generate`	Phải có APP_KEY riêng, không dùng chung với repo
`php artisan optimize` (= config/route/view/event cache)	Tăng tốc rõ rệt. Nhớ: sau khi cache thì `env()` ngoài config trả null
`composer install --no-dev --optimize-autoloader`	Bỏ package dev, tối ưu autoload
Document root trỏ vào `public/`	Trỏ sai = phơi `.env` ra internet
Bật HTTPS + rate limit	Token gửi qua HTTP là token bị lộ
Kiểm tra `storage/` ghi được	Không ghi được log/cache = lỗi 500 bí ẩn
Queue worker + scheduler đang chạy	Không có worker thì job "đẩy vào queue" nằm im mãi (Part 14, 17, 22)

Việc cần làm:

1. Rà lại toàn bộ endpoint: status code đúng chuẩn bảng 10.1 chưa (đặc biệt 201 và 204)
2. Endpoint index: thêm `per_page` (có chặn trần), search, sort có whitelist
3. Định nghĩa limiter `api` và `login`, gắn throttle, test bằng cách gọi liên tục cho tới khi nhận 429
4. Thêm prefix version v1 cho toàn bộ route
5. Chạy `php artisan optimize` ở local, xác nhận app vẫn chạy, rồi `optimize:clear`
6. Viết file `.env.example` đầy đủ và một mục "API endpoints" trong README (method, path, body, response mẫu)

PART SEC — BẢO MẬT ỨNG DỤNG API — MỚI

Bảo mật: tư duy mối đe doạ và các bài kiểm tra

Mục tiêu: Gom các mảnh bảo mật đã rải rác ở Part 04, 06, 09, 10, 18, 19, 22 thành một bản đồ: **ai có thể làm gì sai, ở đâu Laravel đã bảo vệ sẵn, ở đâu bạn phải tự bảo vệ**. Phần này không dạy lại các Part kia mà cho bạn khung tư duy cùng các bài **kiểm tra bằng test** để chứng minh API của mình chống được những lỗi phổ biến nhất.

SEC.1 Cách nghĩ: Laravel bảo vệ gì sẵn, bạn phải tự bảo vệ gì

Nguy cơ	Laravel đã lo	Bạn phải tự làm	Học ở
SQL injection	Query Builder/Eloquent tự tham số hoá giá trị	Allowlist cho tên cột/ `orderBy`; không nối chuỗi vào `DB::raw` / `whereRaw`; không `unprepared()` với dữ liệu người dùng	SQL.9, 10.2
Mass assignment (client gửi thêm field như `is_admin`)	Cơ chế `$fillable` / `#[Fillable]` chặn field lạ	Chỉ khai báo field thực sự cho phép; dùng `$request->validated()` thay vì `$request->all()`	Part 04, 05

Phân quyền hỏng (IDOR): user B sửa/xoá bài của user A bằng cách đổi `id` trên URL	Không tự biết ai sở hữu gì	Policy cho mọi hành động ghi và đọc dữ liệu riêng tư; quyết định trả 403 hay 404	Part 19
Lộ dữ liệu nhạy cảm trong response	`$hidden` trên model; API Resource	Không trả model thô; chỉ liệt kê field công khai trong Resource	Part 06
Lưu mật khẩu	`Hash::make()`, cast `hashed`	Không bao giờ tự mã hoá hay lưu rõ; không ghi mật khẩu/token vào log	Part 09
Dò mật khẩu, spam	Cung cấp `RateLimiter` / `throttle`	Định nghĩa và gắn limiter cho login, đăng ký, endpoint ghi	Part 10
CSRF	Middleware chống giả mạo request cho nhóm `web` (xem SEC.2)	Hiểu API dùng token không bị CSRF; SPA dùng cookie phải theo luồng Sanctum	SEC.2, 09
XSS	Blade tự escape `{{ }}` (nếu bạn có giao diện)	API không tự render HTML nhưng dữ liệu bạn lưu sẽ bị client hiển thị: không tin dữ liệu, escape khi hiển thị (xem SEC.3)	SEC.3
Upload file độc hại	Rule validate file	Validate loại và kích thước, đặt tên do server sinh, lưu ngoài `public/`	Part 18
Cấu hình sai khi deploy	Mặc định an toàn nếu `APP_DEBUG=false`	`APP_DEBUG=false`, `APP_KEY` riêng, HTTPS, document root là `public/`	10.6, 22.7
Thư viện có lỗ hổng	Không	Chạy `composer audit` định kỳ và nâng cấp (xem SEC.5)	SEC.5

SEC.2 CSRF trong Laravel 13: API dùng token thì sao?

CSRF (cross-site request forgery) là khi một website độc hại làm trình duyệt của người đã đăng nhập tự gửi request tới ứng dụng của bạn, với cookie phiên của họ. Điều kiện để dính: ứng dụng nhận diện người dùng bằng **cookie** mà trình duyệt tự đính kèm.

- Trong Laravel 13, middleware `Illuminate\Foundation\Http\Middleware\PreventRequestForgery` nằm trong nhóm `web` theo mặc định. Nó kiểm tra **hai lớp**: (1) header `Sec-Fetch-Site` mà trình duyệt hiện đại tự gắn — nếu cho biết request cùng origin thì cho qua ngay; (2) nếu không đạt (trình duyệt cũ, hoặc kết nối không phải HTTPS), quay về kiểm tra **CSRF token** truyền thống. Nó kiểm tra mọi request `POST`, `PUT`, `PATCH`, `DELETE`, `QUERY`.
- Route trong `routes/api.php` thuộc nhóm `api`, **không** có middleware này (xem bảng nhóm middleware ở Part 07). API xác thực bằng **Bearer token** trong header `Authorization` (Part 09) không bị CSRF, vì trình duyệt không tự gắn header đó.
- Nếu bạn làm SPA dùng **cookie** với cùng domain, phải theo luồng CSRF của Sanctum (docs Sanctum). Docs CSRF nói rõ: với SPA dùng Laravel làm API backend, hãy xem tài liệu Sanctum.
- Muốn loại trừ một URI khỏi kiểm tra (ví dụ webhook của bên thứ ba không biết gửi token), khai trong `bootstrap/app.php`: `$middleware->preventRequestForgery(except: ['stripe/*'])`. Docs khuyên ưu tiên đặt các route kiểu này **ngoài nhóm** `web`. CSRF tự động tắt khi chạy test.

- Chế độ chỉ kiểm origin: `preventRequestForgery(originOnly: true)`; request không đạt nhận **403** thay vì **419** của token mismatch.

> **Tên cũ:** các tài liệu và bài viết trước Laravel 13 gọi middleware này là `VerifyCsrfToken`. Laravel 13 chính thức hoá nó thành `PreventRequestForgery` và thêm kiểm tra origin, vẫn tương thích với CSRF token.

SEC.3 XSS và dữ liệu người dùng nhập

XSS xảy ra khi dữ liệu do người dùng nhập (ví dụ nội dung comment `<script>...</script>`) được hiển thị như HTML/JavaScript ở trình duyệt của người khác. Với API thuần JSON, server không render HTML nên lỗ hổng nằm ở **nơi hiển thị** (frontend, app mobile, email). Quy tắc:

- Lưu dữ liệu nguyên bản, **escape khi hiển thị** chứ không "làm sạch" lúc lưu: cùng một dữ liệu có thể được hiển thị ở nhiều nơi cần cách escape khác nhau.
- Laravel trả mảng/Resource với `Content-Type: application/json`, nên trình duyệt không thực thi nội dung như HTML.
- Nếu có giao diện Blade: `{{ $x }}` tự escape; `{!! $x !!}` **không** escape — chỉ dùng với nội dung bạn tin tưởng hoàn toàn.
- Nếu thực sự phải cho người dùng gửi HTML (rich text), dùng thư viện lọc HTML với danh sách thẻ cho phép; đừng tự viết bộ lọc bằng regex.
- Khi gửi email chứa nội dung người dùng (Part 20), escape tương tự.

SEC.4 Quản lý bí mật

- **Không commit** `.env` (mục 1.6). Commit `.env.example` với giá trị giả. Lỡ commit thì coi như đã lộ: xoay khoá, đổi mật khẩu, tạo `APP_KEY` mới.
- `APP_KEY` dùng để mã hoá cookie/phiên; mỗi môi trường một khoá riêng, không chia sẻ qua chat/ticket.
- Không in token, mật khẩu, thông tin thẻ vào log (`Log::info($request->all())` là lỗi kinh điển).
- Khoá API bên thứ ba lấy từ `.env` qua `config/services.php`, không viết cứng trong code.

SEC.5 Thư viện có lỗ hổng: composer audit

Ứng dụng Laravel kéo theo hàng trăm package. Lỗ hổng thường nằm ở dependency chứ không phải ở code bạn viết. Composer có lệnh `composer audit` kiểm tra các package đã cài với cơ sở dữ liệu cảnh báo bảo mật. Chạy định kỳ (và trong CI), khi báo có lỗ hổng thì `composer update tên-package` lên bản đã vá và chạy lại test. Theo dõi thêm thông báo release và security của Laravel (support policy: bản 13 nhận bug fix đến Q3 2027 và security fix đến 17/03/2028; hãy lên bản mới trước khi hết hạn).

SEC.6 Kiểm tra bằng test: chứng minh thay vì tin

Bảo mật chỉ có giá trị khi được kiểm chứng. Mỗi lỗi bên dưới đều viết được thành một test ngắn bằng Pest (Part 11). Mẫu cấu trúc: dựng dữ liệu bằng factory, gọi API như kẻ tấn công, kiểm tra response.

Test	Kẻ tấn công làm gì	Phải thấy
Sort injection	`GET /api/v1/posts?sort=views;DROP TABLE posts`	Bị chặn bởi allowlist (200 với sort mặc định, hoặc 422 nếu bạn chọn validate); bảng không bị ảnh hưởng

Mass assignment	`POST /api/v1/posts` kèm field thừa `user_id` hoặc `is_admin`	Field thừa bị bỏ qua; `user_id` luôn là người đang đăng nhập
IDOR	User B gọi `PATCH` / `DELETE` trên bài của user A	403 (hoặc 404 nếu bạn chọn giấu sự tồn tại) và dữ liệu không đổi
Chưa đăng nhập	Gọi endpoint ghi không kèm token	401
Lộ field	Đọc `GET /api/v1/users`	Response không có `password`, `remember_token`
Brute force	Gọi `POST /api/v1/login` liên tục	Nhận 429 sau ngưỡng (Part 10)
Token sau logout	Dùng lại token đã đăng xuất	401

Việc cần làm:

1. Chạy `composer audit` trên project, ghi lại kết quả (không có lỗ hổng cũng là một kết quả). Nếu có, nâng cấp package và chạy lại
2. Chạy `git ls-files | grep -E '^\.env$'` (không được in gì) và `git log --all --oneline -- .env` (không được có commit nào) để xác nhận chưa từng commit `.env`
3. Viết 5 test từ bảng SEC.6: sort injection, mass assignment, IDOR, chưa đăng nhập, lộ field. Mỗi test chạy **đỏ** khi bạn cố ý gỡ biện pháp bảo vệ (gỡ allowlist, thêm field vào `$fillable`, bỏ `authorize`...) và **xanh** khi khôi phục
4. Rà `routes/api.php`: mọi route ghi (`POST`, `PUT`, `PATCH`, `DELETE`) đã nằm trong nhóm có middleware xác thực chưa? Dùng `php artisan route:list --path=api` để kiểm tra cột Middleware
5. Tìm trong code mọi chỗ dùng `DB::raw`, `whereRaw`, `selectRaw`, `orderByRaw`, `unprepared` (gợi ý: `grep -rn "Raw\|unprepared" app/`) và xác nhận không chỗ nào ghép dữ liệu người dùng vào chuỗi
6. Đọc docs Laravel mục CSRF Protection, rồi tự trả lời bằng lời của bạn: vì sao request có header `Authorization: Bearer ...` không cần CSRF token, nhưng request dùng cookie phiên thì cần?
7. Đọc OWASP Top 10 tại owasp.org để biết danh mục lỗ hổng web phổ biến nhất và đối chiếu với bảng SEC.1

PART 11 — TUỲ CHỌN (nhưng nên làm)

Testing (Pest / PHPUnit)

Mục tiêu: Viết test gọi thẳng vào API và kiểm tra response — cách nhanh nhất để không bao giờ phải mở Postman test tay lại lần thứ hai. Từ Part 12 trở đi, mỗi phần đều có task viết test tương ứng.

11.1 Pest hay PHPUnit?

Laravel 11+ mặc định dùng Pest (cú pháp function, gọn), nhưng vẫn chạy trên PHPUnit bên dưới nên mọi assertion đều giống nhau. Chọn kiểu nào cũng được — đừng trộn 2 kiểu trong cùng dự án.

```
php artisan make:test PostApiTest                  # feature test (mặc định)
php artisan make:test PostServiceTest --unit       # unit test
php artisan test                                   # chạy toàn bộ
php artisan test --filter=PostApiTest              # chạy 1 file
php artisan test --parallel                        # chạy song song (nhanh hơn nhiều khi
test lớn)
```

11.2 Feature test cho API

```php
// tests/Feature/PostApiTest.php (Pest)
use App\Models\{Post, User};
use Illuminate\Foundation\Testing\RefreshDatabase;
use Laravel\Sanctum\Sanctum;

uses(RefreshDatabase::class);      // mỗi test chạy trên DB sạch, tự rollback sau đó

test('ai cũng xem được danh sách post', function () {
    Post::factory()->count(3)->create();

    $this->getJson('/api/v1/posts')
        ->assertOk()
        ->assertJsonCount(3, 'data')
        ->assertJsonStructure(['data' => [['id', 'title', 'body']], 'meta', 'links']);
});

test('chưa đăng nhập thì không tạo được post', function () {
    $this->postJson('/api/v1/posts', ['title' => 'X', 'body' => 'Y'])
        ->assertUnauthorized();             // 401
});

test('validate báo lỗi 422 đúng field', function () {
    Sanctum::actingAs(User::factory()->create());

    $this->postJson('/api/v1/posts', ['title' => ''])
        ->assertStatus(422)
        ->assertJsonValidationErrors(['title', 'body']);
});

test('tác giả tạo post thành công', function () {
    Sanctum::actingAs($user = User::factory()->create());

    $this->postJson('/api/v1/posts', ['title' => 'Hello', 'body' => 'Nội dung đủ dài'])
        ->assertCreated()                   // 201
        ->assertJsonPath('data.title', 'Hello');

    $this->assertDatabaseHas('posts', ['title' => 'Hello', 'user_id' => $user->id]);
});
```

11.3 Assertion hay dùng

Assertion	Kiểm tra
`assertOk()` / `assertStatus(n)`	200 / status bất kỳ

`assertCreated()` / `assertNoContent()`	201 / 204
`assertUnauthorized()` / `assertForbidden()` / `assertNotFound()`	401 / 403 / 404
`assertJson([...])`	JSON chứa đoạn này (không cần khớp hết)
`assertExactJson([...])`	JSON khớp chính xác toàn bộ
`assertJsonStructure([...])`	Đúng "hình dạng" key, không quan tâm giá trị
`assertJsonPath('data.title', 'X')`	Giá trị tại một đường dẫn cụ thể
`assertJsonCount(3, 'data')`	Số phần tử trong mảng
`assertJsonValidationErrors([...])`	Có lỗi validation ở đúng các field này
`assertDatabaseHas / Missing / Count`	Trạng thái thật trong database

RefreshDatabase làm gì: bọc mỗi test trong một transaction và rollback khi test kết thúc → các test độc lập, không rác dữ liệu. Cấu hình DB test riêng trong `phpunit.xml` (thường dùng SQLite in-memory: `DB_DATABASE=:memory:`) để chạy cực nhanh và không đụng vào DB dev.

Bổ sung — các Fake sẽ dùng ở Part sau: `Queue::fake()` (Part 14), `Event::fake()` (Part 15), `Storage::fake()` (Part 18), `Notification::fake()` (Part 20), `Bus::fake()` cho chain/batch. Chúng thay implementation thật bằng bản ghi nhận, rồi bạn `assert...` xem có được gọi đúng không — đây là lý do Part 00 nhấn mạnh Facade "mock được".

Việc cần làm:

1. Cấu hình `phpunit.xml` dùng SQLite in-memory cho test
2. Viết feature test cho đủ 5 endpoint của posts (index, show, store, update, destroy)
3. Viết test cho 3 case lỗi: 401 (không token), 403 (không phải tác giả), 422 (validate sai)
4. Viết 1 test chứng minh không còn N+1: dùng `DB::listen()` đếm query của endpoint index
5. Chạy `php artisan test` — mục tiêu: toàn bộ xanh

PART 12 — MỚI

Pagination chuyên sâu & xử lý dữ liệu lớn

Mục tiêu: Biết 3 kiểu phân trang của Laravel khác nhau ở tầng SQL như thế nào, chọn đúng kiểu cho đúng bài toán, và biết cách duyệt bảng hàng triệu dòng mà không "nổ" RAM.

12.1 Ba kiểu phân trang — khác nhau ở câu SQL

Method	SQL sinh ra	Có tổng số trang?	Dùng khi
`paginate(15)`	`SELECT COUNT(*)` + `SELECT ... LIMIT 15 OFFSET 30` (2 query)	Có (`total` , `last_page`)	UI có số trang 1, 2, 3... — mặc định cho API admin
`simplePaginate(15)`	Chỉ `SELECT ... LIMIT 16 OFFSET 30` (1 query, lấy dư 1 dòng để biết còn trang sau không)	Không	Chỉ cần nút Prev/Next, bảng lớn mà `COUNT(*)` chậm
`cursorPaginate(15)`	`SELECT ... WHERE id > 15 ORDER BY id LIMIT 15` (không OFFSET)	Không	Infinite scroll, feed thời gian thực, bảng cực lớn

```
// Offset pagination — trang 2
select * from posts order by id asc limit 15 offset 15;

// Cursor pagination — "trang 2" = mọi thứ sau bản ghi cuối của trang 1
select * from posts where id > 15 order by id asc limit 15;
```

Vì sao cursor nhanh hơn với bảng lớn: `OFFSET 100000` bắt database đọc và bỏ qua 100.000 dòng đầu rồi mới lấy 15 dòng bạn cần — càng về sau càng chậm. Cursor dùng `WHERE id > x` nên nhảy thẳng bằng index. Ngoài ra khi dữ liệu thay đổi liên tục (feed mạng xã hội), offset có thể trả trùng/thiếu bản ghi giữa 2 lần gọi; cursor thì không.

Ràng buộc của cursorPaginate (theo docs): query bắt buộc có `orderBy` ; cột sort phải thuộc bảng đang phân trang và phải unique (hoặc tổ hợp cột unique), không được chứa null; không hỗ trợ trang số (chỉ prev/next). URL sinh ra dạng `?cursor=eyJpZCI6MTUs...` — client chỉ việc gửi lại nguyên chuỗi đó.

12.2 Cấu trúc JSON trả về — 2 dạng khác nhau, đừng nhầm

```
return Post::paginate(15);                          // trả paginator THẲNG → JSON phẳng (dạng A)
return PostResource::collection(Post::paginate(15));  // qua Resource → data/links/meta (dạng B, Part 06)
```

```
// Dạng A — paginator thẳng: mọi thứ ở cấp 1
{ "total": 50, "per_page": 15, "current_page": 1, "last_page": 4,
  "first_page_url": "...?page=1", "next_page_url": "...?page=2", "prev_page_url": null,
  "path": "http://.../api/posts", "from": 1, "to": 15,
  "data": [ {...}, {...} ] }

// Dạng B — qua Resource collection: gom vào links/meta
{ "data": [...],
  "links": { "first": "...", "last": "...", "prev": null, "next": "..." },
  "meta":  { "current_page": 1, "from": 1, "last_page": 4, "per_page": 15, "to": 15,
"total": 50 } }
```

Với API, luôn dùng dạng B (qua Resource) để field bên trong `data` được kiểm soát (Part 06). Nhớ `->withQueryString()` để link `next` giữ lại `?search=...&sort=...` của request hiện tại; hoặc `->appends(['sort' => 'votes'])` để thêm tay.

12.3 Chặn trần per_page & page hợp lệ

```
// Form Request cho query string — cùng cơ chế với body
class IndexPostRequest extends FormRequest
{
    public function rules(): array
    {
        return [
            'page'     => ['sometimes', 'integer', 'min:1'],
            'per_page' => ['sometimes', 'integer', 'min:1', 'max:100'],
            'sort'     => ['sometimes',
                Rule::in(['created_at', '-created_at', 'title', '-title'])],
            'search'   => ['sometimes', 'string', 'max:100'],
        ];
    }
}

public function index(IndexPostRequest $request)
{
    return Post::query()
        ->latest()
        ->paginate($request->integer('per_page', 15))
        ->withQueryString()
        ->toResourceCollection();
}
```

Đối chiếu NestJS: bên Nest bạn phải thêm `@Type(() => Number)` vì query string luôn là chuỗi. Bên Laravel rule `integer` chấp nhận chuỗi số, và `$request->integer('per_page', 15)` tự ép kiểu + có default — đỡ một tầng boilerplate.

12.4 Duyệt dữ liệu lớn KHÔNG phân trang — chunk / lazy / cursor

Pagination là cho client. Khi chính bạn cần đi qua 1 triệu dòng (export CSV, gửi mail hàng loạt, migrate dữ liệu), `Post::all()` sẽ nạp cả triệu model vào RAM và chết. Laravel có 3 công cụ:

```php
// 1) chunkById — lấy từng lô 500, callback xử lý, giải phóng RAM sau mỗi lô
Post::where('published_at', null)->chunkById(500, function ($posts) {
    foreach ($posts as $post) { /* ... */ }
});
// dùng chunkById (theo id) thay vì chunk() khi bạn UPDATE chính bảng đang duyệt — chunk() theo OFFSET sẽ bỏ sót dòng

// 2) lazyById — trả LazyCollection, viết như foreach thường nhưng bên dưới vẫn chunk
foreach (Post::lazyById(500) as $post) { /* ... */ }

// 3) cursor — 1 query duy nhất, stream từng dòng; RAM thấp nhất nhưng KHÔNG eager load được
foreach (Post::cursor() as $post) { /* ... */ }
```

	`chunkById`	`lazyById`	`cursor`
Số query	N/500 query	N/500 query	1 query
RAM	500 model mỗi lúc	500 model mỗi lúc	1 model mỗi lúc
Eager load `with()`	✅	✅	❌ (mỗi dòng 1 query → N+1)
Phù hợp	Xử lý theo lô, có thể sửa dữ liệu	Code đọc như vòng lặp thường	Chỉ đọc, không cần quan hệ

Việc export/xử lý lớn thế này **không bao giờ** nên chạy trong request HTTP (timeout, chiếm worker PHP-FPM). Đẩy nó vào Job (Part 14) — đó là lý do Part 12 nằm ngay trước Part 14.

Việc cần làm:

1. Seed 5.000 post (`Post::factory()->count(5000)->create()` trong tinker — dùng `chunk` nếu chậm)
2. Endpoint index: nhận `page`, `per_page` (max 100), `sort` có whitelist qua Form Request như mục 12.3
3. Thêm endpoint `GET /api/posts/feed` dùng `cursorPaginate(20)` theo `id`, test gọi liên tiếp bằng chuỗi `cursor` trả về
4. Dùng `DB::listen()` so sánh SQL sinh ra của `paginate` vs `cursorPaginate` ở trang 200
5. Viết Artisan command `posts:export` ghi 5.000 post ra CSV bằng `lazyById`; đo RAM bằng `memory_get_peak_usage()` so với `Post::all()`
6. Test: `?per_page=1000` phải nhận 422; `?sort=password` phải nhận 422

PART 13 — MỚI

Database Transactions

> **Mục tiêu:** Đảm bảo nhiều thao tác ghi "thành công cùng nhau hoặc thất bại cùng nhau" (atomicity), hiểu bẫy deadlock, và — quan trọng nhất trong Laravel — hiểu vì sao job/event bắn ra **bên trong** transaction có thể chạy trước khi dữ liệu được commit.

13.1 Vấn đề transaction giải quyết

Tạo 1 `Post`, đồng thời tạo `Comment` chào mừng đầu tiên và cộng `posts_count` cho user. Nếu bước 2 hoặc 3 lỗi, bạn còn lại dữ liệu nửa vời. Transaction đảm bảo: hoặc cả 3 cùng được ghi (COMMIT), hoặc cả 3 cùng bị hoàn tác (ROLLBACK) như chưa từng xảy ra.

13.2 Cách chuẩn — `DB::transaction()`

```
use Illuminate\Support\Facades\DB;

public function publish(User $author, array $data): Post
{
    return DB::transaction(function () use ($author, $data) {
        $post = $author->posts()->create($data);
        $post->comments()->create(['body' => 'Chào mừng bài viết đầu tiên!']);
        $author->increment('posts_count');

        return $post;      // giá trị return của closure = giá trị return của
DB::transaction()
    });
}
```

Cơ chế: `BEGIN` → chạy closure → không lỗi thì `COMMIT`; closure `throw` bất kỳ exception nào thì `ROLLBACK` rồi **ném lại exception đó** cho bạn (không nuốt). Không cần try/catch. Mọi query Eloquent/Query Builder bên trong closure đều nằm trong cùng transaction (Laravel dùng chung 1 connection).

> **Đối chiếu NestJS:** bên TypeORM bạn phải gọi mọi thứ qua `manager` được truyền vào callback, dùng nhầm repository thường là "thoát" khỏi transaction. Laravel không có vấn đề đó — vì transaction gắn với connection, và mọi model đều đi qua connection đó trong cùng request. Đơn giản hơn hẳn.

13.3 Deadlock & retry

```
DB::transaction(function () {
    // ...
}, attempts: 5);      // gặp deadlock → tự chạy lại tối đa 5 lần trước khi ném lỗi
```

Deadlock xảy ra khi 2 transaction chờ khoá của nhau (A giữ dòng 1 chờ dòng 2, B giữ dòng 2 chờ dòng 1). Database sẽ tự giết 1 trong 2. Tham số `attempts` cho Laravel tự chạy lại transaction bị giết — với điều kiện closure của bạn **idempotent** (chạy lại không gây tác dụng phụ kép: không gửi mail, không dispatch job bên trong closure).

13.4 Thủ công — khi cần kiểm soát từng bước

```
DB::beginTransaction();
try {
    // ...
    DB::commit();
} catch (\Throwable $e) {
    DB::rollBack();
    throw $e;
}
```

Chỉ dùng khi thực sự cần (ví dụ commit từng phần trong vòng lặp lớn). 95% trường hợp `DB::transaction()` đủ và an toàn hơn — quên `rollBack()` trong catch là để lại transaction treo, khoá bảng cho tới khi request kết thúc.

13.5 Bẫy lớn nhất: job/event bắn ra TRONG transaction

```
DB::transaction(function () {
    $post = Post::create([...]);          // ① INSERT — nhưng CHƯA commit
    ProcessPost::dispatch($post);         // ② job vào queue NGAY LẬP TỨC
});                                       // ③ COMMIT ở đây

Queue worker (process khác) nhận job ở giữa ② và ③:
    → SELECT * FROM posts WHERE id = 42   → KHÔNG THẤY (chưa commit) →
ModelNotFoundException
    → hoặc tệ hơn: transaction rollback, job vẫn chạy với dữ liệu không tồn tại
```

Cách xử lý (theo docs Queues 13.x):

```
// Cách 1 — từng job: chỉ đẩy vào queue sau khi transaction ngoài cùng commit
ProcessPost::dispatch($post)->afterCommit();

// Cách 2 — toàn cục: config/queue.php, connection đang dùng
'after_commit' => true,

// Event cũng vậy: implement ShouldDispatchAfterCommit trên class Event (Part 15)
// Listener queued: implement ShouldQueueAfterCommit
```

Nếu transaction rollback, job/event có `afterCommit` sẽ bị **bỏ luôn** — đúng như mong muốn. Nếu không có transaction nào đang mở, chúng được dispatch ngay như thường.

13.6 Implicit commit — lệnh làm transaction "tự commit" mà bạn không biết

Trong MySQL, các lệnh DDL (`CREATE TABLE` , `ALTER TABLE` , `TRUNCATE` ...) gây implicit commit: chạy chúng qua `DB::statement()` bên trong transaction sẽ commit toàn bộ transaction ngầm, Laravel không hay biết. Đừng chạy migration/schema trong transaction nghiệp vụ.

13.7 Quan sát query — công cụ debug hiệu năng

```php
// AppServiceProvider::boot()
DB::listen(function (QueryExecuted $query) {
    Log::debug($query->toRawSql(), ['ms' => $query->time]);   // toRawSql: SQL đã gắn binding
});

// Cảnh báo khi 1 request tốn > 500ms cho database
DB::whenQueryingForLongerThan(500, function (Connection $connection, QueryExecuted $event) {
    Log::warning('Slow request: total DB time > 500ms');
});
```

Việc cần làm:

1. Viết `PostService::publish()` tạo Post + Comment đầu tiên + tăng `posts_count` trong 1 `DB::transaction()`
2. Test rollback: tạm `throw new \RuntimeException('test')` sau bước tạo Comment → gọi API → xác nhận cả Post lẫn Comment đều KHÔNG có trong DB (`assertDatabaseMissing`)
3. Viết test Pest chứng minh `DB::transaction()` ném lại đúng exception gốc (`expect(fn () => ...)->toThrow(RuntimeException::class)`)
4. Thêm `DB::listen()` log ra `storage/logs` , gọi endpoint index và đếm số query — dùng lại cho task N+1 ở Part 08
5. Đọc trước mục 13.5, giữ lại trong đầu — Part 14 sẽ tái hiện đúng bug này bằng job thật

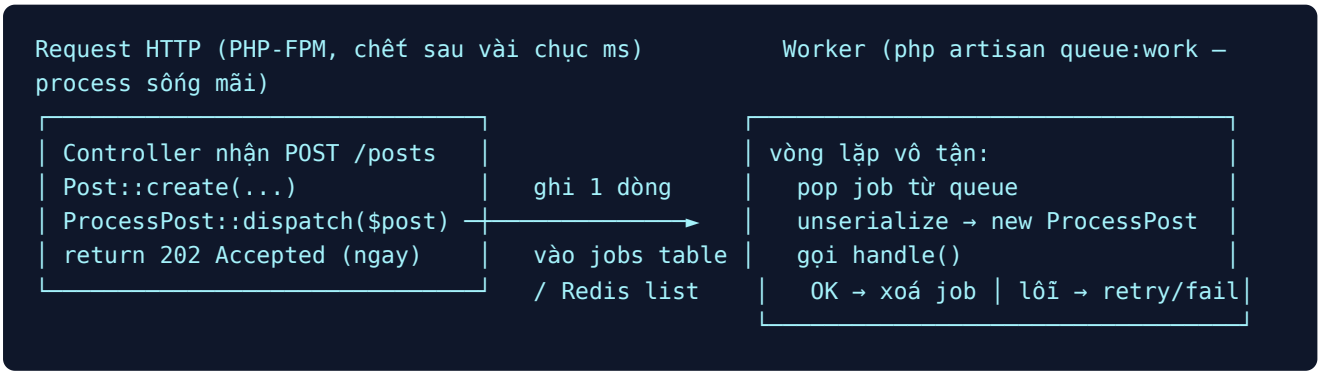
---

PART 14 — MỚI (chuyên sâu)

Queue & Jobs

Mục tiêu: Đưa mọi việc chậm (gửi mail, xử lý ảnh, gọi API bên thứ 3, export) ra khỏi request HTTP; hiểu trọn vòng đời một job từ `dispatch()` tới worker, retry, fail; và vận hành worker đúng cách. Đây là phần phân biệt rõ nhất "biết Laravel" và "làm Laravel thật".

14.1 Mô hình: queue là gì trong bức tranh PHP



Vì PHP-FPM không có event loop, "chạy nền" đồng nghĩa với **ghi mô tả công việc vào một kho (queue)** để một process PHP khác (worker) lấy ra chạy. Đây là câu trả lời của Laravel cho `async` của Node — và nó có lợi thế: worker crash không kéo theo web, scale worker độc lập với web.

14.2 Cấu hình driver

`QUEUE_CONNECTION`	Kho chứa job	Dùng khi
`sync`	Không có kho — chạy ngay trong request	Chỉ để dev nhanh/test; KHÔNG phải queue thật
`database`	Bảng `jobs` trong DB của bạn	Mặc định của project mới, đủ tốt cho app vừa
`redis`	Redis list	Production tải cao, dùng kèm Horizon (Part 22)
`sqs` / `beanstalkd`	Dịch vụ ngoài	Hạ tầng AWS / có sẵn beanstalkd
`failover`	Tự chuyển sang connection dự phòng khi connection chính lỗi	High availability

```
# Driver database cần bảng jobs (project mới thường đã có migration này sẵn)
php artisan make:queue-table
php artisan migrate
```

Bẫy số 1 của người mới: để `QUEUE_CONNECTION=sync` rồi tưởng job "chạy nền" — thực ra nó chạy ngay trong request, request vẫn chậm y nguyên. Và ngược lại: đổi sang `database` mà quên bật worker → job nằm im trong bảng `jobs` mãi mãi, mail không bao giờ gửi.

14.3 Viết job

```
php artisan make:job ProcessPost
```

```php
namespace App\Jobs;

use App\Models\Post;
use App\Services\ImageOptimizer;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Foundation\Queue\Queueable;
use Illuminate\Queue\Attributes\Tries;
use Illuminate\Queue\Attributes\Timeout;
use Illuminate\Queue\Attributes\Backoff;

#[Tries(3)]                 // tối đa 3 lần thử
#[Timeout(120)]             // 1 lần chạy tối đa 120s
#[Backoff([10, 60, 300])]   // chờ 10s, rồi 60s, rồi 300s giữa các lần retry
class ProcessPost implements ShouldQueue
{
    use Queueable;

    public function __construct(
        public Post $post,      // Eloquent model: chỉ serialize ID, worker tự load lại từ DB
    ) {}

    // Container inject dependency vào handle() y như controller
    public function handle(ImageOptimizer $optimizer): void
    {
        $optimizer->optimize($this->post->cover_path);
        $this->post->update(['processed_at' => now()]);
    }

    // Gọi khi job THẤT BẠI HẲN (hết số lần thử)
    public function failed(\Throwable $e): void
    {
        $this->post->update(['processing_error' => $e->getMessage()]);
    }
}
```

- `ShouldQueue` — interface đánh dấu "đẩy vào queue" (không có nó, `dispatch()` sẽ chạy đồng bộ).
- `Queueable` — trait mang các method `onQueue()`, `delay()`, `afterCommit()`... và lo việc serialize.
- **SerializesModels:** model trong constructor chỉ được lưu **ID + class** vào payload; khi worker chạy, nó query lại DB. Hệ quả: dữ liệu luôn mới nhất, nhưng nếu bản ghi đã bị xoá thì job fail với `ModelNotFoundException`. Quan hệ đã `load()` cũng được serialize kèm — dùng `#[WithoutRelations]` để payload nhẹ.

> **Đã đối chiếu docs Queues 13.x:** các attribute `#[Tries]`, `#[Timeout]`, `#[Backoff]`, `#[MaxExceptions]`, `#[FailOnTimeout]`, `#[WithoutRelations]` đều chính thức (namespace `Illuminate\Queue\Attributes\*`). Cách cũ `public int $tries = 3;` / `public function backoff(): array` / `retryUntil(): DateTime` vẫn hoạt động. **Quy tắc quan trọng:** `timeout` của job phải **nhỏ hơn** `retry_after` trong `config/queue.php` (mặc định 90s) — nếu không, job đang chạy dở có thể bị worker khác nhặt lên chạy trùng.

14.4 Dispatch — mọi biến thể

```php
// Đẩy vào queue mặc định
ProcessPost::dispatch($post);

// Queue riêng (để worker ưu tiên) / connection riêng
ProcessPost::dispatch($post)->onQueue('media');
ProcessPost::dispatch($post)->onConnection('redis');

// Chạy sau 10 phút
ProcessPost::dispatch($post)->delay(now()->addMinutes(10));

// Chờ transaction ngoài cùng commit rồi mới vào queue (Part 13.5)
ProcessPost::dispatch($post)->afterCommit();

// Có điều kiện
ProcessPost::dispatchIf($post->hasCover(), $post);

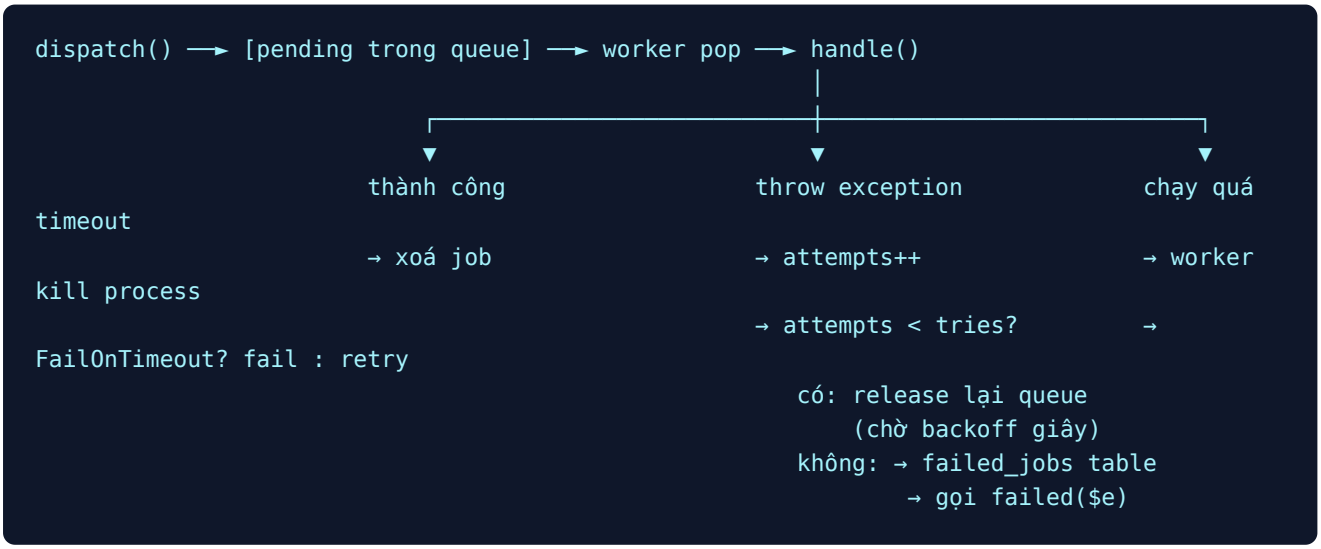
// Ép chạy ngay trong request, bỏ qua queue (debug / test)
ProcessPost::dispatchSync($post);

// Chạy sau khi đã trả response, trong CÙNG process — chỉ cho việc dưới vài giây
ProcessPost::dispatchAfterResponse($post);

// Laravel 13 — định tuyến tập trung trong AppServiceProvider::boot():
Queue::route(ProcessPost::class, connection: 'redis', queue: 'media');
Queue::route([
    ProcessPost::class  => ['redis', 'media'],
    SendDigest::class   => 'emails',
]);
```

Controller nên trả 202: khi bạn `dispatch` rồi return, việc chưa xong. Trả `202 Accepted` + id để client hỏi lại trạng thái, đừng trả 200 như thể đã hoàn tất.

14.5 Vòng đời một job — retry, fail, timeout



Lệnh	Tác dụng

`php artisan queue:failed`	Liệt kê job đã fail (id, class, lý do, thời điểm)
`php artisan queue:retry <id>` / `--all`	Đẩy job fail trở lại queue để thử lại
`php artisan queue:forget <id>` / `queue:flush`	Xoá 1 / xoá hết job fail
`php artisan queue:prune-failed --hours=48`	Dọn job fail cũ (đặt vào Schedule — Part 17)

Retry chỉ có ích khi lỗi là tạm thời (mạng, API ngoài quá tải). Lỗi logic (dữ liệu sai) retry 3 lần vẫn sai — với loại đó hãy `$this->fail($e)` ngay trong `handle()` để khỏi phí tài nguyên, hoặc dùng `#[MaxExceptions(1)]`.

14.6 Chạy worker đúng cách

```
php artisan queue:work                                  # worker cơ bản
php artisan queue:work --queue=media,default --tries=3  # ưu tiên queue 'media' trước 'default'
php artisan queue:work --max-jobs=1000 --max-time=3600  # tự thoát sau 1000 job / 1 giờ (chống rò rỉ RAM)
php artisan queue:listen                                # nạp lại code mỗi job — CHẬM, chỉ để dev
php artisan queue:restart                               # báo mọi worker thoát êm sau job hiện tại
```

Điểm sống còn khi deploy (theo docs): `queue:work` nạp code app **một lần** rồi chạy mãi trong RAM — sửa code xong mà không `queue:restart` thì worker vẫn chạy code CŨ. Mọi script deploy phải có dòng `php artisan queue:restart`. Lệnh này chỉ "ra hiệu", nên cần Supervisor (Part 22) để tự khởi động worker mới.

Ở local, mở terminal thứ 2 chạy `php artisan queue:work` song song với `php artisan serve`. Hoặc `php artisan queue:listen` nếu đang sửa code job liên tục (nó tự nạp lại code, đổi lại chậm hơn).

14.7 Job middleware — kiểm soát cách job chạy

```php
use Illuminate\Queue\Middleware\{WithoutOverlapping, RateLimited};
use Illuminate\Queue\Middleware\{ThrottlesExceptions, Skip};

public function middleware(): array
{
    return [
        // Không cho 2 job của cùng 1 post chạy song song; job đến sau được release, thử lại sau 60s
        (new WithoutOverlapping($this->post->id))->releaseAfter(60),

        // Giới hạn tốc độ (ví dụ API bên thứ 3 cho 100 req/phút) — limiter khai trong RateLimiter::for('external-api', ...)
        new RateLimited('external-api'),

        // Nếu 10 exception trong 5 phút → tạm dừng job loại này 5 phút (circuit breaker)
        new ThrottlesExceptions(10, 5 * 60),

        // Bỏ qua job khi điều kiện đúng (ví dụ post đã bị xoá)
        Skip::when(fn () => $this->post->trashed()),
    ];
}
```

Khi nào dùng cái nào: `WithoutOverlapping` cho tài nguyên chỉ được 1 người sửa (ví dụ số dư, file); `RateLimited` khi gọi API ngoài có quota; `ThrottlesExceptions` khi dịch vụ ngoài đang sập — thay vì 1.000 job cùng fail và retry dồn dập, tạm "ngắt cầu dao".

14.8 Unique, debounce, encrypted

```php
use Illuminate\Contracts\Queue\{ShouldBeUnique, ShouldBeEncrypted};
use Illuminate\Queue\Attributes\{UniqueFor, DebounceFor};

// Chỉ 1 job "reindex post #42" tồn tại trong queue tại một thời điểm (khoá 1 giờ)
#[UniqueFor(3600)]
class ReindexPost implements ShouldQueue, ShouldBeUnique
{
    public function uniqueId(): string { return (string) $this->post->id; }
}

// Debounce (Laravel 13): gom nhiều lần dispatch trong 30s thành 1 lần chạy; tối đa chờ 120s
#[DebounceFor(30, maxWait: 120)]
class UpdateSearchIndex implements ShouldQueue
{
    public function debounceId(): string { return (string) $this->postId; }
}

// Payload chứa dữ liệu nhạy cảm → mã hoá trong queue
class SendInvoice implements ShouldQueue, ShouldBeEncrypted { }
```

14.9 Chain & Batch — nhiều job phối hợp

```php
use Illuminate\Support\Facades\Bus;

// Chain: chạy TUẦN TỰ, job trước fail thì dừng cả chuỗi
Bus::chain([
    new OptimizeCover($post),
    new GenerateThumbnails($post),
    new PublishPost($post),
])->catch(function (\Throwable $e) {
    // 1 job trong chuỗi fail
})->dispatch();

// Batch: chạy SONG SONG nhiều job, theo dõi tiến độ tổng
// (cần bảng: php artisan make:queue-batches-table && php artisan migrate; job dùng trait Batchable)
$batch = Bus::batch([
    new ImportCsvChunk(1),
    new ImportCsvChunk(2),
    new ImportCsvChunk(3),
])->then(fn (Batch $b) => Log::info('all done'))
  ->catch(fn (Batch $b, \Throwable $e) => Log::error('first failure'))
  ->finally(fn (Batch $b) => Log::info('finished'))
  ->name('import-posts')
  ->dispatch();

return ['batch_id' => $batch->id];   // client gọi lại: Bus::findBatch($id)->progress() (0-100)
```

Kết hợp với Part 12: export 1 triệu dòng = chia thành 200 job `ExportChunk` (mỗi job 5.000 dòng, dùng `lazyById`), gom vào 1 batch, `then()` ghép file và gửi notification (Part 20). Đây là kiến trúc chuẩn cho mọi việc nặng.

14.10 Test job — không cần chạy worker

```php
test('publish post đẩy job xử lý vào queue media', function () {
    Queue::fake();                                  // chặn queue thật, chỉ ghi nhận

    Sanctum::actingAs(User::factory()->create());
    $this->postJson('/api/v1/posts', [...])->assertAccepted();   // 202

    Queue::assertPushedOn('media', ProcessPost::class);
    Queue::assertPushed(ProcessPost::class, fn ($job) => $job->post->title === 'Hello');
});

test('job xử lý post đúng logic', function () {
    $post = Post::factory()->create();

    (new ProcessPost($post))->handle(app(ImageOptimizer::class));   // gọi thẳng, không qua queue

    expect($post->fresh()->processed_at)->not->toBeNull();
});
```

Việc cần làm:

1. Đặt `QUEUE_CONNECTION=database`, chạy `make:queue-table` + migrate, mở bảng `jobs` trong DevDb/DB viewer
2. Viết job `ProcessPost` (giả lập việc chậm bằng `sleep(3)`), dispatch từ `store()`, trả 202. Gọi API và xác nhận response về ngay lập tức, bảng `jobs` có 1 dòng
3. Chạy `php artisan queue:work` ở terminal 2 → quan sát job được xử lý, dòng trong bảng biến mất
4. Tái hiện bug Part 13.5: bọc `store()` trong `DB::transaction()` có `sleep(5)` sau khi dispatch, worker chạy song song → job fail vì chưa thấy post. Sửa bằng `->afterCommit()`
5. Cho job throw exception, đặt `#[Tries(3)]` + `#[Backoff([2, 5, 10])]`, quan sát `queue:failed` sau 3 lần; `queue:retry` lại
6. Thêm `WithoutOverlapping` theo `post_id`, dispatch 3 job cùng post → chứng minh chúng chạy lần lượt
7. Viết batch export 5.000 post ra CSV chia 10 job, endpoint `GET /api/exports/{batchId}` trả `progress`
8. Viết 2 test như mục 14.10; đảm bảo `php artisan test` không cần worker vẫn xanh

---

PART 15 — MỚI

Events, Listeners & Observers

Mục tiêu: Tách "chuyện gì vừa xảy ra" khỏi "ai cần phản ứng" — để thêm hành vi mới (gửi mail, ghi log, cập nhật thống kê) mà không sửa lại service gốc. Và hiểu event kết hợp với queue/transaction thế nào.

15.1 Vì sao cần event

Không có event, `PostService::publish()` sẽ phình ra: tạo post → gửi mail cho follower → cập nhật search index → bắn webhook → ghi audit log... Mỗi tính năng mới lại phải sửa hàm này. Với event, service chỉ nói "PostPublished" rồi xong; mỗi listener tự lo phần mình, thêm/bớt listener không đụng service.

15.2 Tạo event & listener

```
php artisan make:event PostPublished
php artisan make:listener SendPostPublishedNotification --event=PostPublished
```

```php
// app/Events/PostPublished.php — chỉ là "túi dữ liệu"
class PostPublished
{
    use Dispatchable, SerializesModels;

    public function __construct(public Post $post) {}
}

// app/Listeners/SendPostPublishedNotification.php
class SendPostPublishedNotification implements ShouldQueue    // queued → chạy ở worker (Part 14)
{
    public function handle(PostPublished $event): void
    {
        $event->post->author->notify(new PostPublishedNotification($event->post));   // Part 20
    }
}

// Bắn event — trong service
PostPublished::dispatch($post);          // hoặc event(new PostPublished($post));
```

> **Đã đối chiếu docs Events 13.x — Event Discovery:** Laravel **tự quét** thư mục `app/Listeners`, thấy method `handle()` hoặc `__invoke()` có type-hint event nào thì tự đăng ký cho event đó — **không cần** khai báo trong `EventServiceProvider` như Laravel ≤10 (file đó không còn tồn tại). Muốn quét thêm thư mục khác: `->withEvents(discover: [__DIR__.'/../app/Domain/*/Listeners'])` trong `bootstrap/app.php`. Kiểm tra bằng `php artisan event:list`. Production nên chạy `php artisan event:cache` (có trong `optimize`).

15.3 Listener chạy nền — cấu hình queue ngay trên listener

```php
use Illuminate\Queue\Attributes\{Connection, Queue, Delay};

#[Queue('notifications')]
#[Delay(60)]
class SendPostPublishedNotification implements ShouldQueue
{
    // Quyết định lúc runtime có nên queue không
    public function shouldQueue(PostPublished $event): bool
    {
        return $event->post->author->wants_notifications;
    }

    public function failed(PostPublished $event, \Throwable $e): void { /* ... */ }
}
```

Listener queued về bản chất là một job (Part 14): có retry, backoff, middleware, `failed()`, `ShouldBeUnique`, `ShouldBeEncrypted`, và cả debounce (`#[DebounceFor]`) y hệt.

15.4 Event & transaction — bài học từ Part 13.5

```php
use Illuminate\Contracts\Events\ShouldDispatchAfterCommit;

// Event chỉ được dispatch SAU khi transaction đang mở commit; rollback thì bỏ event
class PostPublished implements ShouldDispatchAfterCommit { /* ... */ }

// Hoặc ở phía listener queued:
use Illuminate\Contracts\Queue\ShouldQueueAfterCommit;
class SendPostPublishedNotification implements ShouldQueueAfterCommit { /* ... */ }

// Gom nhiều event trong 1 khối, chỉ bắn khi khối chạy xong không lỗi (Laravel 13)
Event::defer(function () use ($data) {
    $post = Post::create($data);
    $post->comments()->create([...]);
});   // model event "created" của cả post lẫn comment chỉ bắn ở đây
```

15.5 Model events & Observer — event có sẵn của Eloquent

Eloquent tự bắn event ở mỗi bước vòng đời: `creating/created`, `updating/updated`, `saving/saved`, `deleting/deleted`, `restoring/restored`. Gom listener cho một model vào 1 class Observer:

```bash
php artisan make:observer PostObserver --model=Post
```

```php
use Illuminate\Database\Eloquent\Attributes\ObservedBy;

#[ObservedBy([PostObserver::class])]        // Laravel 13: đăng ký ngay trên model
class Post extends Model { }

class PostObserver
{
    public function creating(Post $post): void
    {
        $post->slug ??= Str::slug($post->title);   // tự sinh slug trước khi INSERT
    }

    public function deleted(Post $post): void
    {
        Cache::forget("post:{$post->id}");         // Part 16
    }
}
```

Cẩn thận: model event KHÔNG bắn khi bạn dùng query hàng loạt (`Post::where(...)->update([...])`, `->delete()` trên builder) — chỉ bắn khi thao tác trên instance model. Nếu logic trong Observer là bắt buộc (ví dụ audit), đừng dùng mass update.

15.6 Test

```
test('publish bắn event PostPublished', function () {
    Event::fake([PostPublished::class]);       // chỉ fake event này, event khác (model events) vẫn chạy

    app(PostService::class)->publish($author, $data);

    Event::assertDispatched(PostPublished::class, fn ($e) => $e->post->title === 'Hello');
});

test('listener gửi notification', function () {
    Notification::fake();

    (new SendPostPublishedNotification)->handle(new PostPublished($post));

    Notification::assertSentTo($post->author, PostPublishedNotification::class);
});
```

Việc cần làm:

1. Tạo event `PostPublished` + 2 listener: `SendPostPublishedNotification` (queued) và `UpdateAuthorStats` (đồng bộ). Chạy `event:list` xác nhận discovery đã nhận
2. Bắn event từ `PostService::publish()` (bên trong `DB::transaction` của Part 13) — implement `ShouldDispatchAfterCommit`
3. Tạo `PostObserver` tự sinh slug ở `creating`, đăng ký bằng `#[ObservedBy]`
4. Thử `Post::query()->update(['title' => 'x'])` và chứng minh Observer KHÔNG chạy
5. Viết 2 test như 15.6 với `Event::fake()` / `Notification::fake()`

PART 16 — MỚI

Cache & Atomic Locks

Mục tiêu: Giảm tải database bằng cache đúng cách (và biết khi nào cache gây hại), dùng atomic lock để chống race condition — thứ mà PHP "mỗi request một process" đặc biệt dễ dính.

16.1 Vì sao PHP cần cache ngoài process

Nhớ P.1: mỗi request là process mới, biến static/RAM không sống qua request. Muốn "nhớ" kết quả tính toán giữa các request, phải cất ở nơi ngoài process: database, file, Redis, Memcached. Laravel gom tất cả sau một API duy nhất — `Cache` facade — đổi driver chỉ cần đổi `.env`.

`CACHE_STORE`	Đặc điểm	Tags?	Locks?
`database`	Mặc định project mới, bảng `cache` (`make:cache-table` nếu thiếu)	❌	✅
`file`	Đơn giản, chỉ 1 server	❌	✅

`redis`	Nhanh nhất, chia sẻ giữa nhiều server — chuẩn production	✅	✅
`memcached`	Tương tự Redis, ít tính năng hơn	✅	✅
`array`	Chỉ trong 1 request — dùng cho test	✅	✅
`failover`	Tự chuyển sang store dự phòng khi store chính lỗi (mới)	—	—

16.2 API cốt lõi

```
Cache::put('key', $value, 600);                    // TTL giây, hoặc now()->addMinutes(10)
Cache::get('key', 'default');                      // không có → default
Cache::add('key', $value, 600);                    // chỉ ghi nếu CHƯA có — atomic, trả true/false
Cache::forget('key');
Cache::increment('views:post:42');                 // đếm atomic

// Pattern hay dùng nhất: có thì lấy, không thì tính rồi cất
$posts = Cache::remember('posts:latest', 300, fn () =>
    Post::with('author')->latest()->limit(10)->get()
);

// Laravel 13: gia hạn TTL mà không phải đọc-ghi lại giá trị
Cache::touch('posts:latest', 3600);
```

16.3 Stale-while-revalidate — `Cache::flexible()`

```
// [5, 10]: 0-5s trả cache ngay; 5-10s vẫn trả cache CŨ nhưng âm thầm tính lại sau khi trả response;
// quá 10s mới bắt user chờ tính lại
$stats = Cache::flexible('dashboard:stats', [300, 600], fn () => $this->computeStats());
```

Giải quyết vấn đề "cache hết hạn đúng lúc 100 request cùng tới → 100 request cùng tính lại" (cache stampede). Việc tính lại chạy trong deferred function sau khi response đã gửi.

16.4 Xoá cache đúng lúc (cache invalidation)

Câu hỏi khó nhất của cache không phải "cất thế nào" mà "xoá khi nào". Ba chiến lược, kết hợp với Observer (Part 15):

```
// 1) TTL ngắn — chấp nhận dữ liệu trễ vài phút, đơn giản nhất
Cache::remember('posts:latest', 60, ...);

// 2) Xoá chủ động khi dữ liệu đổi — trong PostObserver::saved / deleted
Cache::forget('posts:latest');
Cache::forget("post:{$post->id}");

// 3) Tags — xoá cả nhóm (chỉ redis/memcached)
Cache::tags(['posts'])->put("post:{$post->id}", $post, 3600);
Cache::tags(['posts'])->flush();       // xoá mọi key gắn tag 'posts'
```

Đừng cache quá sớm. Cache là lớp phức tạp thêm (bug "sửa rồi mà vẫn thấy cũ" là kinh điển). Thứ tự đúng: (1) sửa N+1 và thêm index, (2) đo bằng Telescope/Pulse, (3) chỉ cache chỗ thật sự nóng. Và **không bao giờ** cache dữ liệu theo user mà không đưa user id vào key.

16.5 Atomic lock — chống race condition

Tình huống: 2 request cùng lúc "thanh toán đơn #42". Cả 2 đọc `status = pending`, cả 2 chuyển sang `paid`, trừ tiền 2 lần. Transaction (Part 13) không đủ nếu logic có gọi API ngoài. Cần khoá:

```
$lock = Cache::lock("order:{$order->id}:pay", 10);     // khoá tối đa 10s

if ($lock->get()) {                                     // lấy được khoá → xử lý
    try {
        $this->charge($order);
    } finally {
        $lock->release();
    }
} else {
    throw new ConflictException('Đơn hàng đang được xử lý.');   // 409
}

// Gọn hơn: closure tự release
Cache::lock("order:{$order->id}:pay", 10)->get(fn () => $this->charge($order));

// Chờ tối đa 5s để lấy khoá thay vì từ chối ngay
Cache::lock('report', 10)->block(5, fn () => $this->generate());

// Laravel 13: giới hạn đồng thời — tối đa 3 tiến trình cùng chạy
Cache::funnel('image-processing')->limit(3)->releaseAfter(60)->block(10)
    ->then(fn () => $this->process(), fn () => throw new TooManyRequestsHttpException());
```

Job middleware `WithoutOverlapping` (Part 14.7) và `Schedule::withoutOverlapping()` (Part 17) đều dùng chính cơ chế lock này bên dưới. Lock cần store hỗ trợ (database/redis/memcached/file/array) và phải là store **chung** giữa mọi server.

16.6 Memo — cache trong 1 request

```
Cache::memo()->get('settings');   // lần 1 đọc từ store; các lần sau trong CÙNG request đọc từ RAM
```

Giải quyết việc gọi `Cache::get('settings')` 50 lần trong 1 request (50 round-trip Redis) — `memo()` ghi nhớ trong bộ nhớ process cho tới hết request/job.

Việc cần làm:

1. Cache endpoint `GET /api/posts` (trang 1, không filter) bằng `Cache::remember` 60s; đo thời gian response trước/sau bằng middleware `LogRequests` (Part 07)
2. Xoá cache đó trong `PostObserver` khi post được tạo/sửa/xoá — chứng minh "tạo post mới → GET thấy ngay"
3. Đổi `Cache::remember` thành `Cache::flexible([60, 300], ...)`, đọc log để thấy việc tính lại chạy sau response

4. Viết endpoint `POST /api/posts/{post}/publish` dùng `Cache::lock`; gọi 2 lần song song bằng `curl &` → 1 thành công, 1 nhận 409
5. Test: dùng `Cache::spy()` hoặc `config(['cache.default' => 'array'])` để assert key được ghi/xoá đúng

PART 17 — MỚI

Task Scheduling (cron)

Mục tiêu: Định nghĩa mọi việc định kỳ (dọn token, gửi digest, prune job fail) ngay trong code — server chỉ cần đúng 1 dòng cron.

17.1 Khai báo lịch — `routes/console.php`

```
// routes/console.php (Laravel 11+ — không còn app/Console/Kernel.php)
use Illuminate\Support\Facades\Schedule;

Schedule::command('sanctum:prune-expired --hours=24')->daily();          // Part 09
Schedule::command('queue:prune-failed --hours=48')->daily();             // Part 14
Schedule::command('queue:prune-batches --hours=48')->daily();
Schedule::command('posts:export')->dailyAt('02:00')->timezone('Asia/Ho_Chi_Minh');

Schedule::job(new SendWeeklyDigest)->weeklyOn(1, '08:00');               // đẩy job vào
queue (Part 14)

Schedule::call(fn () => Post::where('published_at', '<', now()->subYear())->delete())
    ->name('purge-old-posts')      // BẮT BUỘC đặt name cho closure nếu dùng onOneServer
    ->daily()
    ->onOneServer();
```

Đã đối chiếu docs Scheduling 13.x: lịch định nghĩa trong `routes/console.php` bằng facade `Schedule`; nếu muốn tách riêng, dùng `->withSchedule(function (Schedule $schedule) {...})` trong `bootstrap/app.php`. Xem toàn bộ lịch + lần chạy kế tiếp: `php artisan schedule:list`.

17.2 Tần suất & ràng buộc hay dùng

Method	Ý nghĩa
`everyMinute()` / `everyFiveMinutes()` / `hourly()`	Theo phút / giờ
`daily()` / `dailyAt('13:00')` / `twiceDaily(1, 13)`	Theo ngày
`weeklyOn(1, '8:00')` / `monthlyOn(4, '15:00')`	Thứ 2 hàng tuần / ngày 4 hàng tháng
`cron('0 */6 * * *')`	Biểu thức cron tuỳ ý
`everyTenSeconds()`	Sub-minute: `schedule:run` sẽ chạy hết phút đó thay vì thoát ngay

`weekdays()` / `between('8:00','17:00')` / `when(fn)` / `environments(['production'])`	Ràng buộc thêm
`timezone('Asia/Ho_Chi_Minh')`	Múi giờ riêng cho task (hoặc `schedule_timezone` trong `config/app.php`)

17.3 Ba flag không được quên ở production

```
Schedule::command('reports:build')
    ->hourly()
    ->withoutOverlapping()     // task giờ trước chưa xong → bỏ qua lần này (dùng cache lock, Part 16)
    ->onOneServer()            // nhiều server cùng chạy cron → chỉ 1 server thực thi
    ->runInBackground();       // task dài không chặn task khác cùng thời điểm (chỉ command/exec)
```

`onOneServer()` cần cache driver dùng chung (database/redis/memcached) — đây là lý do Part 16 nằm trước Part 17.

17.4 Chạy scheduler

```
# Server: đúng 1 dòng crontab, chạy mỗi phút — Laravel tự quyết task nào tới giờ
* * * * * cd /path-to-your-project && php artisan schedule:run >> /dev/null 2>&1

# Local: không cài cron, chạy foreground
php artisan schedule:work

# Deploy: nếu có sub-minute task, ngắt schedule:run đang chạy để nó nạp code mới
php artisan schedule:interrupt
```

Lỗi hay gặp: "lịch không chạy" — 95% là chưa thêm dòng crontab (hoặc thêm với user khác không có quyền đọc `.env`), hoặc `schedule:work` đã tắt. `schedule:list` chỉ cho biết lịch **được định nghĩa**, không cho biết cron có đang gọi hay không — kiểm tra bằng `->onSuccess(fn () => Log::info(...))`.

17.5 Theo dõi kết quả

```
Schedule::command('posts:export')->daily()
    ->appendOutputTo(storage_path('logs/export.log'))
    ->emailOutputOnFailure('dev@example.com')
    ->onFailure(fn (Stringable $output) => Log::error("export failed: {$output}"))
    ->pingOnSuccess($healthcheckUrl);     // báo cho dịch vụ giám sát (vd healthchecks.io)
```

Việc cần làm:

1. Đưa 3 lệnh prune (sanctum, failed jobs, batches) vào lịch `daily()`; chạy `schedule:list`
2. Viết Artisan command `posts:publish-scheduled` (publish các post có `scheduled_at <= now()`), lịch `everyMinute()->withoutOverlapping()`
3. Chạy `php artisan schedule:work`, tạo post có `scheduled_at` 2 phút tới, xác nhận nó tự publish

4. Thêm `->onFailure()` ghi log và cố tình làm command lỗi để thấy log
5. Test command bằng `$this->artisan('posts:publish-scheduled')->assertSuccessful()` + `travelTo()` để giả lập thời gian

PART 18 — MỚI

File Storage & Upload

Mục tiêu: Nhận file upload an toàn (validate, đặt tên, lưu ngoài `public/`), phục vụ file qua URL, và đổi từ ổ đĩa local sang S3 mà không sửa code.

18.1 Disk — trừu tượng hoá nơi lưu

Disk (config/filesystems.php)	Lưu ở đâu	Dùng cho
`local`	`storage/app/private`	File riêng tư (hoá đơn, tài liệu) — không truy cập được qua URL
`public`	`storage/app/public` → symlink `public/storage`	File công khai (ảnh bìa, avatar)
`s3`	Amazon S3 / mọi dịch vụ S3-compatible (MinIO, R2, Spaces)	Production nhiều server

```
php artisan storage:link      # tạo symlink public/storage → storage/app/public (chạy 1 lần, cả trên server)
```

Quy tắc bảo mật: file người dùng upload KHÔNG BAO GIỜ ghi thẳng vào `public/` (kẻ tấn công upload `shell.php` rồi gọi qua URL = chiếm server). Luôn đi qua disk; disk `public` cũng chỉ là symlink tới `storage/app/public` và PHP ở đó không được thực thi bởi web server nếu cấu hình đúng.

18.2 Nhận upload

```
// Form Request
public function rules(): array
{
    return [
        'cover' => ['required', File::image()->max(2 * 1024)],   // ảnh, tối đa 2MB (KB)
        // hoặc: ['required', 'file', 'mimes:jpg,png,webp', 'max:2048']
    ];
}

// Controller
public function uploadCover(UploadCoverRequest $request, Post $post)
{
    // store(): tự sinh tên file duy nhất, đuôi theo MIME thật (không tin đuôi client gửi)
    $path = $request->file('cover')->store('covers', 'public');     // →
"covers/a1b2c3.jpg"

    // storeAs(): tự đặt tên
    $path = $request->file('cover')->storeAs('covers', "post-{$post->id}.jpg", 'public');

    $post->update(['cover_path' => $path]);        // lưu ĐƯỜNG DẪN tương đối, không lưu
URL

    ProcessPost::dispatch($post);                  // resize/optimize ở worker (Part 14)

    return ['url' => Storage::disk('public')->url($path)];
}
```

> **Đã đối chiếu docs Filesystem 13.x:** `store($dir, $disk)` / `storeAs($dir, $name, $disk)` trên `UploadedFile`, hoặc `Storage::putFile($dir, $file)` / `putFileAs`. Tên file tự sinh là ID duy nhất; phần mở rộng lấy từ MIME thực tế. Đường dẫn được chuẩn hoá (loại ký tự lạ) — nhưng vẫn nên tự sanitize tên nếu dùng `storeAs` với tên do client cung cấp.

18.3 Đọc, tải, xoá, URL tạm

```
Storage::disk('public')->url($path);                       // URL công khai
Storage::disk('local')->download($path, 'invoice.pdf');    // response tải file riêng tư
(qua controller có auth)
Storage::disk('s3')->temporaryUrl($path, now()->addMinutes(5));   // link S3 có hạn —
không cần đi qua PHP
Storage::disk('public')->exists($path);
Storage::disk('public')->delete($path);                    // nhớ gọi trong
PostObserver::deleted
```

Đổi `FILESYSTEM_DISK=s3` + biến AWS trong `.env` → toàn bộ code trên chạy với S3 mà không sửa gì. Đây là giá trị của tầng trừu tượng Flysystem.

18.4 Test upload không cần đĩa thật

```
test('upload cover cho post', function () {
    Storage::fake('public');                                    // disk giả trong bộ nhớ
tạm
    Queue::fake();
    Sanctum::actingAs($user = User::factory()->create());
    $post = Post::factory()->for($user)->create();

    $this->postJson("/api/v1/posts/{$post->id}/cover", [
        'cover' => UploadedFile::fake()->image('cover.jpg', 800, 600),
    ])->assertOk();

    Storage::disk('public')->assertExists($post->fresh()->cover_path);
    Queue::assertPushed(ProcessPost::class);
});

test('từ chối file không phải ảnh', function () {
    $this->postJson(..., ['cover' => UploadedFile::fake()->create('virus.php', 10)])
        ->assertStatus(422);
});
```

Việc cần làm:

1. Chạy `storage:link`; thêm cột `cover_path` vào posts
2. Endpoint `POST /api/v1/posts/{post}/cover` với Form Request validate ảnh ≤ 2MB, lưu vào disk `public`, chỉ tác giả được upload (Policy — Part 19)
3. Resource trả thêm `cover_url` (dùng `Storage::url`), null nếu chưa có
4. Xoá file trong `PostObserver::deleted`
5. (Tuỳ chọn) Cài MinIO bằng Docker, đổi disk sang `s3`, xác nhận code không đổi
6. Viết 2 test như 18.4

PART 19 — MỚI

Authorization chuyên sâu: Gate & Policy

Mục tiêu: Tập trung mọi câu hỏi "ai được làm gì" vào một chỗ (Policy), không rải `if ($user->id === ...)` khắp controller; biết trả 403 hay 404 cho đúng; và biết dùng Form Request `authorize()` đúng cách.

19.1 Gate vs Policy

	Gate	Policy
Hình thức	Closure đặt tên, khai trong `AppServiceProvider::boot()`	Class gom logic quanh 1 model

Dùng cho	Quyền không gắn model: "xem dashboard admin", "xuất báo cáo"	Quyền trên bản ghi: sửa/xoá/xem post này
Ví dụ	`Gate::define('view-dashboard', fn (User $u) => $u->is_admin)`	`PostPolicy::update(User $u, Post $p)`

Docs ví Gate/Policy như Route/Controller: bắt đầu bằng Gate để hiểu, nhưng app thật nên dùng Policy.

19.2 Viết Policy đầy đủ

```
php artisan make:policy PostPolicy --model=Post    # sinh sẵn viewAny/view/create/update/delete/restore/forceDelete
```

```
namespace App\Policies;

use Illuminate\Auth\Access\Response;

class PostPolicy
{
    // Chạy TRƯỚC mọi method: admin được làm tất cả; return null → rơi xuống method thường
    public function before(User $user, string $ability): ?bool
    {
        return $user->is_admin ? true : null;
    }

    public function viewAny(?User $user): bool { return true; }          // ?User: cho phép khách (guest)

    public function view(?User $user, Post $post): bool
    {
        return $post->published_at !== null || $user?->id === $post->user_id;
    }

    public function create(User $user): bool { return $user->hasVerifiedEmail(); }

    public function update(User $user, Post $post): Response
    {
        return $user->id === $post->user_id
            ? Response::allow()
            : Response::deny('Bạn không phải tác giả bài viết này.');   // message đi ra response 403
    }

    public function delete(User $user, Post $post): Response
    {
        return $user->id === $post->user_id
            ? Response::allow()
            : Response::denyAsNotFound();     // trả 404 thay vì 403: không tiết lộ bài viết tồn tại
    }
}
```

Đã đối chiếu docs Authorization 13.x: Policy được **tự động phát hiện** khi tên khớp quy ước (`Post` → `PostPolicy` trong `app/Policies` hoặc `app/Models/Policies`); không cần đăng ký. Nếu tên không theo quy ước: `#[UsePolicy(PostPolicy::class)]` trên model hoặc `Gate::policy(Post::class, PostPolicy::class)` . Khách (chưa đăng nhập) mặc định luôn bị từ chối — muốn cho phép phải khai `?User $user` .

19.3 Gọi Policy — 4 cách, dùng nhất quán

```
// 1) Attribute trên controller (Laravel 13) – gọn nhất, chạy như middleware 'can'
use Illuminate\Routing\Attributes\Controllers\Authorize;

#[Authorize('update', 'post')]                  // 'post' = tên route parameter
public function update(UpdatePostRequest $request, Post $post) { }

#[Authorize('create', Post::class)]             // không có model instance
public function store(StorePostRequest $request) { }

#[Authorize('create', [Comment::class, 'post'])] // policy nhận thêm tham số (post cha)
public function storeComment(Post $post) { }

// 2) Route
Route::put('/posts/{post}', [PostController::class, 'update'])->can('update', 'post');

// 3) Trong code – throw AuthorizationException → 403 (hoặc 404 nếu denyAsNotFound)
Gate::authorize('update', $post);

// 4) Form Request – gộp "được phép không" và "dữ liệu hợp lệ không" vào 1 chỗ
class UpdatePostRequest extends FormRequest
{
    public function authorize(): bool
    {
        return $this->user()->can('update', $this->route('post'));
    }
}
```

Thứ tự chạy khi dùng Form Request: `authorize()` chạy TRƯỚC `rules()` . Nghĩa là user không có quyền nhận 403 ngay, không bị lộ lỗi validation. Với API, cách 1 hoặc 4 là đủ — đừng làm cả hai cho cùng 1 action (kiểm tra 2 lần vô ích).

19.4 Quyền trên danh sách — lọc thay vì từ chối

```
// Sai: lấy hết rồi kiểm tra từng cái (N lần policy, lộ số lượng)
// Đúng: giới hạn ngay từ query
public function index(Request $request)
{
    return Post::query()
        ->when(! $request->user()?->is_admin, fn ($q) =>
            $q->whereNotNull('published_at')->orWhere('user_id', $request->user()?->id))
        ->paginate();
}
```

19.5 Test policy

```
test('tác giả sửa được post của mình', function () {
    $author = User::factory()->create();
    $post = Post::factory()->for($author)->create();

    expect($author->can('update', $post))->toBeTrue();
    expect(User::factory()->create()->can('update', $post))->toBeFalse();
});

test('user khác xoá post nhận 404', function () {
    Sanctum::actingAs(User::factory()->create());
    $post = Post::factory()->create();

    $this->deleteJson("/api/v1/posts/{$post->id}")->assertNotFound();   // denyAsNotFound
});
```

Việc cần làm:

1. Hoàn thiện `PostPolicy` với `before`, `viewAny/view` (cho guest), `create`, `update` (deny có message), `delete` (`denyAsNotFound`)
2. Thêm cột `is_admin` cho users; chứng minh admin sửa được post của người khác nhờ `before()`
3. Chuyển toàn bộ kiểm tra quyền trong controller sang `#[Authorize]` (hoặc Form Request `authorize()`) — xoá hết `if ($user->id === ...)` rải rác
4. Viết `CommentPolicy`: chỉ tác giả comment hoặc tác giả post được xoá comment
5. Viết test cho 4 case: tác giả OK, người khác 403 (update), người khác 404 (delete), admin OK

PART 20 — MỚI

Mail & Notifications (queued)

Mục tiêu: Gửi thông báo qua nhiều kênh (mail, database, Slack...) bằng một class duy nhất, luôn đẩy qua queue, và test được mà không gửi mail thật.

20.1 Notification vs Mailable

Mailable = 1 email cụ thể (template, subject). **Notification** = "một thông báo" có thể đi qua nhiều kênh (`mail`, `database`, `broadcast`, `slack`, `vonage`), kênh mail bên trong vẫn dùng `MailMessage`. Với ứng dụng thật, gần như luôn viết Notification — nó bao trùm Mailable và cho user tự chọn kênh.

20.2 Viết notification

```
php artisan make:notification PostPublishedNotification
php artisan make:notifications-table && php artisan migrate    # nếu dùng kênh database
```

```php
namespace App\Notifications;

use Illuminate\Bus\Queueable;
use Illuminate\Contracts\Queue\ShouldQueue;
use Illuminate\Notifications\Messages\MailMessage;
use Illuminate\Notifications\Notification;

class PostPublishedNotification extends Notification implements ShouldQueue   // luôn
queue
{
    use Queueable;

    public function __construct(public Post $post)
    {
        $this->afterCommit();          // chờ transaction commit (Part 13.5)
    }

    public function via(object $notifiable): array
    {
        return $notifiable->prefers_email ? ['mail', 'database'] : ['database'];
    }

    public function toMail(object $notifiable): MailMessage
    {
        return (new MailMessage)
            ->subject("Bài viết \"{$this->post->title}\" đã được đăng")
            ->greeting("Chào {$notifiable->name},")
            ->line('Bài viết của bạn đã xuất bản thành công.')
            ->action('Xem bài viết', url("/posts/{$this->post->slug}"))
            ->line('Cảm ơn bạn!');
    }

    public function toArray(object $notifiable): array      // kênh database: lưu JSON này
    {
        return ['post_id' => $this->post->id, 'title' => $this->post->title];
    }
}
```

20.3 Gửi

```php
$user->notify(new PostPublishedNotification($post));                    // 1 người
(trait Notifiable)
Notification::send($followers, new PostPublishedNotification($post));     // nhiều người
Notification::route('mail', 'ops@example.com')->notify(new SystemAlert($e));  // không cần
model User

$user->notify((new PostPublishedNotification($post))->delay(now()->addMinutes(10)));
```

Kết nối với Part 15: listener `SendPostPublishedNotification` chỉ có đúng 1 dòng `notify()`. Vì notification đã `ShouldQueue`, listener có thể để đồng bộ (không cần queue 2 lớp).

20.4 Đọc notification trong app (kênh database)

```
$user->notifications;                          // tất cả (bảng notifications, cột data =
JSON toArray)
$user->unreadNotifications;                    // chưa đọc
$user->unreadNotifications->markAsRead();

// API: GET /api/v1/notifications → NotificationResource::collection($request->user()-
>notifications()->paginate())
```

20.5 Local: xem mail mà không gửi thật

Đặt `MAIL_MAILER=log` (mail ghi vào `storage/logs/laravel.log`) hoặc dùng Mailpit (Docker, có giao diện web xem mail). Production dùng SMTP/Resend/Postmark/SES qua `config/mail.php`.

20.6 Test

```
test('publish gửi notification cho tác giả', function () {
    Notification::fake();

    app(PostService::class)->publish($author, $data);

    Notification::assertSentTo($author, PostPublishedNotification::class,
        fn ($n, array $channels) => in_array('mail', $channels) && $n->post->title ===
'Hello');
    Notification::assertCount(1);
});
```

Việc cần làm:

1. Tạo `PostPublishedNotification` (mail + database), `ShouldQueue`, `afterCommit()`
2. Gửi từ listener Part 15; chạy worker và xem mail trong log / Mailpit
3. Endpoint `GET /api/v1/notifications` + `POST /api/v1/notifications/{id}/read`
4. Thêm cột `prefers_email` cho users, `via()` theo cột đó
5. Test với `Notification::fake()` cho cả 2 nhánh của `via()`

PART 21 — MỚI

Kiến trúc dự án lớn: Service, Action, DTO, Enum

Mục tiêu: Biết đặt logic ở đâu khi controller và model bắt đầu phình. Không có "kiến trúc đúng duy nhất" — mục tiêu là code dễ tìm, dễ test, dễ đổi.

21.1 Bản đồ "cái gì ở đâu"

Loại logic	Đặt ở	Ví dụ
Nhận request, trả response	Controller (mỏng)	`PostController::store` — 3 dòng
Hợp lệ hoá input + quyền	Form Request + Policy	`StorePostRequest`, `PostPolicy`
Định dạng output	API Resource	`PostResource`
Một nghiệp vụ hoàn chỉnh (nhiều bước)	Action class (1 class = 1 việc) hoặc Service	`PublishPostAction`
Truy vấn phức tạp, tái dùng	Query scope (`#[Scope]`) hoặc Query class	`Post::published()->popular()`
Phản ứng phụ (mail, log, index)	Event + Listener / Observer	Part 15
Việc chậm	Job	Part 14
Dữ liệu truyền giữa các lớp	DTO (readonly class) / Enum	`PublishPostData`, `PostStatus`
Vòng đời model (slug, cache)	Observer	`PostObserver`

21.2 Action class — đơn vị nghiệp vụ test được

```php
namespace App\Actions\Posts;

final class PublishPostAction
{
    public function __construct(
        private readonly ImageOptimizer $images,   // container inject như mọi class khác
    ) {}

    public function handle(User $author, PublishPostData $data): Post
    {
        return DB::transaction(function () use ($author, $data) {
            $post = $author->posts()->create($data->toArray());
            ProcessPost::dispatch($post)->afterCommit();
            PostPublished::dispatch($post);                // ShouldDispatchAfterCommit
            return $post;
        });
    }
}

// Controller
public function store(StorePostRequest $request, PublishPostAction $action)
{
    $post = $action->handle($request->user(), PublishPostData::fromRequest($request));
    return $post->toResource()->response()->setStatusCode(202);
}
```

Action vs Service: Service = 1 class nhiều method quanh 1 chủ đề (`PostService`); Action = 1 class 1 method (`PublishPostAction`). Action dễ test và dễ đặt tên hơn khi nghiệp vụ nhiều; Service đủ tốt khi ít. Cùng một Action có thể được gọi từ Controller, Artisan command, Job — đó là mục đích: tách nghiệp vụ khỏi cổng vào.

21.3 DTO bằng readonly class

```php
namespace App\Data;

final readonly class PublishPostData
{
    public function __construct(
        public string $title,
        public string $body,
        public PostStatus $status = PostStatus::Draft,
        public ?\DateTimeImmutable $scheduledAt = null,
    ) {}

    public static function fromRequest(StorePostRequest $request): self
    {
        return new self(
            title: $request->validated('title'),
            body: $request->validated('body'),
            status: PostStatus::from($request->validated('status', 'draft')),
            scheduledAt: $request->date('scheduled_at')?->toImmutable(),
        );
    }

    public function toArray(): array
    {
        return [
            'title'        => $this->title,
            'body'         => $this->body,
            'status'       => $this->status,
            'scheduled_at' => $this->scheduledAt,
        ];
    }
}
```

Lợi ích so với mảng `$request->validated()`: IDE gợi ý được field, sai tên field là lỗi ngay, kiểu dữ liệu chắc chắn (enum, DateTime chứ không phải string). Đây chính là vai trò DTO bên NestJS — nhưng tách khỏi validation (validation nằm ở Form Request).

21.4 Enum cho trạng thái — hết "magic string"

```
enum PostStatus: string
{
    case Draft = 'draft';
    case Scheduled = 'scheduled';
    case Published = 'published';

    public function canTransitionTo(self $next): bool
    {
        return match($this) {
            self::Draft     => in_array($next, [self::Scheduled, self::Published]),
            self::Scheduled => $next === self::Published,
            self::Published => false,
        };
    }
}

// Model: cast → đọc ra enum, ghi vào string
protected function casts(): array { return ['status' => PostStatus::class]; }

// Validation: Rule::enum(PostStatus::class)
// Query: Post::where('status', PostStatus::Published)
// Route binding: /posts/status/{status} với type-hint PostStatus $status → tự ép, sai →
404
```

21.5 Query scope — đặt tên cho truy vấn

```
use Illuminate\Database\Eloquent\Attributes\Scope;

class Post extends Model
{
    #[Scope]
    protected function published(Builder $query): void
    {
        $query->whereNotNull('published_at')->where('published_at', '<=', now());
    }

    #[Scope]
    protected function popular(Builder $query, int $min = 100): void
    {
        $query->where('views', '>=', $min);
    }
}

Post::published()->popular(500)->latest()->paginate();
```

> **Đã đối chiếu docs Eloquent 13.x:** `#[Scope]` trên method `protected` là cách mới; cách cũ `public function scopePublished(Builder $query)` vẫn chạy. Global scope (áp mọi query) dùng `#[ScopedBy([TenantScope::class])]`.

21.6 Cấu trúc thư mục gợi ý cho app vừa & lớn

```
app/
├── Actions/Posts/PublishPostAction.php     ← nghiệp vụ
├── Data/PublishPostData.php                ← DTO
├── Enums/PostStatus.php
├── Events/  Listeners/  Jobs/  Notifications/  Observers/  Policies/
├── Http/Controllers/  Requests/  Resources/  Middleware/
├── Models/
├── Services/                                ← tích hợp bên ngoài (PaymentGateway,
ImageOptimizer)
└── Support/                                 ← helper thuần, không phụ thuộc framework

Dự án rất lớn → nhóm theo domain: app/Domain/Posts/{Actions,Data,Models,Policies...}
(nhớ withEvents(discover: [...]) để listener ngoài app/Listeners vẫn được tìm thấy — Part
15)
```

21.7 Công cụ giữ chất lượng code

Công cụ	Làm gì	Lệnh
Laravel Pint	Format code theo PSR-12 (có sẵn)	`./vendor/bin/pint`
Larastan (PHPStan)	Phân tích tĩnh: bắt lỗi kiểu, method không tồn tại — trước khi chạy	`composer require --dev larastan/larastan` → `./vendor/bin/phpstan analyse`
Pest	Test (Part 11) — mục tiêu: mọi Action/Policy/Job có test	`php artisan test --parallel`
Rector	Tự nâng cú pháp PHP/Laravel mới (vd chuyển `$fillable` → attribute)	`./vendor/bin/rector`

Việc cần làm:

1. Tách `PostService::publish()` thành `PublishPostAction` + `PublishPostData` (readonly DTO)
2. Thêm `PostStatus` enum với `canTransitionTo()`; ném exception 409 (Part 07) khi chuyển trạng thái sai
3. Viết scope `published()` bằng `#[Scope]`, dùng ở index và feed
4. Cài Larastan, chạy level 5, sửa hết lỗi báo
5. Viết unit test cho `PublishPostAction` (không qua HTTP) và cho `PostStatus::canTransitionTo`

PART 22 — MỚI

Production: deploy, worker, Octane, quan sát hệ thống

Mục tiêu: Hiểu một app Laravel chạy thật cần những process nào, deploy sao cho không mất job, và nhìn thấy được chuyện gì đang xảy ra bên trong.

Docker ở local: nếu bạn dùng **Laravel Sail** (PART SAIL), mỗi loại process ở 22.1 chính là một container chạy cùng một image ứng dụng; các lệnh dưới đây viết cho máy thật, thêm tiền tố `sail` khi chạy trong Sail. Phần bảo mật tổng hợp: **PART SEC**.

22.1 Một app Laravel production gồm 3 loại process

```
Nginx ──► PHP-FPM (nhiều worker)   : xử lý HTTP request          ← Part 00
          php artisan queue:work   : xử lý job (Supervisor giữ sống) ← Part 14
          cron → schedule:run      : mỗi phút, chạy task tới giờ     ← Part 17
          Redis                    : cache + queue + lock            ← Part 14, 16
          MySQL/PostgreSQL         : dữ liệu
```

Thiếu bất kỳ mảnh nào: web vẫn lên nhưng mail không gửi, lịch không chạy, lock không hoạt động — lỗi "âm thầm" khó thấy nhất.

22.2 Supervisor — giữ worker luôn sống

```
; /etc/supervisor/conf.d/laravel-worker.conf
[program:laravel-worker]
process_name=%(program_name)s_%(process_num)02d
command=php /var/www/app/artisan queue:work redis --queue=media,default --tries=3 --
timeout=90 --max-time=3600
autostart=true
autorestart=true
stopasgroup=true
killasgroup=true
user=www-data
numprocs=4                      ; 4 worker song song
redirect_stderr=true
stdout_logfile=/var/www/app/storage/logs/worker.log
stopwaitsecs=3600               ; đợi job dài chạy xong khi stop
```

```
sudo supervisorctl reread && sudo supervisorctl update
sudo supervisorctl start "laravel-worker:*"
```

Theo docs Queues: `stopwaitsecs` phải lớn hơn thời gian chạy của job dài nhất, nếu không Supervisor kill job giữa chừng khi restart. `--max-time` cho worker tự thoát định kỳ (Supervisor khởi động lại) — chống rò rỉ bộ nhớ ở process sống lâu.

22.3 Script deploy chuẩn

```
php artisan down --retry=30                    # (tuỳ chọn) maintenance mode
git pull
composer install --no-dev --optimize-autoloader --no-interaction
php artisan migrate --force                    # --force: không hỏi xác nhận ở production
php artisan optimize                           # config + route + view + event cache
php artisan queue:restart                      # worker nạp code mới (Supervisor tự start
lại)
php artisan schedule:interrupt                 # nếu có sub-minute task
php artisan up
```

Thứ tự quan trọng: `migrate` trước `queue:restart` — worker mới phải chạy trên schema mới. Và luôn `optimize:clear` khi debug "sửa config mà không ăn".

22.4 Horizon — dashboard cho queue Redis

`composer require laravel/horizon` → `php artisan horizon:install`. Horizon thay `queue:work` bằng `php artisan horizon` (1 process quản lý nhiều worker, tự cân bằng theo tải), kèm dashboard `/horizon`: job đang chạy, fail, thời gian chờ, throughput. Chỉ dùng với driver `redis`. Với app có queue nghiêm túc, đây là mặc định.

22.5 Octane — khi PHP-FPM không đủ nhanh

`composer require laravel/octane` → chạy app bằng Swoole/FrankenPHP/RoadRunner: app **boot 1 lần rồi giữ trong RAM**, mỗi request không phải nạp lại framework → nhanh gấp nhiều lần. Đổi lại, mô hình P.1 thay đổi: singleton sống qua nhiều request, biến static "dính" giữa các request. Vì thế Part 03 nhấn mạnh `scoped()` và Octane docs có danh sách "dependency injection pitfalls". Chỉ cân nhắc Octane khi đã đo và biết bottleneck là bootstrap chứ không phải DB.

22.6 Quan sát hệ thống

Công cụ	Cho biết	Dùng ở
Telescope	Từng request: query (kèm N+1), job, mail, exception, cache hit/miss	Local / staging (nặng, không nên bật production)
Pulse	Tổng quan production: request chậm, job chậm, query chậm, server load	Production (nhẹ)
Nightwatch	Dịch vụ giám sát chính chủ (trace, log, alert)	Production, có phí
`storage/logs/laravel.log` + `LOG_STACK`	Log — cấu hình `config/logging.php`, gửi Slack/Sentry khi error	Mọi nơi
`/up`	Health check cho load balancer	Production

22.7 Checklist bảo mật & cấu hình

- `APP_DEBUG=false`, `APP_KEY` riêng, `.env` không commit, document root = `public/`
- `SESSION_SECURE_COOKIE=true`, HTTPS bắt buộc, `TrustProxies` cấu hình đúng khi sau load balancer (nếu không, `$request->ip()` là IP của LB → rate limit sai)
- Rate limit (Part 10), CORS whitelist, validate mọi input (Part 05), policy mọi hành động ghi (Part 19)
- Backup DB có kiểm tra restore; `php artisan db:monitor` cảnh báo connection
- Không dùng `DB::raw` / `whereRaw` với input người dùng; không `unprepared()`

Việc cần làm:

1. Cài Telescope ở local, mở `/telescope`, xem lại request index + job `ProcessPost` + mail — đối chiếu số query với Part 08
2. Viết `deploy.sh` theo 22.3; chạy thử trên máy local với `APP_ENV=production`, `APP_DEBUG=false`
3. Cài Supervisor (hoặc Docker) chạy 2 worker; kill 1 worker giữa chừng và chứng minh job được retry bởi worker còn lại
4. (Tuỳ chọn) Cài Redis + Horizon, chuyển `QUEUE_CONNECTION=redis`, xem dashboard khi chạy batch export
5. Viết README mục "Vận hành": các process cần chạy, lệnh deploy, chỗ xem log

---

PART 23 — MỚI

Lộ trình master Laravel — làm gì tiếp theo

Mục tiêu: Sau khi xong Part PHP–22, bạn đã vượt mức "biết dùng". Phần này là bản đồ để đi tiếp tới mức thiết kế được hệ thống và đọc được mã nguồn framework.

23.1 Ba tầng năng lực

Tầng	Bạn làm được gì	Tài liệu này phủ
1. Dùng được	CRUD API có auth, validate, resource, test cơ bản	Part PHP–11
2. Làm thật được	Việc nền, event, cache, lịch, file, quyền, thông báo; deploy và vận hành; kiến trúc code khi app lớn	Part 12–22
3. Master	Thiết kế hệ thống nhiều dịch vụ, tối ưu ở tầng DB/Redis, đọc & mở rộng mã nguồn framework, viết package, mentor người khác	Lộ trình dưới đây

23.2 Chủ đề tiếp theo, theo thứ tự nên học

Chủ đề	Vì sao	Docs

Query Builder nâng cao: subquery, `whereExists`, window function, index & EXPLAIN	90% vấn đề hiệu năng thật nằm ở DB, không phải PHP	queries, eloquent-relationships (subquery selects)
Giao diện: Blade, Livewire, Inertia, Vite, starter kit	Tài liệu này chỉ xây API. Nếu muốn làm cả giao diện thì đây là các hướng chính thức của Laravel	blade, frontend, vite, starter-kits; Livewire và Inertia (package cộng đồng được docs giới thiệu)
Broadcasting + Reverb + Echo	Realtime (thông báo, chat) — WebSocket server chính chủ	broadcasting, reverb
Sanctum SPA auth / Passport / Socialite	Cookie-based auth cho SPA cùng domain; OAuth2; đăng nhập Google/GitHub	sanctum, passport, socialite
Multi-tenancy, global scope, database per tenant	SaaS — bài toán kiến trúc kinh điển	eloquent (global scopes), package spatie/laravel-multitenancy
Laravel AI SDK + vector search (Laravel 13)	Agent, embedding, `whereVectorSimilarTo` với pgvector — tính năng mới nhất	ai-sdk, search
Scout (full-text search với Meilisearch/Typesense)	Tìm kiếm thật, thay `LIKE '%...%'`	scout
Concurrency, Context, Process facade, HTTP client	Chạy song song, log có ngữ cảnh, gọi lệnh hệ thống, gọi API ngoài có retry	concurrency, context, processes, http-client
Pennant (feature flags), Cashier (thanh toán)	Bật tắt tính năng theo user; Stripe/Paddle	pennant, billing
Viết package riêng (Service Provider, publish config, testbench)	Hiểu framework từ phía "người mở rộng"	packages

23.3 Dự án nên tự xây để lên tầng 3

1. **Blog API hoàn chỉnh** (chính dự án này) — thêm tag (belongsToMany), search (Scout), realtime comment (Reverb), soft delete + restore, versioning v2.
2. **Hệ thống đặt hàng** — giỏ hàng, thanh toán (Cashier), tồn kho với atomic lock, email hoá đơn PDF qua queue, báo cáo doanh thu theo lịch. Đây là bài luyện transaction + lock + queue nặng nhất.
3. **SaaS multi-tenant nhỏ** — mỗi công ty 1 workspace, global scope theo tenant, invite user qua signed URL, feature flag bằng Pennant.
4. **Một package Composer** — ví dụ `your-vendor/laravel-audit-log`: Observer ghi mọi thay đổi model, có config publish, có test bằng Orchestra Testbench, publish lên Packagist.

23.4 Thói quen của người master

- **Đọc mã nguồn framework** khi thắc mắc: `vendor/laravel/framework/src/Illuminate/`. Bắt đầu từ `Foundation/Application.php`, `Container/Container.php`, `Routing/Router.php`, `Queue/Worker.php`. Mọi "phép thuật" đều là PHP thường trong đó.
- **Đo trước khi tối ưu**: Telescope/Pulse → tìm query chậm → EXPLAIN → index → rồi mới cache.

- **Test là mặc định**: mỗi Action/Policy/Job/Listener có test; CI chạy `pint --test`, `phpstan`, `pest`.
- **Theo dõi release**: `laravel.com/docs/releases` mỗi Q1, upgrade guide, Laravel News, Laracasts (Laravel From Scratch, Learn Laravel Queues), tài khoản GitHub `laravel/framework` (đọc PR để biết tính năng mới hình thành thế nào).
- **Viết lại tài liệu này bằng lời của bạn** sau khi xong — dạy lại là cách kiểm tra hiểu chắc nhất.

23.5 Checklist tự đánh giá "đã master chưa"

Câu hỏi	Trả lời được không?
Vì sao singleton trong PHP-FPM chỉ sống 1 request, và Octane thay đổi điều đó thế nào?	
Job dispatch trong transaction có thể chạy trước khi commit — tại sao, và 3 cách xử lý?	
Khi nào `cursorPaginate` thay `paginate`? Ràng buộc của nó?	
`timeout` của job và `retry_after` của queue liên quan gì nhau?	
Vì sao `Post::query()->update()` không bắn model event?	
Cache stampede là gì, `Cache::flexible` giải quyết thế nào?	
`onOneServer()` hoạt động nhờ cơ chế nào, cần điều kiện gì?	
Vì sao `env()` trả null sau `config:cache`?	
Deploy xong mà mail cũ vẫn gửi bằng template cũ — nguyên nhân và lệnh sửa?	
Khác nhau giữa 401 và 403; giữa `Response::deny()` và `denyAsNotFound()`?	

Trả lời trôi chảy cả 10 câu bằng lời của bạn, kèm ví dụ từ dự án đã làm — bạn đã ở tầng 3.